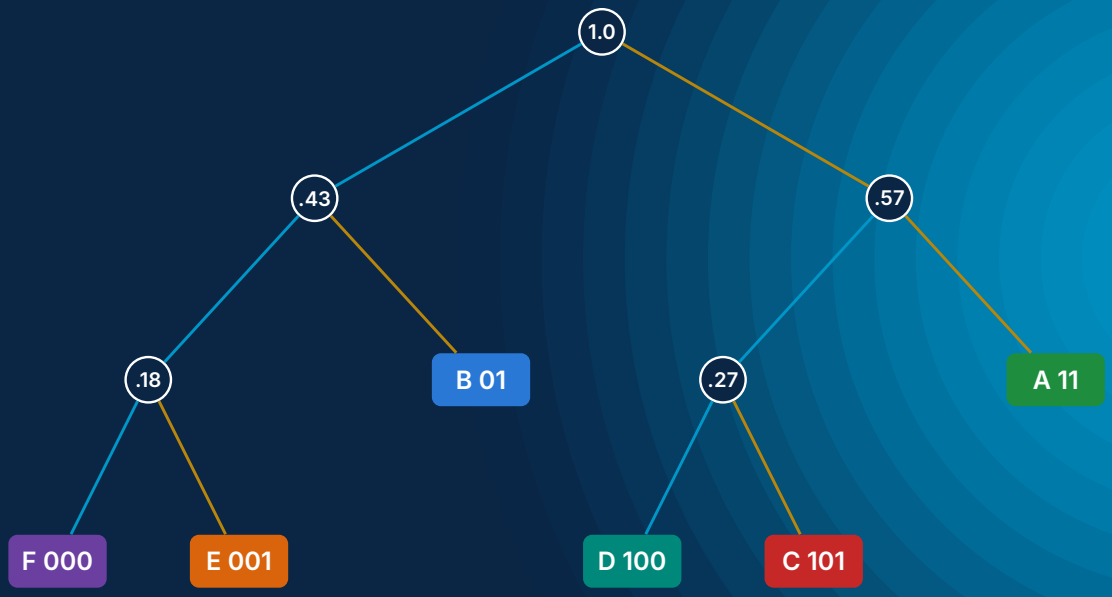


H = 2.42 bits/symbol L = 2.45 bits/symbol efficiency 98.9%

FROM FUNDAMENTALS TO ADVANCED APPLICATIONS

INFORMATION THEORY & COMPRESSION ALGORITHMS

A Complete Illustrated Guide with Mathematical Derivations, Solved Problems, Block Diagrams, Python and MATLAB Implementations, and Cybersecurity Applications

Entropy • Leakage • Huffman • Arithmetic • LZW • Markov sources • Anomaly detection • Channel capacity • Hamming • BCH

Mahmoud Abdelaziz

Military Technical College, Cairo

Version 2 • 2026

Information Theory & Compression Algorithms: From Fundamentals to Advanced Applications

Version 2, October 2026 (adds block diagrams for every system).

Every program in this book was executed. Printed outputs are real outputs of Python 3.13 (Chapters 15–18) and of GNU Octave 8.4 for the MATLAB versions (Appendix B). Random experiments use fixed seeds and can be repeated exactly. All numerical answers in the worked examples were checked with the same code.

Typeset with XeLaTeX. Diagrams drawn with Ti*k*Z and Matplotlib.

How to Use This Book

This book is written for undergraduate students of information theory and compression, including students of cybersecurity. It starts from zero — probability is explained when it is needed — and ends at the level of a professional who can analyse leakage, design codes and write working compressors and error-correcting decoders.

Nine teaching steps

Every important idea follows the same path:

1 Intuition · 2 Analogy · 3 Picture · 4 Maths · 5 Derivation · 6 Example · 7 Python · 8 Security · 9 Practice

The colour code

Definition 0.1 • Royal blue boxes are definitions

Precise statements of new terms.

Information theory concept • Cyan boxes

Key information-theory ideas and results.

Derivation • Purple boxes

Step-by-step mathematical derivations.

Worked Example 0.1 • Green boxes are worked examples

Every number is computed in full.

Important note

Orange boxes are important notes.

! Common mistake

Red boxes are common mistakes and warnings.

Cybersecurity application

Slate boxes connect the idea to cybersecurity.

Teal boxes and code are laboratories

All code is real, executed Python 3 (MATLAB versions in Appendix B).

Course map

The nine modules of the course plan are kept in their original order and extended:

Course-plan module	Chapters in this book
1 Cybersecurity as uncertainty management	Ch. 1
2 Entropy as a measure of security strength	Ch. 2
3 Conditional entropy and information leakage	Ch. 3
4 Mutual information and attacker advantage	Ch. 4
5 Coding, redundancy and exploitable structure	Ch. 5 (source coding), 6 (Shannon–Fano), 7 (Huffman), 8 (arithmetic), 9 (LZW), 10 (comparison, compression and security)
6 Sources with memory and behavioural security	Ch. 11
7 Intrusion detection and anomaly detection	Ch. 12
8 Channels, noise and secure communication	Ch. 13
9 Error, reliability and integrity	Ch. 14 (Hamming, SECDED, (5,1) repetition, BCH(15,7))
Laboratories	Ch. 15: eleven labs
Assessment	Ch. 16 (50 solved problems), Ch. 17 (question bank), Ch. 18 (final project)

Conventions

- Logarithms are base 2 and information is measured in *bits*, unless stated otherwise. $0 \log_2 0 = 0$.
- Random variables are capital letters (X), their values small letters (x), and $p(x) = P(X = x)$.
- Codewords are written left to right; the first bit is sent first.
- Python code uses only the standard library, except for plotting (Matplotlib). The code folder `itlab` contains one file per laboratory and a unit-test file.

What is new in version 2

Sixteen new **block diagrams**, marked "**Block diagram.**" in their captions, show each system as engineers draw it: key generation, leakage measurement, the traffic-analysis attacker, the source-coding system, the Shannon–Fano, Huffman, arithmetic and LZW coders, the fair comparison of compressors, the compression side channel, learning a Markov model, the anomaly detector, the Blahut–Arimoto algorithm, the Hamming SECDED and BCH(15,7) decoders, and the layers of a secure link. The theory, the worked examples, the Python and MATLAB code and all results are unchanged.

Contents

How to Use This Book iii

I Foundations of Information Theory

1 Cybersecurity as Uncertainty Management 2

1.1 What is information theory? 2
1.2 Information versus data 3
1.3 Uncertainty and probability 3
1.4 Measuring information: self-information 3
1.5 Introduction to entropy 4
1.6 Why information has mathematical value 5
1.7 Cybersecurity as uncertainty management 5
1.8 Module summary 6

2 Entropy as a Measure of Security Strength 8

2.1 Entropy from first principles 8
2.2 Binary entropy 9
2.3 Properties of entropy 10
2.4 Entropy of sources with many symbols 10
2.5 Joint entropy: a first look 11
2.6 Entropy versus predictability 11
2.7 Cybersecurity focus: password and key strength 11
2.8 Module summary 13

3 Conditional Entropy and Information Leakage 15

3.1 Joint and marginal distributions 15
3.2 Conditional entropy 16
3.3 The chain rule 16
3.4 Entropy after observation 17
3.5 Information leakage models 17

3.6 Module summary . . . 19

4 Mutual Information and Attacker Advantage . . . 21

4.1 Mutual information: intuition . . . 21
4.2 Properties . . . 22
4.3 Kullback–Leibler divergence . . . 23
4.4 Cybersecurity focus: attacker advantage . . . 23
4.5 Limitations. . . . 24
4.6 Module summary . . . 25

II Source Coding and Data Compression

5 Introduction to Source Coding . . . 28

5.1 Why data compression is necessary . . . 28
5.2 Codes and codewords . . . 29
5.3 Unique decodability and prefix-free codes . . . 30
5.4 Measuring a code . . . 30
5.5 The Kraft–McMillan inequality . . . 31
5.6 Shannon's source coding theorem . . . 31
5.7 Module summary (5.1). . . . 32

6 Shannon–Fano Coding . . . 34

6.1 The idea . . . 34
6.2 The algorithm . . . 34
6.3 A complete example with seven symbols . . . 35
6.4 Why Shannon–Fano is not always optimal . . . 36
6.5 Python implementation . . . 37
6.6 Summary . . . 37

7 Huffman Coding . . . 39

7.1 Principles. . . . 39
7.2 The algorithm . . . 39
7.3 Worked examples . . . 40
7.4 Optimality . . . 41
7.5 Efficiency, compression ratio and complexity. . . . 42
7.6 Huffman versus Shannon–Fano . . . 42
7.7 Python implementation . . . 42
7.8 Summary . . . 43

8 Arithmetic Coding . . . 45

8.1 Why arithmetic coding? . . . 45
8.2 Cumulative tables and interval subdivision . . . 46
8.3 Example 1: a dyadic source . . . 46
8.4 Example 2: decimal probabilities . . . 46
8.5 Decoding and stopping . . . 47
8.6 Precision limits and practical implementations . . . 47
8.7 Arithmetic coding versus Huffman . . . 48
8.8 Python implementation . . . 49
8.9 Summary . . . 49

9 LZW Compression . . . 50

9.1 Dictionary-based compression . . . 50
9.2 The LZW algorithm . . . 51
9.3 Example 1: ABABABA . . . 51
9.4 Example 2: TOBEORNOTTOBEORTOBEORNOT . . . 52
9.5 Dynamic code widths and compression ratio . . . 53
9.6 Advantages, limitations and applications . . . 54
9.7 Python implementation . . . 54
9.8 Summary . . . 55

10 Comparing Compressors; Compression and Cybersecurity . . . 56

10.1 How to compare fairly . . . 56
10.2 Measured results . . . 57
10.3 Overall comparison . . . 58
10.4 Compression and cybersecurity . . . 58
10.5 Summary . . . 60

III Information Sources with Memory

11 Sources with Memory and Behavioural Security . . . 62

11.1 Memoryless sources and sources with memory . . . 62
11.2 Markov chains . . . 63
11.3 The Asymptotic Equipartition Property . . . 65
11.4 Estimating a Markov model from data . . . 66
11.5 Cybersecurity focus: behaviour as a source . . . 66
11.6 Module summary . . . 66

12 Intrusion Detection and Anomaly Detection . . . 69

12.1 Statistical anomaly detection . . . 69
12.2 Empirical and model distributions . . . 70
12.3 Three information-theoretic scores . . . 70
12.4 A worked detector . . . 71
12.5 Thresholds, false positives and false negatives . . . 71
12.6 Information-theoretic behavioural analysis in practice . . . 72
12.7 Module summary . . . 73

IV Channels, Noise and Reliable Communication

13 Channels, Noise and Secure Communication . . . 76

13.1 The communication system model . . . 76
13.2 Discrete memoryless channels . . . 77
13.3 Mutual information across a channel . . . 77
13.4 Channel capacity . . . 77
13.5 Shannon's noisy-channel coding theorem . . . 79
13.6 The data processing inequality . . . 79
13.7 Reliability versus confidentiality . . . 79
13.8 Module summary . . . 80

14 Error, Reliability and Integrity . . . 82

14.1 Error detection and correction (Chapter 9.1 of the plan) . . . 82
14.2 Hamming codes (Chapter 9.2) . . . 83
14.3 Simple multi-bit error correction (Chapter 9.3) . . . 85
14.4 Implementations and simulation (Chapter 9.4) . . . 87
14.5 Module summary . . . 87

V Practical Python Laboratories

15 Python Laboratories . . . 91

15.1 Lab 1 — Entropy calculator . . . 91
15.2 Lab 2 — Shannon–Fano encoder and decoder . . . 94
15.3 Lab 3 — Huffman encoder and decoder . . . 97
15.4 Lab 4 — Arithmetic encoder and decoder . . . 100
15.5 Lab 5 — LZW compressor and decompressor . . . 104
15.6 Lab 6 — Compression algorithm comparison . . . 107

15.7 Lab 7 — Hamming error detection and correction . . . 109
15.8 Lab 8 — Multi-bit correction: repetition and BCH . . . 112
15.9 Lab 9 — Markov chains . . . 116
15.10 Lab 10 — Channel capacity . . . 119
15.11 Lab 11 — Entropy-based anomaly detection . . . 121
15.12 Running all tests . . . 123

VI Solved Problems, Exams and Final Project

16 Fully Solved Problems . . . 129
16.1 Entropy and self-information . . . 129
16.2 Joint, conditional entropy and leakage . . . 131
16.3 Mutual information and KL divergence . . . 132
16.4 Source coding basics . . . 133
16.5 Shannon–Fano . . . 134
16.6 Huffman . . . 135
16.7 Arithmetic coding . . . 136
16.8 LZW . . . 137
16.9 Markov sources . . . 137
16.10 Channels . . . 138
16.11 Error control . . . 139

17 Examination Question Bank . . . 140
17.1 Multiple-choice questions . . . 140
17.2 Short-answer questions . . . 142
17.3 Numerical exercises . . . 143
17.4 Programming challenges . . . 143
17.5 Answer keys and solutions . . . 144

18 Final Project: An Information Theory and Compression Laboratory . . . 147
18.1 Goals . . . 147
18.2 Source coding and channel coding: two separate layers . . . 147
18.3 Project architecture . . . 149
18.4 The program . . . 149
18.5 Sample results . . . 152
18.6 Testing instructions . . . 153
18.7 Extensions . . . 153

Appendix A — Mathematical Prerequisites . . . 155

Appendix B — MATLAB Implementations 157

Appendix C — Glossary 176

Appendix D — Bibliography 178

PART I

Foundations of Information Theory

Uncertainty, probability, entropy, leakage and mutual information — the language for measuring secrets. Modules 1–4 of the course plan.

CHAPTER

1



Cybersecurity as Uncertainty Management

IN THIS CHAPTER YOU WILL LEARN

- what information is, and how it differs from data;
- random experiments, random variables and distributions;
- self-information: the surprise of an outcome;
- a first look at entropy and Shannon's idea;
- threat models and the attacker's knowledge;
- how many bits a password or a key really has.

1.1 What is information theory?

Real-world analogy

A friend tells you "the sun rose this morning". You learn nothing: you were sure already. Another friend tells you "you won the lottery". You learn a lot: it was very unlikely. Information is the *removal of uncertainty*, and the less likely a message, the more information it brings.

Information theory was created by Claude Shannon in his 1948 paper *A Mathematical Theory of Communication*. Shannon asked two practical questions:

1. How much can a message be *compressed* without losing anything? (Answer: down to its entropy — Part II.)
2. How fast can we send messages *reliably* over a noisy channel? (Answer: up to the channel capacity — Part IV.)

The same mathematics answers a third question that is central to security: *how much does an attacker know about a secret, and how much does each observation teach him?* This is why this course treats cybersecurity as uncertainty management.

1.2 Information versus data

Definition 1.1 • Data and information

Data is the raw representation: symbols, bits, files. **Information** is what the data tells us that we did not know before, measured by the reduction of uncertainty.

A 1-megabyte file of zeros is a lot of data and almost no information: you can describe it in one sentence. A 1-megabyte file of truly random bits is the same amount of data but cannot be described more briefly — it is all information. Compression (Part II) removes the difference between the two.

1.3 Uncertainty and probability

1.3.1 Random experiments and random variables

Definition 1.2 • Random experiment, outcome, event

A **random experiment** is a process whose result cannot be predicted with certainty (tossing a coin, choosing a password, receiving a packet). Each possible result is an **outcome**; the set of all outcomes is the **sample space** Ω; a set of outcomes is an **event**.

Definition 1.3 • Discrete random variable and distribution

A **discrete random variable** X takes values in a finite or countable set $\mathcal{X}$ (the **alphabet**). Its **probability distribution** is $p(x) = P(X = x)$, with

$$p(x) \geq 0 \quad \text{and} \quad \sum_{x \in \mathcal{X}} p(x) = 1.$$

Worked Example 1.1 • Three distributions

- Fair coin: $\mathcal{X} = \{H, T\}$, $p(H) = p(T) = \frac{1}{2}$.
- Fair die: $\mathcal{X} = \{1, \dots, 6\}$, $p(x) = \frac{1}{6}$.
- Daily weather report: $p(\text{sunny}) = \frac{1}{2}$, $p(\text{cloudy}) = \frac{1}{4}$, $p(\text{rain}) = \frac{1}{8}$, $p(\text{snow}) = \frac{1}{8}$. Check: $\frac{1}{2} + \frac{1}{4} + \frac{1}{8} + \frac{1}{8} = 1$.

1.4 Measuring information: self-information

We want a number $I(x)$ that measures the surprise of outcome x. Three natural requirements:

1. it depends only on the probability $p = p(x)$;
2. a certain outcome ($p = 1$) gives 0; rarer outcomes give more;
3. for two *independent* outcomes, surprises add: $I(p_1 p_2) = I(p_1) + I(p_2)$.

Derivation • Why a logarithm?

Requirement 3 says I turns products into sums. The only continuous functions with $f(ab) = f(a) + f(b)$ are $f(a) = -K \log a$ (up to the choice of constant $K > 0$, which only fixes the unit). Requirement 2 forces the minus sign. Choosing base 2 and $K = 1$ makes the unit the *bit*: the surprise of one fair coin toss, $-\log_2 \frac{1}{2} = 1$.

Information theory concept • Self-information

$$I(x) = -\log_2 p(x) \quad \text{bits.}$$

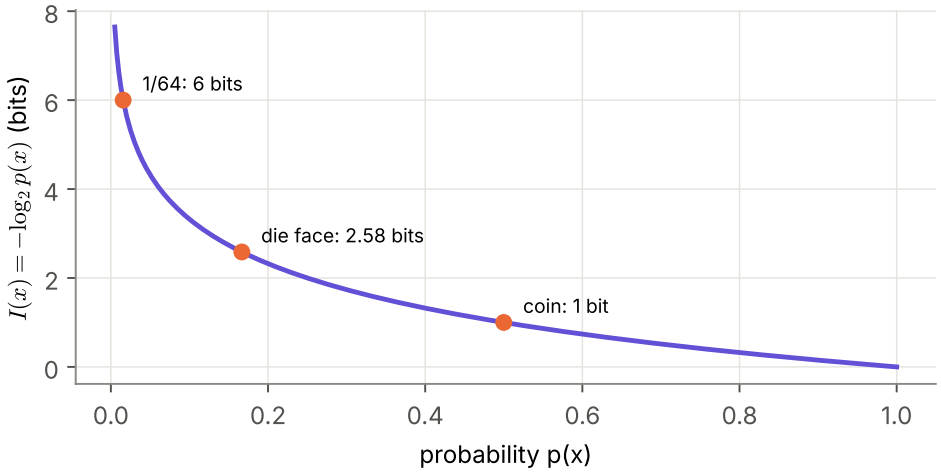


Figure 1.1. Self-information grows without limit as the probability goes to zero.

Worked Example 1.2 • Surprise in bits

Outcome	$p(x)$	$I(x) = -\log_2 p(x)$
heads of a fair coin	$1/2$	1 bit
face 4 of a fair die	$1/6$	$\log_2 6 = 2.585$ bits
ace of spades from a deck	$1/52$	$\log_2 52 = 5.700$ bits
weather “snow”	$1/8$	3 bits

Useful trick: $\log_2 x = \ln x / \ln 2 = \log_{10} x / 0.30103$.

1.5 Introduction to entropy

Self-information measures one outcome. Before the experiment we do not know which outcome will happen, so we take the *average* surprise:

Information theory concept • Entropy (first look)

$$H(X) = \sum_x p(x)\, I(x) = -\sum_x p(x) \log_2 p(x) \quad \text{bits.}$$

$H(X)$ is the average uncertainty about X before we observe it — equivalently, the average information we gain when we observe it.

Worked Example 1.3 • Entropy of the weather report

$$H = \tfrac{1}{2}(1) + \tfrac{1}{4}(2) + \tfrac{1}{8}(3) + \tfrac{1}{8}(3) = 0.5 + 0.5 + 0.375 + 0.375 = 1.75 \text{ bits}.$$

Four equally likely reports would give $\log_2 4 = 2$ bits. The skewed distribution is more predictable, so it carries less information — and, as Chapter 7 will show, its reports can be sent with only 1.75 bits each on average (codes 0, 10, 110, 111).

1.6 Why information has mathematical value

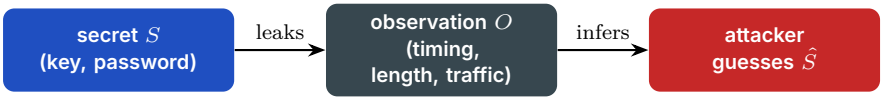
Shannon's key insight was to separate information from meaning. The *content* of a message does not matter; only the *probabilities* of the possible messages do. This makes information measurable, like length or energy, and leads to hard limits that no engineer can beat:

- no lossless compressor can use fewer than H bits per symbol on average;
- no code can send reliably faster than the channel capacity C;
- no attacker can learn more about a secret than the mutual information between the secret and what he observes.

1.7 Cybersecurity as uncertainty management

Definition 1.4 • Threat model

A **threat model** states what the attacker wants (the asset), what he can do (capabilities), and what he already knows (prior knowledge). In information-theoretic terms the attacker's knowledge about a secret S is a probability distribution $p(s)$; security means keeping his uncertainty about S high.



before: uncertainty $H(S) \quad \longrightarrow \quad$ after: $H(S \mid O)$
(leakage $= \quad H(S) - H(S \mid O)$, Chapters 3–4)

Confidentiality as uncertainty

The three classic goals are *confidentiality*, *integrity* and *availability*. Information theory gives confidentiality a number: the attacker's remaining uncertainty about the secret. Integrity is linked to error detection and correction (Chapter 14), and availability to reliable communication over noisy channels (Chapter 13).

1.7.1 Passwords and keys in bits

If a secret is chosen *uniformly* from N possibilities, every value has $p = 1/N$, so $H = \log_2 N$ bits and an attacker who tries all values needs on average $(N+1)/2$ guesses.

Worked Example 1.4 • How many bits?

Secret (chosen uniformly)	N	$H = \log_2 N$
4-digit PIN	10^4	13.29 bits
6-digit one-time code	10^6	19.93 bits
8 lowercase letters	26^8	$8 \log_2 26 = 37.60$ bits
12 printable ASCII characters	94^{12}	$12 \log_2 94 = 78.66$ bits
AES-128 key	2^{128}	128 bits

Each extra bit doubles the attacker's work. 128 bits means about 1.7×10^{38} guesses on average.

! "Uniformly" is the key word

These numbers are correct only if every value is equally likely. Humans do not choose passwords uniformly: "123456" is far more likely than "k#8Qz!pR". Chapter 2 shows how to measure the strength of a skewed distribution, and why the average (Shannon entropy) can be misleading for security.

1.8 Module summary

Key points

- Information is reduction of uncertainty; it depends only on probabilities.
- Self-information $I(x) = -\log_2 p(x)$; entropy is the average self-information.
- Security can be described as keeping the attacker's uncertainty about the secret high.
- A uniform secret from N values has $\log_2 N$ bits.

Formula sheet

Distribution	$p(x) \geq 0, \sum p(x) = 1$
Self-information	$I(x) = -\log_2 p(x)$
Entropy	$H(X) = -\sum p(x) \log_2 p(x)$
Uniform secret	$H = \log_2 N$; average guesses $(N+1)/2$
Change of base	$\log_2 x = \ln x / \ln 2$

Visual recap • Module 1



Review questions

1. Why does a certain event carry no information?
2. Give an example of a lot of data with little information.
3. What three things does a threat model describe?
4. Why are the bit values in the password table upper bounds for human-chosen passwords?

Solved exercises

Problem 1.1. A message is one of 32 equally likely commands. How much information does it carry?

Solution $\log_2 32 = 5$ bits.

Problem 1.2. An event has probability 0.01. Find its self-information.

Solution $-\log_2 0.01 = \log_2 100 = 6.644$ bits.

Problem 1.3. A secret is a 5-letter word from a list of 4096 words, chosen uniformly. How many bits does it have, and how many guesses does an attacker need on average?

Solution $\log_2 4096 = 12$ bits; $(4096 + 1)/2 = 2048.5$ guesses.

Problem 1.4. Two independent events have probabilities $1/4$ and $1/8$. Find the self-information of both happening.

Solution $I = -\log_2(1/32) = 5$ bits $= 2 + 3$ bits: surprises of independent events add.

Problem 1.5. Compute the entropy of a source with $p = (0.5, 0.25, 0.25)$.

Solution $0.5(1) + 0.25(2) + 0.25(2) = 1.5$ bits.

CHAPTER

2

Entropy as a Measure of Security Strength



IN THIS CHAPTER YOU WILL LEARN

- Shannon entropy from first principles and its axioms;
- binary entropy and its graph;
- properties: minimum, maximum, and their proofs;
- entropy of many-symbol sources, worked in full;
- a first look at joint entropy;
- password and key strength, and why entropy alone is not enough.

2.1 Entropy from first principles

The twenty-questions game

A friend thinks of one of 8 animals, all equally likely. You ask yes/no questions. Halving the list each time, you always need exactly 3 questions: $\log_2 8 = 3$. If one animal is chosen half of the time, you ask "is it the cat?" first and often finish in one question. Entropy is the *smallest average number of yes/no questions* needed to find the outcome.

Definition 2.1 • Shannon entropy

For a discrete random variable X with distribution $p(x)$,

$$H(X) = -\sum_{x \in \mathcal{X}} p(x) \log_2 p(x) = \mathbb{E}\left[-\log_2 p(X)\right] \quad \text{bits},$$

with the convention $0 \log_2 0 = 0$ (an impossible outcome contributes nothing).

2.1.1 Axiomatic motivation

Shannon showed that H is the only function (up to the unit) with three reasonable properties:

1. **Continuity.** A small change in the probabilities changes H only a little.

2. **Monotonicity.** For N equally likely outcomes, H grows with N: more choices, more uncertainty.
3. **Grouping (chain) rule.** If a choice is split into two stages, the total uncertainty is the uncertainty of the first stage plus the weighted uncertainty of the second.

Worked Example 2.1 • The grouping rule on numbers

$p = (\frac{1}{2}, \frac{1}{3}, \frac{1}{6})$. Directly: $H = \frac{1}{2}(1) + \frac{1}{3}\log_2 3 + \frac{1}{6}\log_2 6 = 0.5 + 0.5283 + 0.4308 = 1.4591$ bits. In two stages: first decide "first symbol or not" with $(\frac{1}{2}, \frac{1}{2})$: 1 bit. With probability $\frac{1}{2}$ we then choose between the last two with conditional probabilities $(\frac{2}{3}, \frac{1}{3})$: $H_2(\frac{1}{3}) = 0.9183$ bits. Total $1 + \frac{1}{2}(0.9183) = 1.4591$ bits ✓.

2.2 Binary entropy

Definition 2.2 • Binary entropy function

For a binary source with $P(1) = p$:

$$H_2(p) = -p\log_2 p - (1-p)\log_2(1-p).$$

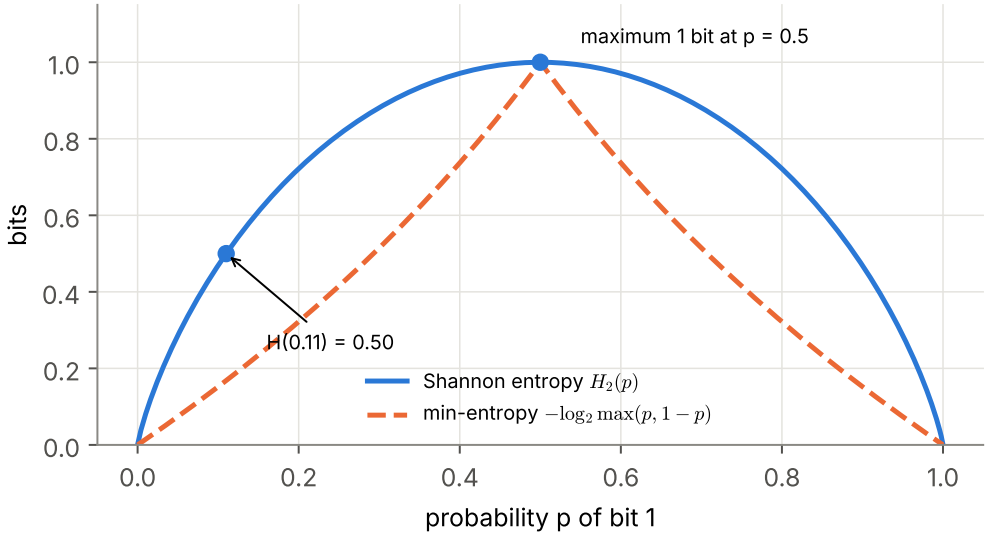


Figure 2.1. Binary entropy (solid) and min-entropy (dashed). Both are 1 bit only for a fair coin.

Worked Example 2.2 • Values of H_2

p	0	0.01	0.1	0.11	0.25	0.5
$H_2(p)$	0	0.0808	0.4690	0.4999	0.8113	1

Example for $p = 0.1$: $-0.1\log_2 0.1 - 0.9\log_2 0.9 = 0.1(3.3219) + 0.9(0.1520) = 0.3322 + 0.1368 = 0.4690$ bits.

The curve is symmetric ($H_2(p) = H_2(1-p)$), concave, 0 at the ends and 1 at $p = \frac{1}{2}$. A biased random bit generator that outputs 1 with probability 0.11 produces only half a bit of entropy per

output bit.

2.3 Properties of entropy

Information theory concept • Bounds

For an alphabet of N symbols:

$$0 \le H(X) \le \log_2 N.$$

$H = 0$ if and only if one outcome is certain; $H = \log_2 N$ if and only if X is uniform.

Derivation • Proof of the bounds

Lower bound. Each term $-p \log_2 p \ge 0$ for $0 \le p \le 1$, so $H \ge 0$; it is 0 only when every p is 0 or 1.

Upper bound. Use the inequality $\ln z \le z - 1$ (equality only at $z = 1$). Then

$$\begin{aligned} H(X) - \log_2 N &= \sum_x p(x) \log_2 \frac{1}{N\,p(x)} = \frac{1}{\ln 2} \sum_x p(x) \ln \frac{1}{Np(x)} \\ &\le \frac{1}{\ln 2} \sum_x p(x) \Big(\frac{1}{Np(x)} - 1 \Big) = \frac{1}{\ln 2} \Big(\sum_x \frac{1}{N} - 1 \Big) = 0. \end{aligned}$$

Equality needs $\frac{1}{Np(x)} = 1$ for every x, i.e. the uniform distribution.

Other properties:

- **Relabelling does not matter.** H depends only on the list of probabilities, not on the symbols.
- **Concavity.** Mixing two distributions never decreases entropy: $H(\lambda p + (1-\lambda)q) \ge \lambda H(p) + (1-\lambda)H(q)$.
- **Adding impossible symbols** (probability 0) does not change H.

2.4 Entropy of sources with many symbols

Worked Example 2.3 • Four symbols

$p = (0.4, 0.3, 0.2, 0.1)$.

$-0.4 \log_2 0.4$	$= 0.4 \times 1.3219$	$= 0.5288$
$-0.3 \log_2 0.3$	$= 0.3 \times 1.7370$	$= 0.5211$
$-0.2 \log_2 0.2$	$= 0.2 \times 2.3219$	$= 0.4644$
$-0.1 \log_2 0.1$	$= 0.1 \times 3.3219$	$= 0.3322$
H		$= 1.8464$ bits

Maximum for 4 symbols is 2 bits, so this source is 92.3% as uncertain as a uniform one.

Worked Example 2.4 • Entropy of a text

The word ABRACADABRA has 11 letters: A×5, B×2, R×2, C×1, D×1. Relative frequencies give

$$H = \tfrac{5}{11}\log_2\tfrac{11}{5} + 2\cdot\tfrac{2}{11}\log_2\tfrac{11}{2} + 2\cdot\tfrac{1}{11}\log_2 11 = 0.5170 + 0.8943 + 0.6290 = 2.0404 \text{ bits/letter.}$$

The maximum for 5 letters is $\log_2 5 = 2.3219$. (Lab 1 prints exactly these numbers.)

2.5 Joint entropy: a first look

Definition 2.3 • Joint entropy

For two random variables with joint distribution $p(x, y)$:

$$H(X,Y) = -\sum_{x,y} p(x,y)\log_2 p(x,y).$$

- Two independent fair bits: four equally likely pairs, $H(X,Y) = 2 = H(X) + H(Y)$.
- A fair bit copied twice ($Y = X$): only 00 and 11 occur, $H(X,Y) = 1 < H(X) + H(Y)$.

In general $H(X,Y) \le H(X) + H(Y)$, with equality exactly for independent variables. Chapter 3 develops this into conditional entropy and leakage.

2.6 Entropy versus predictability

Low entropy means predictable; high entropy means unpredictable. A source with entropy H can be predicted, compressed and guessed according to its entropy:

- **compression:** about H bits per symbol are needed (Chapter 5);
- **prediction:** the best guess of the next symbol is right with probability $\max_x p(x)$;
- **guessing:** an attacker trying the most likely values first succeeds faster when the distribution is skewed.

2.7 Cybersecurity focus: password and key strength

Definition 2.4 • Min-entropy

$$H_\infty(X) = -\log_2 \max_x p(x).$$

It measures the chance of the attacker's *single best guess*: $P(\text{success in one guess}) = 2^{-H_\infty}$. Always $H_\infty(X) \le H(X)$, with equality for the uniform distribution.

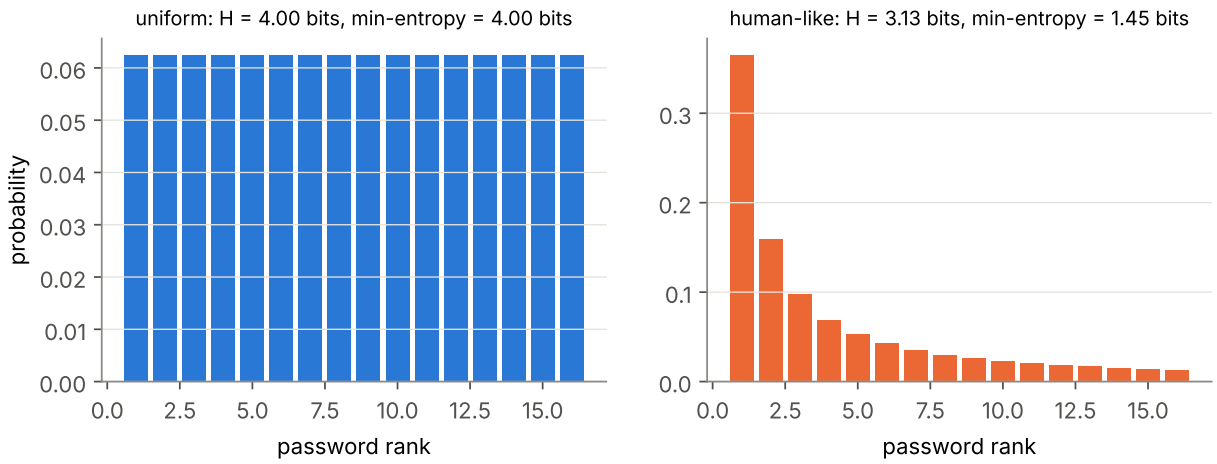


Figure 2.2. Two distributions over 16 passwords. Left: uniform. Right: human-like (Zipf, $p_k \propto k^{-1.2}$).

Worked Example 2.5 • Uniform versus human-like passwords

Both distributions in Fig. 2.2 use 16 passwords. Uniform: $H = H_\infty = 4$ bits. Human-like: the most popular password has probability 0.365, so $H_\infty = -\log_2 0.365 = 1.45$ bits, while the Shannon entropy is 3.13 bits. An attacker succeeds with the first guess 36.5% of the time — far worse than "3.13 bits" suggests.

Worked Example 2.6 • Why entropy alone does not guarantee security

A system generates a password as follows: with probability $\frac{1}{2}$ it uses "123456"; otherwise it picks uniformly one of 2^{40} random strings.

$$H = \tfrac{1}{2}\log_2 2 + \tfrac{1}{2}(\log_2 2 + \log_2 2^{40}) = 0.5 + 0.5 \times 41 = 21 \text{ bits.}$$

21 bits looks acceptable, but $H_\infty = -\log_2 \frac{1}{2} = 1$ bit: one guess breaks half of all accounts. **For security, the worst case matters.** Use min-entropy (or guessing entropy) to rate secrets; use Shannon entropy for compression and averages.

Key randomness and secret generation

- Cryptographic keys must come from a generator whose output is (close to) uniform; then $H = H_\infty =$ key length.
- Physical noise sources are biased. Designers measure min-entropy per raw bit and use a *randomness extractor* (for example a cryptographic hash) to compress, say, 256 raw bits with 0.5 bit min-entropy each into a 128-bit key.
- Weak distributions are the real danger: predictable seeds (time of day, process ID) give keys with very low entropy even if the key is 128 bits long.

Fig. 2.3 shows how a real system turns a biased physical noise source into a key that can be trusted.

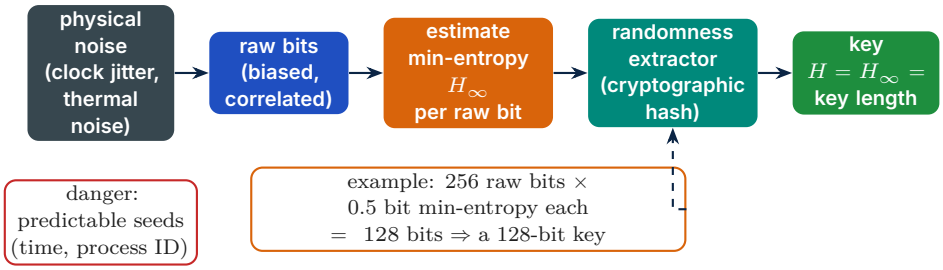


Figure 2.3. Block diagram. From a physical noise source to a uniform key: measure the worst case (min-entropy), then compress it with an extractor.

Python: the entropy calculator (full code in Lab 1)

```python
from math import log2
def entropy(probs):
    return -sum(p * log2(p) for p in probs if p > 0)
print(entropy([0.4, 0.3, 0.2, 0.1]))      # 1.8464393446710154
```

2.8 Module summary

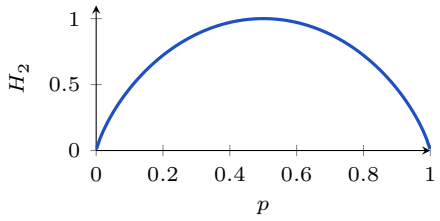
Key points

- $H(X)$ is the average surprise and the minimum average number of yes/no questions.
- $0 \le H \le \log_2 N$; maximum only for the uniform distribution.
- $H_2(p)$ is symmetric with maximum 1 bit at $p = \frac{1}{2}$.
- For guessing attacks, min-entropy H_∞ is the safe measure; H can hide weak spots.

Formula sheet

Entropy	$H(X) = -\sum p \log_2 p$
Binary entropy	$H_2(p) = -p \log_2 p - (1-p) \log_2 (1-p)$
Bounds	$0 \le H \le \log_2 N$
Joint entropy	$H(X,Y) = -\sum p(x,y) \log_2 p(x,y) \le H(X) + H(Y)$
Min-entropy	$H_\infty = -\log_2 \max p(x) \le H$

Visual recap • Module 2



uniform $\Rightarrow$ maximum entropy $\log_2 N$
certain $\Rightarrow$ zero entropy
security: check the *worst case* $H_\infty = -\log_2 \max p$

Review questions

1. State the three axioms that lead to Shannon's entropy.
2. Why is the binary entropy curve symmetric?
3. Which distribution maximises entropy for a fixed alphabet size?
4. Give an example where H is large but a secret is weak.

Solved exercises

Problem 2.1. Find H for $p = (\frac{1}{8}, \frac{1}{8}, \frac{1}{8}, \frac{1}{8}, \frac{1}{2})$.

Solution $4 \cdot \frac{1}{8} \cdot 3 + \frac{1}{2} \cdot 1 = 1.5 + 0.5 = 2$ bits.

Problem 2.2. A random bit generator outputs 1 with probability 0.25. How many entropy bits does 1000 output bits contain?

Solution $1000\, H_2(0.25) = 1000 \times 0.8113 = 811.3$ bits.

Problem 2.3. Find the min-entropy of $p = (0.5, 0.25, 0.25)$ and compare with H.

Solution $H_\infty = -\log_2 0.5 = 1$ bit; $H = 1.5$ bits; $H_\infty < H$ as always for non-uniform distributions.

Problem 2.4. X is uniform on 8 values and $Y = X \bmod 2$. Find $H(X, Y)$.

Solution Y is a function of X, so the pair has the same 8 equally likely values: $H(X, Y) = H(X) = 3$ bits.

Problem 2.5. A 10-character password uses 62 symbols uniformly. How many bits? How many characters from 94 symbols give at least the same?

Solution $10 \log_2 62 = 59.54$ bits. Need $n \log_2 94 \geq 59.54$, $n \geq 59.54/6.555 = 9.08$, so 10 characters (9 give only 59.0 bits).

CHAPTER

3

Conditional Entropy and Information Leakage



IN THIS CHAPTER YOU WILL LEARN

- joint, marginal and conditional distributions;
- conditional entropy $H(Y|X)$ and how to compute it;
- the chain rule of entropy, with proof;
- uncertainty after an observation;
- leakage $= H(S) - H(S|O)$;
- side-channel, metadata and partial-disclosure examples.

3.1 Joint and marginal distributions

Real-world analogy

A security guard sees whether employees carry an umbrella. He cannot see the sky. Still, umbrellas tell him something about the weather. Conditional entropy measures how much uncertainty about the weather remains *after* he sees the umbrellas.

Definition 3.1 • Joint, marginal and conditional probability

For two random variables X and Y:

- joint: $p(x, y) = P(X = x, Y = y)$;
- marginals: $p(x) = \sum_y p(x, y)$ and $p(y) = \sum_x p(x, y)$;
- conditional: $p(y|x) = \dfrac{p(x, y)}{p(x)}$ for $p(x) > 0$ (Bayes: $p(x|y) = \dfrac{p(y|x)p(x)}{p(y)}$).

Worked Example 3.1 • Weather X and umbrella Y

$p(x,y)$	$Y=$ no umbrella	$Y=$ umbrella	$p(x)$
$X=$ sunny	0.30	0.10	**0.40**
$X=$ rainy	0.15	0.45	**0.60**
$p(y)$	**0.45**	**0.55**	1

Row sums give $p(x)$, column sums give $p(y)$. Conditional: $p(\text{umbrella}|\text{rainy}) = 0.45/0.60 = 0.75$, $p(\text{rainy}|\text{umbrella}) = 0.45/0.55 = 0.818$.

3.2 Conditional entropy

Definition 3.2 • Conditional entropy

The entropy of Y given that $X = x$ is $H(Y|X=x) = -\sum_y p(y|x)\log_2 p(y|x)$. Averaging over x:

$$H(Y|X) = \sum_x p(x)\, H(Y|X=x) = -\sum_{x,y} p(x,y)\log_2 p(y|x).$$

It is the average uncertainty that remains about Y when X is known.

Worked Example 3.2 • Conditional entropies of the umbrella table

$H(Y|X)$ **(umbrella given weather).** Sunny row: $p(y|\text{sunny}) = (0.75, 0.25)$, entropy $H_2(0.25) = 0.8113$. Rainy row: $(0.25, 0.75)$, entropy 0.8113. So $H(Y|X) = 0.4(0.8113) + 0.6(0.8113) = 0.8113$ bits.

$H(X|Y)$ **(weather given umbrella).** Column "no": $p(x|\text{no}) = (0.30, 0.15)/0.45 = (0.667, 0.333)$, entropy 0.9183. Column "umbrella": $(0.10, 0.45)/0.55 = (0.182, 0.818)$, entropy 0.6840. $H(X|Y) = 0.45(0.9183) + 0.55(0.6840) = 0.4132 + 0.3762 = 0.7894$ bits.

Compare $H(X) = H_2(0.4) = 0.9710$ bits: seeing umbrellas reduces the guard's uncertainty about the weather by 0.1815 bits.

3.3 The chain rule

Information theory concept • Chain rule of entropy

$$H(X,Y) = H(X) + H(Y|X) = H(Y) + H(X|Y).$$

More generally $H(X_1, \dots, X_n) = \sum_{i=1}^{n} H(X_i|X_1, \dots, X_{i-1})$.

Derivation • Proof

Write $p(x,y) = p(x)p(y|x)$ and take $-\log_2$: $-\log_2 p(x,y) = -\log_2 p(x) - \log_2 p(y|x)$. Average both sides over $p(x,y)$:

$$H(X,Y) = -\sum_{x,y} p(x,y)\log_2 p(x) - \sum_{x,y} p(x,y)\log_2 p(y|x) = H(X) + H(Y|X),$$

because $\sum_y p(x,y) = p(x)$ in the first sum. The general form follows by applying the two-

variable rule repeatedly.

Worked Example 3.3 • Checking the chain rule

For the umbrella table: $H(X,Y) = -(0.3\log_2 0.3 + 0.1\log_2 0.1 + 0.15\log_2 0.15 + 0.45\log_2 0.45) = 1.7822$ bits. $H(X)+H(Y|X) = 0.9710+0.8113 = 1.7823$ ✓ (rounding), and $H(Y)+H(X|Y) = 0.9928+0.7894 = 1.7822$ ✓.

3.4 Entropy after observation

Information theory concept • Conditioning reduces entropy — on average

$$H(X|Y) \le H(X),$$

with equality if and only if X and Y are independent. (Proof in Chapter 4: $H(X)-H(X|Y) = I(X;Y) \ge 0$.)

! A single observation can increase uncertainty

The inequality holds on *average*, not for every value y. Example: an intrusion $X = 1$ happens with probability 0.1, so $H(X) = H_2(0.1) = 0.469$ bits. An alarm Y fires in all intrusions and in 10% of normal periods: $p(0,0) = 0.8$, $p(0,1) = 0.1$, $p(1,1) = 0.1$. When the alarm fires, intrusion and no intrusion are equally likely: $H(X|Y=1) = 1$ bit > 0.469. But on average $H(X|Y) = 0.8(0) + 0.2(1) = 0.2$ bits < 0.469.

3.5 Information leakage models

Definition 3.3 • Leakage

A secret S and an observation O available to the attacker. The **leakage** is the reduction of the attacker's uncertainty:

$$\mathcal{L} = H(S) - H(S|O).$$

The **remaining uncertainty** is $H(S|O)$. (In Chapter 4 we will see that $\mathcal{L} = I(S;O)$, the mutual information.)

Worked Example 3.4 • Side channel: a timing leak of the key weight

A 2-bit key $K \in \{00, 01, 10, 11\}$ is uniform: $H(K) = 2$ bits. A faulty implementation takes a time proportional to the number of ones W (Hamming weight) of the key, and the attacker measures it.

$p(k,w)$	$W=0$	$W=1$	$W=2$
00	1/4	0	0
01	0	1/4	0
10	0	1/4	0
11	0	0	1/4
$p(w)$	1/4	1/2	1/4

If $W=0$ or $W=2$, the key is known: $H(K|W=w)=0$. If $W=1$, two keys remain: 1 bit. So

$$H(K|W)=\tfrac{1}{4}(0)+\tfrac{1}{2}(1)+\tfrac{1}{4}(0)=0.5 \text{ bits}, \qquad \mathcal{L}=2-0.5=1.5 \text{ bits}.$$

The timing leak reveals 75% of the key's uncertainty. For an n-bit key, the weight leaks $H(W)=H(\text{Binomial}(n,\tfrac{1}{2}))\approx\tfrac{1}{2}\log_2(\tfrac{\pi e n}{2})$ bits (about 4.5 bits for $n=128$), which is small but can be combined over many measurements.

Worked Example 3.5 • Partial secret disclosure

A 16-digit random number (uniform) has $16\log_2 10=53.15$ bits. A receipt prints the last 4 digits. They are independent of the other 12, so $H(S|O)=12\log_2 10=39.86$ bits and the leakage is exactly $4\log_2 10=13.29$ bits.

Worked Example 3.6 • Metadata leakage: message length

An encrypted reply is "YES" or "NO", equally likely, and encryption preserves length. The ciphertext reveals 3 or 2 bytes, so $H(\text{reply}|\text{length})=0$: *all* 1 bit leaks although the content is perfectly encrypted. Padding both answers to the same length makes $H(\text{reply}|\text{length})=1$ bit again.

Fig. 3.1 gives the general recipe that all three examples followed.

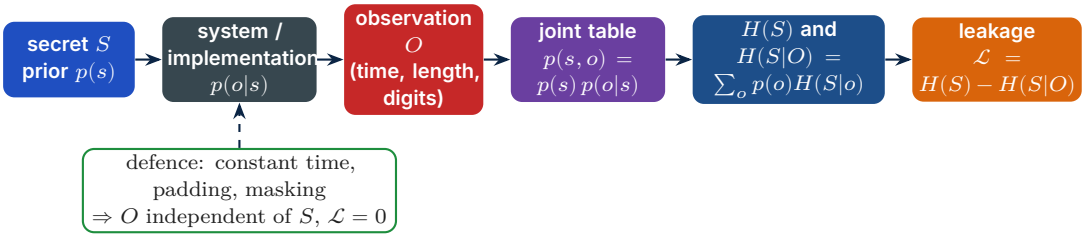


Figure 3.1. Block diagram. Measuring information leakage: model the implementation as a channel from the secret to the observation.

What leaks in practice

- **Side channels:** execution time, power consumption, electromagnetic emission, cache behaviour.
- **Metadata:** message size and timing, IP addresses, who talks to whom.
- **Partial disclosure:** masked card numbers, password hints, error messages that say "wrong username" vs "wrong password".

Each can be modelled as a channel from the secret to an observation; $H(S|O)$ tells what

remains secret.

Python: conditional entropy from a joint table

```python
from entropy import entropy, conditional_entropy     # Lab 1 module
P = [[0.25, 0, 0], [0, 0.25, 0], [0, 0.25, 0], [0, 0, 0.25]]   # rows K, cols W
PW = [list(col) for col in zip(*P)]                    # rows W, cols K
print(conditional_entropy(PW))     # H(K|W) = 0.5
```

3.6 Module summary

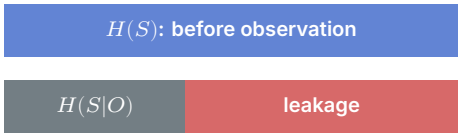
Key points

- Marginals are row/column sums; conditionals divide by the marginal.
- $H(Y|X)$ averages the entropy of each row of conditional probabilities.
- Chain rule: $H(X,Y) = H(X) + H(Y|X)$.
- Leakage $= H(S) - H(S|O)$; conditioning never increases entropy on average.

Formula sheet

Marginal	$p(x) = \sum_y p(x,y)$
Conditional	$p(y\|x) = p(x,y)/p(x)$
Conditional entropy	$H(Y\|X) = \sum_x p(x) H(Y\|X=x)$
Chain rule	$H(X,Y) = H(X) + H(Y\|X)$
Leakage	$\mathcal{L} = H(S) - H(S\|O) \geq 0$

Visual recap • Module 3



remaining uncertainty + leakage = original uncertainty

Review questions

1. How do you obtain $p(x)$ from a joint table?
2. Is $H(X|Y) = H(Y|X)$ in general? Use the umbrella example.
3. Why can one observation increase uncertainty although conditioning reduces entropy?
4. How does padding reduce metadata leakage?

Solved exercises

Problem 3.1. X is a fair bit and $Y = X$. Find $H(Y|X)$ and $H(X|Y)$.

Solution Knowing one gives the other: both are 0. $H(X, Y) = 1$.

Problem 3.2. X, Y independent fair bits. Find $H(X|Y)$.

Solution $H(X|Y) = H(X) = 1$ bit: no leakage.

Problem 3.3. A 3-bit uniform key leaks its parity (XOR of the bits). Find the remaining uncertainty.

Solution Each parity value leaves 4 equally likely keys: $H(K|P) = 2$ bits; leakage 1 bit.

Problem 3.4. Given $p(x, y)$: $p(0, 0) = 0.5$, $p(0, 1) = 0.25$, $p(1, 0) = 0$, $p(1, 1) = 0.25$, find $H(X)$, $H(X, Y)$ and $H(Y|X)$.

Solution $p(x) = (0.75, 0.25)$, $H(X) = 0.8113$. $H(X, Y) = 0.5(1) + 0.25(2) + 0.25(2) = 1.5$. $H(Y|X) = 1.5 - 0.8113 = 0.6887$ bits.

CHAPTER

4

Mutual Information and Attacker Advantage



IN THIS CHAPTER YOU WILL LEARN

- mutual information: meaning, definition, Venn diagram;
- the relations between $H(X)$, $H(Y)$, $H(X,Y)$, $H(X|Y)$, $I(X;Y)$;
- independence and zero information;
- Kullback–Leibler divergence (relative entropy);
- how KL and mutual information are connected;
- traffic analysis: bits an attacker learns from packet sizes.

4.1 Mutual information: intuition

Real-world analogy

Two witnesses describe the same accident. What they say *in common* is shared information; what only one of them says is private. Mutual information is the size of the common part: how much one variable tells about the other.

Definition 4.1 • Mutual information

$$I(X;Y) = \sum_{x,y} p(x,y) \log_2 \frac{p(x,y)}{p(x)\,p(y)}.$$

Equivalent forms:

$$I(X;Y) = H(X) - H(X|Y) = H(Y) - H(Y|X) = H(X) + H(Y) - H(X,Y).$$

Derivation • Why the three forms are equal

Write $\frac{p(x,y)}{p(x)p(y)} = \frac{p(x|y)}{p(x)}$. Then

$$I(X;Y) = \sum_{x,y} p(x,y)[-\log_2 p(x)] - \sum_{x,y} p(x,y)[-\log_2 p(x|y)] = H(X) - H(X|Y).$$

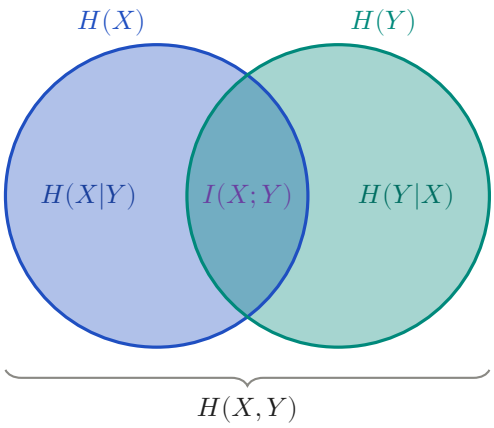


Figure 4.1. Entropy Venn diagram. Each circle is the entropy of one variable; the overlap is the mutual information; the union is the joint entropy.

By symmetry of the definition in x and y, also $I = H(Y) - H(Y|X)$. Using the chain rule $H(X|Y) = H(X,Y) - H(Y)$ gives $I = H(X) + H(Y) - H(X,Y)$.

4.2 Properties

- **Symmetric:** $I(X;Y) = I(Y;X)$ — X tells as much about Y as Y about X.
- **Non-negative:** $I(X;Y) \geq 0$, with equality iff X and Y are independent.
- **Bounded:** $I(X;Y) \leq \min\{H(X), H(Y)\}$.
- **Self-information:** $I(X;X) = H(X)$ — entropy is the information a variable has about itself.

Worked Example 4.1 • The umbrella table again

From Chapter 3: $H(X) = 0.9710$, $H(Y) = 0.9928$, $H(X,Y) = 1.7822$. So

$$I(X;Y) = 0.9710 + 0.9928 - 1.7822 = 0.1815 \text{ bits},$$

the same as $H(X) - H(X|Y) = 0.9710 - 0.7894$. Umbrellas reveal about 0.18 bits about the weather per person.

Worked Example 4.2 • Independence gives zero

If $p(x,y) = p(x)p(y)$ for all pairs, every log term is $\log_2 1 = 0$, so $I = 0$. Example: $p(x) = (0.4, 0.6)$, $p(y) = (0.3, 0.7)$, $p(x,y) = (0.12, 0.28, 0.18, 0.42)$: $H(X,Y) = H(X) + H(Y) = 0.9710 + 0.8813 = 1.8523$, $I = 0$.

4.3 Kullback–Leibler divergence

Definition 4.2 • Relative entropy (KL divergence)

For two distributions p and q on the same alphabet,

$$D(p\|q) = \sum_x p(x) \log_2 \frac{p(x)}{q(x)}.$$

It is the average number of *extra* bits needed when data from p is coded with a code designed for q.

Information theory concept • Gibbs' inequality

$D(p\|q) \geq 0$, with equality if and only if $p = q$.

Derivation • Proof

With $\ln z \leq z - 1$: $-D(p\|q)\ln 2 = \sum_x p(x) \ln \frac{q(x)}{p(x)} \leq \sum_x p(x)\left(\frac{q(x)}{p(x)} - 1\right) = \sum_x q(x) - 1 \leq 0$.

Worked Example 4.3 • KL is not symmetric

$p = (0.5, 0.5)$ (fair coin), $q = (0.9, 0.1)$ (biased coin).

$$D(p\|q) = 0.5 \log_2 \tfrac{0.5}{0.9} + 0.5 \log_2 \tfrac{0.5}{0.1} = 0.5(-0.8480) + 0.5(2.3219) = 0.7370 \text{ bits},$$

$$D(q\|p) = 0.9 \log_2 \tfrac{0.9}{0.5} + 0.1 \log_2 \tfrac{0.1}{0.5} = 0.9(0.8480) + 0.1(-2.3219) = 0.5310 \text{ bits}.$$

So KL is not a distance in the geometric sense. If $q(x) = 0$ while $p(x) > 0$, $D(p\|q) = \infty$: a model that says "impossible" to something that happens is infinitely wrong. (This is why Chapter 12 smooths estimated probabilities.)

Information theory concept • Mutual information is a KL divergence

$$I(X;Y) = D(p(x,y) \,\|\, p(x)p(y)).$$

Mutual information measures how far the true joint distribution is from the "independent" one. Gibbs' inequality then proves $I \geq 0$, and hence $H(X|Y) \leq H(X)$.

4.4 Cybersecurity focus: attacker advantage

Worked Example 4.4 • Traffic analysis of encrypted web browsing

A user visits one of three sites $W \in \{A, B, C\}$ with probabilities $(0.5, 0.3, 0.2)$. The traffic is encrypted, but the attacker sees the response size class $S \in \{\text{small}, \text{medium}, \text{large}\}$ with the channel $p(s|w)$:

$p(s\|w)$	small	medium	large
A	0.80	0.15	0.05
B	0.10	0.70	0.20
C	0.05	0.15	0.80

Step 1 — joint table $p(w,s) = p(w)p(s|w)$: row A $(0.40, 0.075, 0.025)$, row B $(0.03, 0.21, 0.06)$, row C $(0.01, 0.03, 0.16)$.
Step 2 — marginal of S: $p(s) = (0.44, 0.315, 0.245)$.
Step 3 — entropies: $H(W) = 1.4855$, $H(S) = 1.5433$, $H(W,S) = 2.4514$ bits.
Step 4 — mutual information: $I(W;S) = 1.4855 + 1.5433 - 2.4514 = 0.5773$ bits.
Step 5 — remaining uncertainty: $H(W|S) = 1.4855 - 0.5773 = 0.9082$ bits.
Step 6 — attacker's best guess. Posteriors $p(w|s)$: small $\to (0.909, 0.068, 0.023)$, medium $\to (0.238, 0.667, 0.095)$, large $\to (0.102, 0.245, 0.653)$. Guessing the most likely site: success $= 0.40 + 0.21 + 0.16 = 0.77$, compared with 0.5 without observing traffic. The size channel raises the attacker's success from 50% to 77%.

Fig. 4.2 draws the attack of Worked Example 4.4 as a system with its numbers.

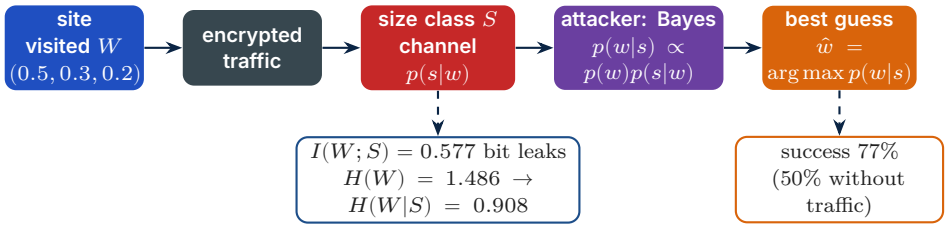


Figure 4.2. Block diagram. Traffic analysis: encryption hides the content, but the size class is a channel from the secret to the attacker.

Protocol leakage and statistical inference

- Encryption hides content, not sizes and timing. $I(\text{secret}; \text{observable})$ quantifies what still leaks.
- Defences (padding, constant-rate traffic, dummy packets) aim to make I small — at a bandwidth cost.
- Repeated observations add up: with n independent observations the attacker can learn up to nI bits (but never more than $H(W)$).

4.5 Limitations

- I is an *average*. A system can have small I but leak a secret completely in rare cases (compare min-entropy, Chapter 2).
- I requires a probability model. In practice $p(x, y)$ is estimated from data; small samples give biased (too large) estimates.
- I says how much is learnable, not how an attacker learns it or at what computational cost.

4.6 Module summary

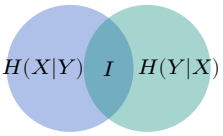
Key points

- $I(X;Y) = H(X) - H(X|Y)$: shared information; zero iff independent.
- $D(p\|q) \geq 0$, not symmetric, infinite if q forbids what p allows.
- $I(X;Y) = D(p_{XY}\|p_X p_Y)$.
- Attacker advantage from an observable = mutual information between secret and observable.

Formula sheet

Mutual information	$I = H(X) - H(X\|Y) = H(X) + H(Y) - H(X,Y)$
KL divergence	$D(p\|q) = \sum p \log_2(p/q) \geq 0$
Link	$I(X;Y) = D(p_{XY}\|p_X p_Y)$
Bounds	$0 \leq I(X;Y) \leq \min(H(X), H(Y))$

Visual recap • Module 4



overlap = mutual information = what the observation reveals;
$I = 0 \Leftrightarrow$ independent; $I = D(p_{XY}\|p_X p_Y)$

Review questions

1. Give three equivalent formulas for $I(X;Y)$.
2. Why is $D(p\|q)$ not a true distance?
3. What is $I(X;Y)$ if $Y = X$? If Y is independent of X?
4. Why can a small I still hide a dangerous leak?

Solved exercises

Problem 4.1. X uniform on $\{0,1,2,3\}$, $Y = X \bmod 2$. Find $I(X;Y)$.

Solution $H(Y) = 1$, $H(Y|X) = 0$ (deterministic), so $I = 1$ bit.

Problem 4.2. Compute $D(p\|q)$ for $p = (\frac{1}{2}, \frac{1}{4}, \frac{1}{4})$, $q = (\frac{1}{3}, \frac{1}{3}, \frac{1}{3})$.

Solution $\log_2 3 - H(p) = 1.5850 - 1.5 = 0.0850$ bits.

Problem 4.3. A BSC with crossover 0.1 has a uniform input. Find $I(X;Y)$.

Solution $H(Y) = 1$, $H(Y|X) = H_2(0.1) = 0.4690$, $I = 0.5310$ bits.

Problem 4.4. Show that if $H(X|Y) = 0$ then $I(X;Y) = H(X)$, and interpret for security.

Solution $I = H(X) - 0$. The observation reveals the secret completely: total leakage.

PART II

Source Coding and Data Compression

Removing redundancy: prefix codes, the source coding theorem, Shannon–Fano, Huffman, arithmetic coding and LZW. Module 5, greatly extended.

CHAPTER

5



Introduction to Source Coding

IN THIS CHAPTER YOU WILL LEARN

- why we compress, lossless versus lossy;
- fixed-length and variable-length codes;
- unique decodability, prefix-free and instantaneous codes;
- average length, efficiency, compression ratio, redundancy;
- the Kraft–McMillan inequality, with proof;
- Shannon's source coding theorem: $H \le L < H + 1$;
- block coding: approaching the entropy.

5.1 Why data compression is necessary

Real-world analogy

In Morse code the most common English letter, E, is a single dot, while the rare Q is "dash dash dot dash". Samuel Morse counted letters in a printer's type case and gave short codes to frequent letters. That is the whole idea of variable-length source coding, discovered a century before Shannon explained why it works.

Storage and bandwidth cost money and energy. Compression saves them by removing *redundancy*: predictable structure that does not carry new information.

Definition 5.1 • Lossless and lossy compression

Lossless compression reproduces the original exactly (ZIP, PNG, the four algorithms of this part). **Lossy** compression accepts small, controlled changes for much higher compression (JPEG, MP3, video codecs). Text, programs, keys and logs need lossless compression.

5.2 Codes and codewords

Definition 5.2 • Source code

A **source code** assigns to each source symbol x a binary **codeword** $c(x)$ of length $\ell(x)$. A message is encoded by concatenating the codewords.

5.2.1 Fixed-length codes

With N symbols, a fixed-length code needs $\lceil \log_2 N \rceil$ bits per symbol. ASCII uses 8 bits (7 used) for every character. Simple and robust, but wasteful when some symbols are much more frequent.

5.2.2 Variable-length codes

Worked Example 5.1 • Four codes for the weather source

Source: sunny $\frac{1}{2}$, cloudy $\frac{1}{4}$, rain $\frac{1}{8}$, snow $\frac{1}{8}$ ($H = 1.75$ bits).

Symbol	Code A	Code B	Code C	Code D
sunny	00	0	0	0
cloudy	01	1	01	10
rain	10	00	011	110
snow	11	11	0111	111
average length	2	1.25	1.875	1.75
uniquely decodable?	yes	no	yes	yes
instantaneous?	yes	no	no	yes

Code B is too short to be true: "00" could be "sunny sunny" or "rain". **Code C** can always be decoded, but you must look ahead: after "0" you do not know if the codeword ends until you see the next bit. **Code D** is the best: decodable at once, and its average length equals the entropy.

Fig. 5.1 shows the source-coding system that Part II studies, with the quantities used to measure it.

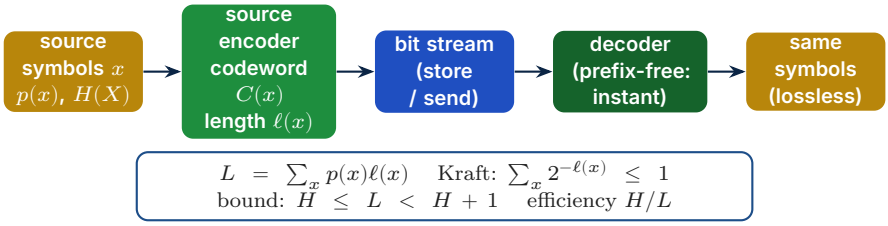


Figure 5.1. Block diagram. A lossless source-coding system and its measures (Sections 5.4–5.6).

5.3 Unique decodability and prefix-free codes

Definition 5.3 • Uniquely decodable, prefix-free, instantaneous

- A code is **uniquely decodable** if every finite bit string comes from at most one sequence of symbols.
- A code is **prefix-free** (a prefix code) if no codeword is the beginning of another codeword.
- A code is **instantaneous** if each symbol can be decoded as soon as its last bit arrives. Prefix-free $\Leftrightarrow$ instantaneous.

Every prefix-free code is uniquely decodable; the converse is false (code C).

5.3.1 Code trees

A prefix-free code is a binary tree: each left branch is a 0, each right branch a 1, and codewords are *leaves*. Decoding is a walk from the root to a leaf, then back to the root.

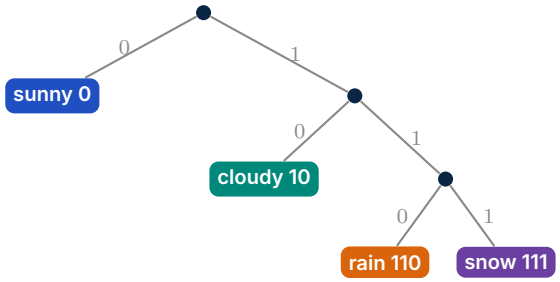


Figure 5.2. Code D as a tree. Codewords sit only at leaves, so the code is prefix-free. A codeword of length ℓ "uses" a fraction $2^{-\ell}$ of the tree.

5.4 Measuring a code

Definition 5.4 • Average length, efficiency, redundancy, compression ratio

$$L = \sum_x p(x)\,\ell(x) \ \text{ bits/symbol}, \qquad \eta = \frac{H(X)}{L}, \qquad \text{redundancy } = 1-\eta \ \text{ (or } L-H \text{ in bits)},$$

$$\text{compression ratio} = \frac{\text{original size}}{\text{compressed size}} \quad \Big(\text{e.g. } \frac{8 \text{ bits}}{L} \text{ for ASCII text}\Big).$$

Worked Example 5.2 • Measuring code D

$L = \frac{1}{2}(1) + \frac{1}{4}(2) + \frac{1}{8}(3) + \frac{1}{8}(3) = 1.75$ bits, $\eta = 1.75/1.75 = 100\%$, redundancy 0. Compared with the 2-bit fixed code A, the compression ratio is $2/1.75 = 1.14$; compared with 8-bit ASCII text it is $8/1.75 = 4.57$.

5.5 The Kraft–McMillan inequality

Information theory concept • Kraft–McMillan inequality

A prefix-free (and even any uniquely decodable) binary code with lengths $\ell_1, \dots, \ell_N$ exists if and only if

$$\sum_{i=1}^{N} 2^{-\ell_i} \le 1.$$

Derivation • Proof for prefix-free codes

Necessity. Let $\ell_{\max}$ be the longest length. At depth $\ell_{\max}$ the full tree has $2^{\ell_{\max}}$ nodes. A leaf at depth ℓ_i blocks all $2^{\ell_{\max}-\ell_i}$ nodes below it, and different leaves block disjoint sets (prefix-free). So $\sum_i 2^{\ell_{\max}-\ell_i} \le 2^{\ell_{\max}}$; divide by $2^{\ell_{\max}}$.
Sufficiency. Sort the lengths $\ell_1 \le \ell_2 \le \dots$ and give symbol i the first ℓ_i bits of the binary expansion of $\sum_{j<i} 2^{-\ell_j}$. Because these numbers are at least $2^{-\ell_i}$ apart, no codeword is a prefix of a later one.
(McMillan extended necessity to all uniquely decodable codes, so prefix codes lose nothing.)

Worked Example 5.3 • Kraft checks

Lengths $(1,2,3,3)$: $\frac{1}{2}+\frac{1}{4}+\frac{1}{8}+\frac{1}{8}=1$ — a complete prefix code exists (code D).
Lengths $(1,1,2)$: $\frac{1}{2}+\frac{1}{2}+\frac{1}{4}=1.25>1$ — impossible; this is code B, which is not decodable.
Lengths $(2,2,2,3)$: $0.875<1$ — possible, with one unused branch (the code can be shortened).

5.6 Shannon's source coding theorem

Information theory concept • Source coding theorem (single symbols)

For any uniquely decodable code, $L \ge H(X)$. There exists a prefix code with

$$H(X) \le L < H(X) + 1.$$

Derivation • Proof

Lower bound. Let $q_i = 2^{-\ell_i} / \sum_j 2^{-\ell_j}$ (a distribution). Then

$$L - H = \sum_i p_i \ell_i + \sum_i p_i \log_2 p_i = \sum_i p_i \log_2 \frac{p_i}{q_i} - \log_2 \sum_j 2^{-\ell_j} = D(p\|q) - \log_2 \sum_j 2^{-\ell_j} \ge 0,$$

because $D \ge 0$ (Gibbs) and $\sum_j 2^{-\ell_j} \le 1$ (Kraft).
Upper bound. Choose *Shannon lengths* $\ell_i = \lceil -\log_2 p_i \rceil$. Then $2^{-\ell_i} \le p_i$, so Kraft holds and a prefix code exists, and $\ell_i < -\log_2 p_i + 1$ gives $L < H + 1$.

Worked Example 5.4 • Shannon code lengths

$p = (0.4, 0.3, 0.2, 0.1)$, $H = 1.8464$. Lengths $\lceil -\log_2 p \rceil = (2, 2, 3, 4)$. Kraft: $\frac{1}{4} + \frac{1}{4} + \frac{1}{8} + \frac{1}{16} = 0.6875 \leq 1$. $L = 0.4(2) + 0.3(2) + 0.2(3) + 0.1(4) = 2.4$ bits: inside $[1.846, 2.846)$. Huffman (Chapter 7) does better: $L = 1.9$.

5.6.1 Approaching the entropy with blocks

Coding blocks of n symbols at once gives $nH \leq L_n < nH + 1$, so the length per symbol satisfies

$$H \leq \frac{L_n}{n} < H + \frac{1}{n} \longrightarrow H.$$

Worked Example 5.5 • Block coding of a biased binary source

$P(0) = 0.9$, $P(1) = 0.1$, $H = 0.4690$ bits. With Huffman codes on blocks:

block length n	1	2	3	4
L_n/n (bits/symbol)	1.000	0.645	0.533	0.493
efficiency $H/(L_n/n)$	46.9%	72.7%	88.0%	95.2%

For $n = 2$: block probabilities $(0.81, 0.09, 0.09, 0.01)$, Huffman lengths $(1, 2, 3, 3)$, $L_2 = 0.81 + 0.18 + 0.27 + 0.03 = 1.29$, i.e. 0.645 per symbol. Arithmetic coding (Chapter 8) reaches the same goal without building huge block tables.

Redundancy is exploitable structure

The redundancy $L - H$ is predictability. Attackers exploit it: redundant plaintext makes cryptanalysis easier, predictable protocol fields help forge messages, and compressibility reveals content (Chapter 10). Defenders use it too: a ciphertext or a key should look *incompressible* — a good test that it is close to random.

5.7 Module summary (5.1)

Key points

- Prefix-free codes are instantaneous and lose nothing compared with other decodable codes.
- Kraft: $\sum 2^{-\ell_i} \leq 1$ is the condition for a prefix code to exist.
- No decodable code beats the entropy; Shannon lengths give $L < H + 1$; blocks approach H.

Formula sheet

Average length	$L = \sum p(x)\ell(x)$
Efficiency, redundancy	$\eta = H/L, \;\; 1-\eta$
Kraft–McMillan	$\sum 2^{-\ell_i} \le 1$
Source coding theorem	$H \le L < H+1; \;\; H \le L_n/n < H + 1/n$
Shannon lengths	$\ell_i = \lceil -\log_2 p_i \rceil$

Review questions

1. Is every uniquely decodable code instantaneous? Give an example.
2. What does the Kraft sum equal for a complete binary code tree?
3. Why can no code have $L < H$?
4. How does block coding help a very skewed binary source?

Solved exercises

Problem 5.1. Is $\{0, 10, 110, 1110, 1111\}$ prefix-free? Compute its Kraft sum.

Solution Yes (no codeword starts another). $\frac{1}{2} + \frac{1}{4} + \frac{1}{8} + \frac{1}{16} + \frac{1}{16} = 1$.

Problem 5.2. Can a prefix code have lengths $(1, 2, 2, 3)$?

Solution $\frac{1}{2} + \frac{1}{4} + \frac{1}{4} + \frac{1}{8} = 1.125 > 1$: no.

Problem 5.3. A source has $H = 2.2$ bits and a code with $L = 2.5$. Find efficiency and redundancy.

Solution $\eta = 2.2/2.5 = 88\%$; redundancy 12% (0.3 bits/symbol).

Problem 5.4. Give Shannon lengths for $p = (0.5, 0.25, 0.125, 0.125)$ and the average length.

Solution $(1, 2, 3, 3)$, $L = 1.75 = H$: for dyadic probabilities the Shannon code is optimal.

CHAPTER

6



Shannon–Fano Coding

IN THIS CHAPTER YOU WILL LEARN

- the Shannon–Fano algorithm in ten steps;
- sorting tables and balanced partitions;
- building the code tree;
- encoding and decoding a message;
- average length, efficiency and redundancy;
- why Shannon–Fano is not always optimal;
- a working Python encoder and decoder.

6.1 The idea

A fair split of the class

To find a hidden student with yes/no questions, a smart teacher does not ask "is it Ahmed?" first. She splits the class into two groups that are *equally likely* to contain the student, asks which group, and repeats. Each answer then gives close to one full bit. Shannon–Fano coding (Shannon 1948, Fano 1949) builds codes exactly this way.

6.2 The algorithm

1. **Sort** the symbols by decreasing probability.
2. **Divide** the list into two parts whose total probabilities are as close as possible.
3. **Assign** bit 0 to the first (upper) part and bit 1 to the second.
4. **Repeat** steps 2–3 inside each part until every part has one symbol.
5. **Read** each codeword as the sequence of bits received by the symbol (this is the code tree).
6. **Encode** a message by concatenating codewords.
7. **Decode** by reading bits until they match a codeword (the code is prefix-free).
8. **Compute** $L = \sum p\ell$.

9. **Compute** efficiency H/L and redundancy $1 - H/L$.
10. **Compare** with the entropy bound $H \leq L < H + 1$.

Tie rule used in this book

When two split points are equally balanced, take the first one. Ties between equal probabilities are broken alphabetically. Different rules can give different (equally valid) codes; the Python code uses these rules.

Fig. 6.1 shows the same algorithm as a flow chart.

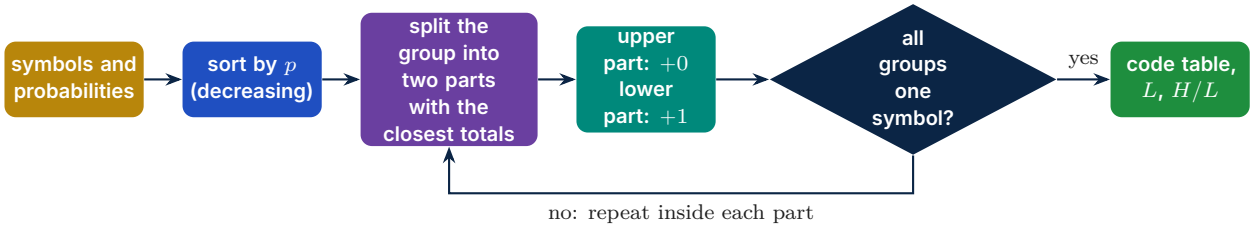


Figure 6.1. Block diagram. Shannon–Fano coding: top-down splitting.

6.3 A complete example with seven symbols

Worked Example 6.1 • Building the code

Source: A 0.25, B 0.20, C 0.15, D 0.15, E 0.10, F 0.10, G 0.05 (already sorted).
Split 1 (all symbols, total 1.00). Candidate splits after A: 0.25|0.75 (difference 0.50); after B: 0.45|0.55 (0.10); after C: 0.60|0.40 (0.20). Best: {A,B} gets 0, {C,D,E,F,G} gets 1.
Split 2 ({A,B}). A → 0, B → 1. Codes: A = 00, B = 01.
Split 3 ({C,...,G}, total 0.55). After C: 0.15|0.40 (0.25); after D: 0.30|0.25 (0.05); after E: 0.40|0.15 (0.25). Best: {C,D} → 10, {E,F,G} → 11.
Split 4 ({C,D}). C = 100, D = 101.
Split 5 ({E,F,G}, total 0.25). After E: 0.10|0.15 (0.05); after F: 0.20|0.05 (0.15). Best: E = 110, {F,G} → 111.
Split 6 ({F,G}). F = 1110, G = 1111.

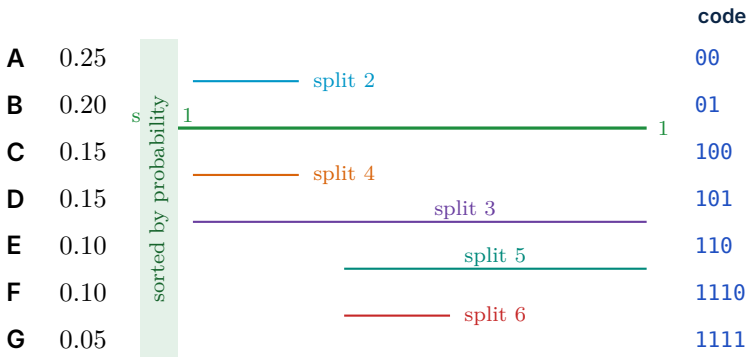


Figure 6.2. Shannon–Fano partition diagram: each line is a split; symbols above a line get 0, below get 1, inside their group.

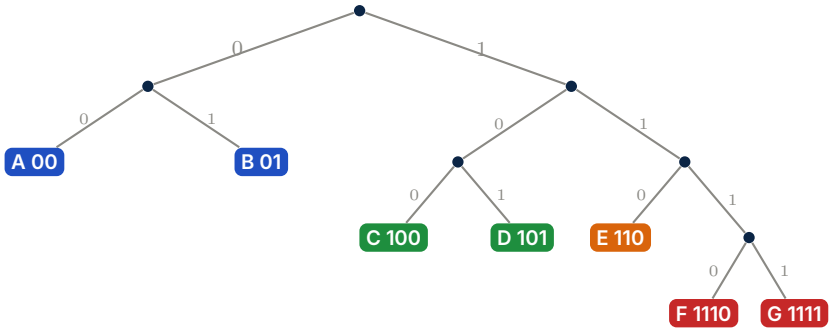


Figure 6.3. The resulting Shannon–Fano code tree.

Worked Example 6.2 • Encoding, decoding and measuring

Encode CABBAGE: C 100 | A 00 | B 01 | B 01 | A 00 | G 1111 | E 110 → `100000101001111110` (18 bits; ASCII would need 56).
Decode `100000101001111110`: read 1,0,0 → "100" = C; then 0,0 → A; 0,1 → B; 0,1 → B; 0,0 → A; 1,1,1,1 → G; 1,1,0 → E. Result CABBAGE ✓.
Average length: $L = 0.25(2)+0.20(2)+0.15(3)+0.15(3)+0.10(3)+0.10(4)+0.05(4) = 2.70$ bits/symbol.
Entropy: $H = 0.25(2) + 0.20(2.3219) + 2(0.15)(2.7370) + 2(0.10)(3.3219) + 0.05(4.3219) = 2.6660$ bits/symbol.
Efficiency: $\eta = 2.6660/2.70 = 98.74\%$; redundancy 1.26%. Check: $2.666 \le 2.70 < 3.666$ ✓. Compression ratio versus a 3-bit fixed code: $3/2.7 = 1.11$.

6.4 Why Shannon–Fano is not always optimal

Shannon–Fano works top-down: it makes the first split as fair as possible, and then cannot change it. A fair first split can force bad splits later.

Worked Example 6.3 • A case where Shannon–Fano loses

$p = (0.40, 0.17, 0.15, 0.14, 0.09, 0.05)$ for A–F, $H = 2.2998$ bits.
Shannon–Fano. First split {A,B} | {C,D,E,F} (0.57|0.43). Codes A 00, B 01, C 10, D 110, E 1110, F 1111. $L = 0.8 + 0.34 + 0.30 + 0.42 + 0.36 + 0.20 = 2.42$ bits.
Huffman (Chapter 7) gives A a 1-bit code: A 0, B 111, C 110, D 100, E 1011, F 1010; $L = 2.34$ bits.
Shannon–Fano wastes 0.08 bits/symbol here. The symbol A has probability 0.40, so its ideal length is $-\log_2 0.4 = 1.32$ bits: a 2-bit code is too long, but the balanced first split puts B together with A.

Important note

Shannon–Fano codes are usually close to optimal, but there is no optimality guarantee. Huffman's bottom-up method is provably optimal among all symbol-by-symbol prefix codes, so in exams and in practice Huffman is preferred.

6.5 Python implementation

Core of the encoder (full program with decoder in Lab 2)

```python
def shannon_fano(probs):
    items = sorted(probs.items(), key=lambda kv: (-kv[1], kv[0]))
    codes = {s: "" for s, _ in items}
    def split(group):
        if len(group) == 1:
            return
        total, run, best_k, best = sum(p for _, p in group), 0.0, 1, 2.0
        for k in range(1, len(group)):              # left part = group[:k]
            run += group[k - 1][1]
            if abs(2 * run - total) < best - 1e-12:
                best_k, best = k, abs(2 * run - total)
        for s, _ in group[:best_k]: codes[s] += "0"
        for s, _ in group[best_k:]: codes[s] += "1"
        split(group[:best_k]); split(group[best_k:])
    split(items)
    return codes
```

Codebooks reveal statistics

A transmitted Shannon–Fano or Huffman codebook tells everybody which symbols are frequent. Short codewords leak that their symbols are common, and the length of an encoded message depends on its content. Chapter 10 discusses how such lengths become side channels.

6.6 Summary

- Sort, split into balanced halves, assign 0/1, recurse.
- Fast and simple; codes are prefix-free and usually close to the entropy.
- Not always optimal: the top-down split can be wrong for very probable symbols. Huffman is optimal.

Solved exercises

Problem 6.1. Build a Shannon–Fano code for $(0.5, 0.25, 0.125, 0.125)$.

Solution Splits: $\{0.5\}|\{0.25, 0.125, 0.125\}$, then $\{0.25\}|\{0.125, 0.125\}$. Codes 0, 10, 110, 111; $L = H = 1.75$: 100% efficient.

Problem 6.2. Shannon–Fano for $(0.35, 0.17, 0.17, 0.16, 0.15)$: find the code and L.

Solution Split {A,B} (0.52) | {C,D,E} (0.48); then C | {D,E}. Codes A 00, B 01, C 10, D 110, E 111. $L = 0.70 + 0.34 + 0.34 + 0.48 + 0.45 = 2.31$ bits ($H = 2.2328$; Huffman gives 2.30).

Problem 6.3. Decode `1110001111101` with the seven-symbol code of this chapter.

Solution 1110 = F, 00 = A, 1111 = G, 101 = D: “FAGD”.

CHAPTER

7



Huffman Coding

IN THIS CHAPTER YOU WILL LEARN

- the greedy idea: merge the two least probable symbols;
- priority queues and step-by-step merge tables;
- building the tree and reading the codes;
- encoding, decoding, average length, efficiency;
- why Huffman is optimal (and under which assumptions);
- Huffman versus Shannon–Fano; complexity $O(n \log n)$.

7.1 Principles

A knock-out tournament in reverse

In a tournament the strongest teams play the most rounds. In a Huffman tree the *least* probable symbols are merged first, so they end up deepest (longest codewords), and the most probable symbols stay near the root (shortest codewords).

David Huffman found the algorithm in 1952 as an MIT student, solving a term-paper problem that his teacher, Robert Fano, had worked on himself. The key observation: in an optimal code the two least probable symbols can be given codewords of the same (maximum) length that differ only in the last bit. So merge them into one "super-symbol" and solve the smaller problem — a *greedy* algorithm.

7.2 The algorithm

1. Put every symbol in a **priority queue** keyed by probability (a min-heap).
2. Remove the two nodes with the smallest probabilities.
3. Create a parent node whose probability is their sum; label the edges 0 and 1.
4. Insert the parent back into the queue.

5. Repeat steps 2–4 until one node (the root, probability 1) remains.
6. Read each codeword along the path from the root to the leaf.

Conventions of this book

The first node removed (smaller probability) gets bit 0, the second gets bit 1. Ties are broken by age: the node that entered the queue first is removed first (leaves enter in alphabetical order). Other conventions give different codewords but the *same lengths and the same average length*.

Fig. 7.1 shows the algorithm as a flow chart. Compare it with Fig. 6.1: Shannon–Fano works top-down, Huffman bottom-up.

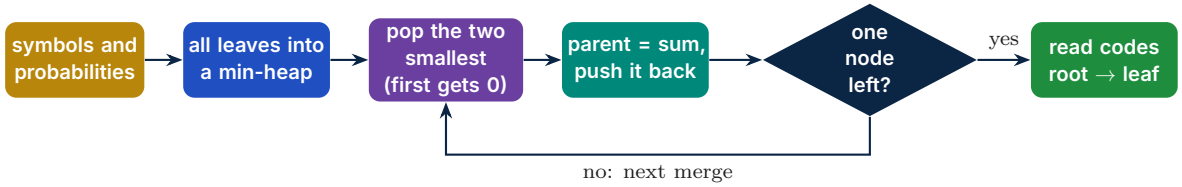


Figure 7.1. Block diagram. Huffman coding: bottom-up merging with a priority queue.

7.3 Worked examples

Worked Example 7.1 • Six symbols, step by step

Source: A 0.30, B 0.25, C 0.15, D 0.12, E 0.10, F 0.08 ($H = 2.4224$ bits).

Step	Queue before the step (sorted)	Merge (0 + 1)	New node
1	F.08 E.10 D.12 C.15 B.25 A.30	F + E	FE 0.18
2	D.12 C.15 FE.18 B.25 A.30	D + C	DC 0.27
3	FE.18 B.25 DC.27 A.30	FE + B	FEB 0.43
4	DC.27 A.30 FEB.43	DC + A	DCA 0.57
5	FEB.43 DCA.57	FEB + DCA	root 1.00

Reading the tree (Fig. 7.2): A 11, B 01, C 101, D 100, E 001, F 000.
$L = 0.30(2) + 0.25(2) + 0.15(3) + 0.12(3) + 0.10(3) + 0.08(3) = 2.45$ bits; $\eta = 2.4224/2.45 = 98.87\%$.
Encode FACADE: 000 11 101 11 100 001 → `0001110111100001` (16 bits).
Decode by walking the tree: 000 → F, 11 → A, 101 → C, 11 → A, 100 → D, 001 → E ✓.

Worked Example 7.2 • Huffman beats Shannon–Fano

Source from Chapter 6: A 0.40, B 0.17, C 0.15, D 0.14, E 0.09, F 0.05 ($H = 2.2998$).

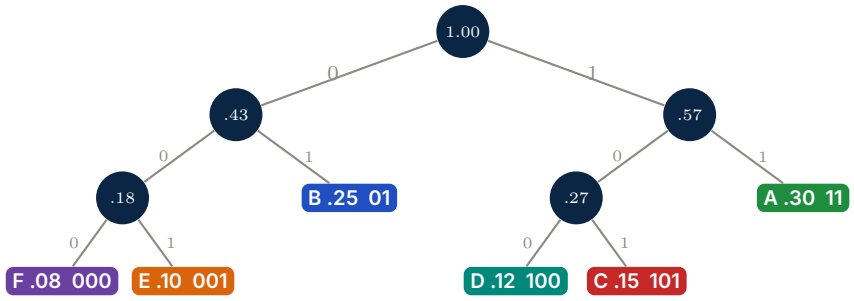


Figure 7.2. Huffman tree of Worked Example 7.3. Internal nodes show merged probabilities.

Step	Merge (0 + 1)	New node	Note
1	F.05 + E.09	FE 0.14	
2	D.14 + FE.14	DFE 0.28	tie: leaf D is older, so it gets 0
3	C.15 + B.17	CB 0.32	
4	DFE.28 + CB.32	DFECB 0.60	
5	A.40 + DFECB.60	root	

Codes: A 0, D 100, F 1010, E 1011, C 110, B 111. $L = 0.40 + 3(0.17) + 3(0.15) + 3(0.14) + 4(0.09) + 4(0.05) = 2.34$ bits, $\eta = 98.3\%$, while Shannon–Fano gave 2.42 bits (95.0%).

Worked Example 7.3 • Huffman coding of a text from counts

ABRACADABRA: counts A 5, B 2, R 2, C 1, D 1 (Huffman works with counts as well as probabilities).
Merges: C1 + D1 = 2; B2 + R2 = 4; CD2 + BR4 = 6; A5 + CDBR6 = 11.
Codes: A 0, C 100, D 101, B 110, R 111. Encoded text: `0 110 111 0 100 0 101 0 110 111 0` = 23 bits (ASCII: 88 bits, ratio 3.83). $L = 23/11 = 2.0909$ bits/letter; $H = 2.0404$; $\eta = 97.6\%$.

7.4 Optimality

Information theory concept • Huffman optimality

Among all prefix codes that encode *one symbol at a time* with a *known, fixed* distribution, the Huffman code has the smallest average length. Therefore $H \le L_{\text{Huffman}} < H + 1$.

Derivation • Sketch of the proof

Lemma 1. In an optimal code, a more probable symbol never has a longer codeword (otherwise swapping them reduces L).
Lemma 2. In an optimal code the longest codewords come in sibling pairs (a leaf with no sibling could move up one level), and we may assume the two least probable symbols are siblings at the deepest level.
Induction. Merge those two symbols x, y into z with $p_z = p_x + p_y$. For any code of the reduced source, splitting z back adds exactly p_z to the average length: $L = L' + p_z$. So

minimising L is the same as minimising L' — the same problem with one symbol fewer. Huffman does exactly this at every step.

! What "optimal" does not mean

Huffman is optimal only for symbol-by-symbol codes with integer lengths. It can be far from H when one symbol is very likely (a binary source with $P(0) = 0.99$ still needs 1 bit/symbol while $H = 0.08$). Block Huffman or arithmetic coding (Chapter 8) fixes this. And if the probabilities used are wrong, the code is not optimal for the real data (the loss is the KL divergence, Chapter 4).

7.5 Efficiency, compression ratio and complexity

- Efficiency $\eta = H/L$; compression ratio $= 8/L$ against ASCII (or $\lceil \log_2 N \rceil / L$ against a fixed-length code).
- **Time complexity.** $n - 1$ merges, each with heap operations costing $O(\log n)$: building the code is $O(n \log n)$ for n symbols. Encoding is a table look-up per symbol; decoding walks the tree, $O(\ell)$ per symbol.
- **Memory.** The tree has $2n - 1$ nodes. The decoder needs the code: either the tree, the counts, or the code lengths (canonical Huffman codes need only the lengths).

7.6 Huffman versus Shannon–Fano

	Shannon–Fano	Huffman
Direction	top-down: split groups	bottom-up: merge smallest
Optimal?	not always	yes, among symbol codes
Complexity	$O(n \log n)$ sort + splits	$O(n \log n)$ with a heap
Typical use	history, teaching	DEFLATE (ZIP, PNG), JPEG, MP3 back-ends

7.7 Python implementation

Core of the Huffman builder (full program in Lab 3)

```python
import heapq
from itertools import count
def huffman(probs):
    tick = count()                              # tie-breaker: older first
    heap = [(p, next(tick), s) for s, p in sorted(probs.items())]
    heapq.heapify(heap)
    while len(heap) > 1:
        p0, _, n0 = heapq.heappop(heap)         # smallest  -> bit 0
        p1, _, n1 = heapq.heappop(heap)         # 2nd       -> bit 1
        heapq.heappush(heap, (p0 + p1, next(tick), (n0, n1)))
    codes = {}
    def walk(node, prefix):
        if isinstance(node, tuple):
            walk(node[0], prefix + "0"); walk(node[1], prefix + "1")
        else:
            codes[node] = prefix or "0"
    walk(heap[0][2], "")
    return codes
```

Line by line. The heap entries are triples (probability, age, node); Python compares them left to right, so equal probabilities are ordered by age and nodes are never compared. A tuple is an internal node with children (0-child, 1-child). The walk appends 0 or 1 at each level.

Huffman codes and data leakage

Variable-length codes make the size of the output depend on the content: a message of frequent symbols becomes short. Without encryption, Huffman-coded data is also easy to partially decode from the middle once synchronised. With encryption after compression, the content is hidden but the *length* still leaks information (Chapter 10).

7.8 Summary

- Repeatedly merge the two least probable nodes; edges get 0/1; codes are paths.
- Optimal for symbol codes: $H \le L < H + 1$.
- $O(n \log n)$; used inside DEFLATE, JPEG and many standards.

Solved exercises

Problem 7.1. Huffman code for $(0.4, 0.3, 0.2, 0.1)$; find L and η.

Solution Merge 0.1+0.2=0.3; then 0.3+0.3=0.6 (leaf 0.3 older gets 0); then 0.4+0.6. Lengths $(1, 2, 3, 3)$: $L = 0.4 + 0.6 + 0.6 + 0.3 = 1.9$; $\eta = 1.8464/1.9 = 97.2\%$.

Problem 7.2. A source has 8 equally likely symbols. What does Huffman produce?

Solution A complete tree of depth 3: every codeword has 3 bits; $L = H = 3$.

Problem 7.3. Why can a Huffman code for a binary source never compress it?

Solution Two symbols always get codes 0 and 1 (1 bit each): $L = 1$. Compression needs blocks of symbols or arithmetic coding.

CHAPTER

8



Arithmetic Coding

IN THIS CHAPTER YOU WILL LEARN

- why arithmetic coding beats symbol codes;
- cumulative probability tables and interval subdivision;
- encoding a whole message as one number;
- step-by-step decoding, and how the decoder knows when to stop;
- three fully solved examples with number-line diagrams;
- finite precision: integer coders with scaling;
- arithmetic coding versus Huffman.

8.1 Why arithmetic coding?

Zooming into a map

To tell a friend where a house is, you can say "Egypt", then "Cairo", then "Nasr City", then "street 9". Each word zooms into a smaller part of the previous region. Arithmetic coding does the same with the interval $[0, 1)$: each symbol zooms into a sub-interval whose size equals its probability. At the end, *one number* inside the final small interval identifies the whole message.

Huffman must give every symbol a whole number of bits, so a symbol with probability 0.9 still costs at least 1 bit although its information is only 0.152 bits. Arithmetic coding spends $-\log_2 p$ bits per symbol *on average, without rounding*: the whole message costs about $-\log_2 P(\text{message}) + 2$ bits.

8.2 Cumulative tables and interval subdivision

Definition 8.1 • Cumulative probability table

Order the symbols (here: by symbol name). Symbol s gets the slot $[\,F(s^-),\,F(s)\,)$, where $F(s^-)$ is the total probability of the symbols before s and $F(s) = F(s^-) + p(s)$.

Information theory concept • Interval update

Start with $[\,\text{low}, \text{high}) = [0, 1)$. For each symbol s:

$$w = \text{high} - \text{low}, \qquad \text{high} \leftarrow \text{low} + w\,F(s), \qquad \text{low} \leftarrow \text{low} + w\,F(s^-).$$

After the message, the interval width equals $P(\text{message}) = \prod p(s_i)$. The encoder outputs the shortest binary fraction $0.b_1b_2\ldots b_k$ such that the whole dyadic interval $[0.b_1\ldots b_k,\ 0.b_1\ldots b_k + 2^{-k})$ lies inside $[\text{low}, \text{high})$. This needs at most $\lceil -\log_2 P(\text{message})\rceil + 1$ bits.

8.3 Example 1: a dyadic source

Worked Example 8.1 • Encoding ABAC

$p(A) = \frac{1}{2}$, $p(B) = \frac{1}{4}$, $p(C) = \frac{1}{4}$. Table: A $[0, \frac{1}{2})$, B $[\frac{1}{2}, \frac{3}{4})$, C $[\frac{3}{4}, 1)$.

Symbol	low before	width w	slot	new interval	new width
A	0	1	$[0, \frac{1}{2})$	$[0,\ \frac{1}{2})$	$\frac{1}{2}$
B	0	$\frac{1}{2}$	$[\frac{1}{2}, \frac{3}{4})$	$[\frac{1}{4},\ \frac{3}{8})$	$\frac{1}{8}$
A	$\frac{1}{4}$	$\frac{1}{8}$	$[0, \frac{1}{2})$	$[\frac{1}{4},\ \frac{5}{16})$	$\frac{1}{16}$
C	$\frac{1}{4}$	$\frac{1}{16}$	$[\frac{3}{4}, 1)$	$[\frac{19}{64},\ \frac{5}{16})$	$\frac{1}{64}$

Final interval $[0.296875,\ 0.3125)$, width $\frac{1}{64} = P(ABAC) = \frac{1}{2}\cdot\frac{1}{4}\cdot\frac{1}{2}\cdot\frac{1}{4}$.
Tag. $\frac{19}{64} = 0.010011_2$ and $[\frac{19}{64}, \frac{20}{64})$ is exactly the final interval, so the code is `010011`: 6 bits $= -\log_2 \frac{1}{64}$. Huffman (A 0, B 10, C 11) would also use $1+2+1+2 = 6$ bits: for dyadic probabilities both are perfect.

8.4 Example 2: decimal probabilities

Worked Example 8.2 • Encoding BAC

$p(A) = 0.5$, $p(B) = 0.3$, $p(C) = 0.2$. Table: A $[0, 0.5)$, B $[0.5, 0.8)$, C $[0.8, 1)$.
B: $w = 1$, new $[0 + 1(0.5),\ 0 + 1(0.8)) = [0.5,\ 0.8)$.
A: $w = 0.3$, new $[0.5 + 0.3(0),\ 0.5 + 0.3(0.5)) = [0.5,\ 0.65)$.
C: $w = 0.15$, new $[0.5 + 0.15(0.8),\ 0.5 + 0.15(1)) = [0.62,\ 0.65)$.
Width $0.03 = 0.3 \times 0.5 \times 0.2$; information $-\log_2 0.03 = 5.06$ bits.
Tag search. $k = 5$: smallest multiple of $\frac{1}{32}$ above 0.62 is $\frac{20}{32} = 0.625$, but $0.625 + \frac{1}{32} = 0.65625 > 0.65$: no. $k = 6$: $\frac{40}{64} = 0.625$ and $0.625 + \frac{1}{64} = 0.640625 \le 0.65$: yes. Code `101000`

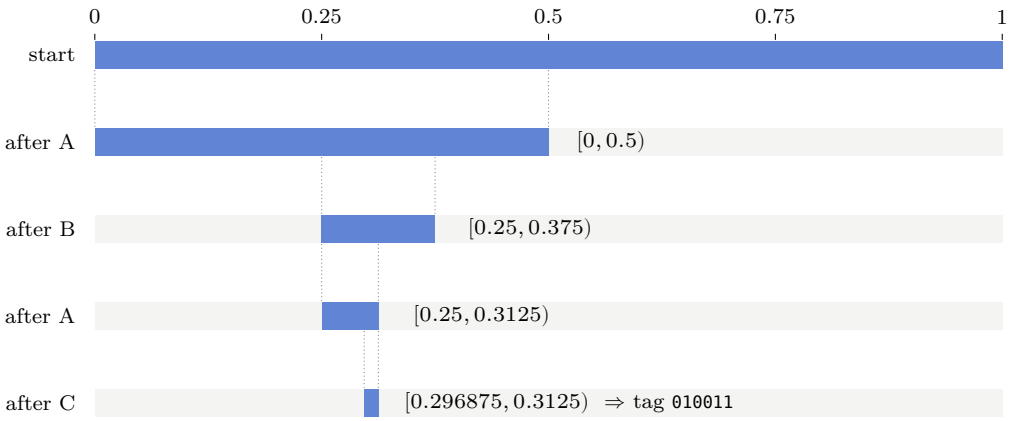


Figure 8.1. Interval narrowing for ABAC. Each row keeps the slot of the next symbol inside the previous interval.

(6 bits).

8.5 Decoding and stopping

Information theory concept • Decoding rule

Let v be the received number. Repeat: find the symbol s whose slot contains $t = \dfrac{v - \text{low}}{w}$; output s; update the interval exactly like the encoder.

The number v lies inside infinitely many nested intervals, so the decoder **must be told when to stop**. Two standard ways:

1. send the message length n first (used by the Python integer coder and by most file formats);
2. reserve a **terminator** symbol (EOF) with a small probability and stop when it is decoded.

Worked Example 8.3 • Example 3: decoding with a terminator

Alphabet # 0.2 (terminator), A 0.4, B 0.4. Table: # $[0, 0.2)$, A $[0.2, 0.6)$, B $[0.6, 1)$. Message BA# encodes to the interval $[0.68,\ 0.712)$ and the tag `101100`, i.e. $v = 0.6875$.
Step 1: interval $[0, 1)$, $t = 0.6875 \in [0.6, 1) \Rightarrow$ B; new interval $[0.6, 1)$.
Step 2: $t = (0.6875 - 0.6)/0.4 = 0.21875 \in [0.2, 0.6) \Rightarrow$ A; new interval $[0.68, 0.84)$.
Step 3: $t = (0.6875 - 0.68)/0.16 = 0.046875 \in [0, 0.2) \Rightarrow$ #: **stop**. Output "BA".
Without the terminator the decoder would happily continue producing symbols from the same number.

8.6 Precision limits and practical implementations

The interval width shrinks exponentially: after 100 symbols with $p = 0.5$ it is 2^{-100}, far below double precision ($\approx 2^{-52}$). Practical coders therefore use **integers** and **renormalise**:

- keep low and high as 32-bit integers in $[0, 2^{32})$;

- **E1:** if the interval is in the lower half, output 0 and double it;
- **E2:** if it is in the upper half, output 1, subtract one half and double it;
- **E3:** if it straddles the middle but lies in the middle half $[\frac{1}{4}, \frac{3}{4})$, double around the middle and remember a *pending* bit, which is output (inverted) after the next E1/E2 decision.

This is the classic method of Witten, Neal and Cleary (1987), implemented in `encode_int` and `decode_int` (Lab 4). Integer rounding costs a tiny inefficiency (frequencies must be integers, total $< 2^{30}$).

Fig. 8.2 shows the integer encoder as a block diagram.

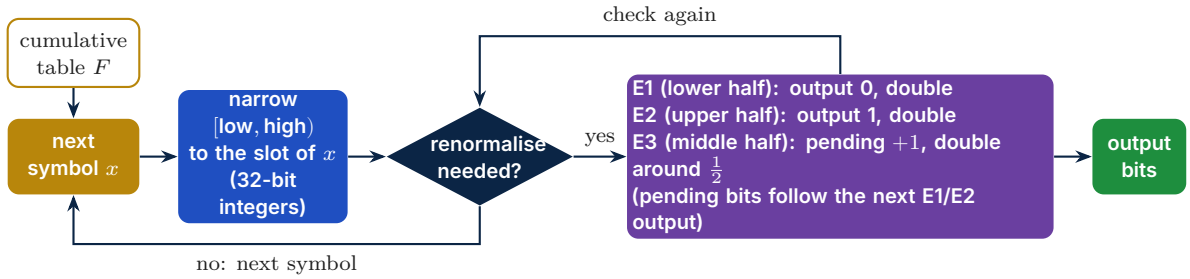


Figure 8.2. Block diagram. Integer arithmetic encoder with E1/E2/E3 renormalisation (Witten, Neal and Cleary, 1987).

Worked Example 8.4 • The integer coder on real data

For a shuffled string of 90 A's and 10 B's: $nH = 100 \times H_2(0.1) = 46.9$ bits. The integer arithmetic coder produced **48 bits**; Huffman needs **100 bits** (one per symbol). On the four data sets of Chapter 10 the arithmetic payload is always within 2 bits of nH.

8.7 Arithmetic coding versus Huffman

	Huffman	Arithmetic
Bits for a message	sum of integer codeword lengths	$\approx -\log_2 P(\text{msg}) + 2$
Overhead per symbol	up to 1 bit	$\approx 2/n$ bits
Skewed binary sources	poor (1 bit/symbol)	excellent
Adaptive models	tree must be rebuilt	easy: change the table
Speed	very fast table look-up	slower (multiplications)
Error sensitivity	error may resynchronise	one bit error ruins the rest
Used in	DEFLATE, JPEG baseline	CABAC (H.264/H.265), JPEG 2000, modern compressors

8.8 Python implementation

Exact teaching coder (full exact and integer coders in Lab 4)

```python
from fractions import Fraction
from math import ceil
def encode_exact(message, table):           # table: s -> (F(s-), F(s))
    low, high = Fraction(0), Fraction(1)
    for s in message:
        w = high - low
        low, high = low + w * table[s][0], low + w * table[s][1]
    k = 1
    while True:                              # shortest tag inside [low, high)
        m = ceil(low * 2**k)
        if Fraction(m + 1, 2**k) <= high:
            return format(m, "0%db" % k)
        k += 1
```

Fragility and leakage

Arithmetic-coded data has almost no redundancy: one flipped bit usually destroys everything after it, so it must be protected by error control (Chapter 14) and integrity checks. The coded length still follows the message probability — an attacker who sees lengths learns $-\log_2 P(\text{message})$ approximately.

8.9 Summary

- Each symbol narrows the interval in proportion to its probability; the final width is $P(\text{message})$.
- About $-\log_2 P + 2$ bits per message: no per-symbol rounding loss.
- The decoder needs the length or a terminator; practical coders use integer scaling (E1, E2, E3).

Solved exercises

Problem 8.1. With $p(A) = 0.8$, $p(B) = 0.2$ (A first), find the interval for AAB.

Solution A: $[0, 0.8)$; A: $[0, 0.64)$; B: $[0.512, 0.64)$. Width $0.128 = 0.8 \cdot 0.8 \cdot 0.2$.

Problem 8.2. How many bits does the tag of a message with probability 0.001 need at most?

Solution $\lceil -\log_2 0.001 \rceil + 1 = \lceil 9.97 \rceil + 1 = 11$ bits.

Problem 8.3. Decode 3 symbols from $v = 0.7$ with A $[0, 0.5)$, B $[0.5, 0.8)$, C $[0.8, 1)$.

Solution $t = 0.7 \to$ B, interval $[0.5, 0.8)$; $t = (0.7 - 0.5)/0.3 = 0.667 \to$ B, interval $[0.65, 0.74)$; $t = (0.7 - 0.65)/0.09 = 0.556 \to$ B. Output BBB.

CHAPTER

9



LZW Compression

IN THIS CHAPTER YOU WILL LEARN

- dictionary-based compression and repeated patterns;
- LZW initialisation, encoding and dictionary updates;
- decoding without sending the dictionary;
- the special case "code not yet in the dictionary";
- dynamic code widths and compression ratio;
- full traces for ABABABA and TOBEORNOTTOBEORTOBEORNOT;
- strengths, limits and applications.

9.1 Dictionary-based compression

Abbreviations in your notes

When a lecture repeats "information theory" many times, you start writing "IT". You invent the abbreviation the first time the phrase appears and use it afterwards. A reader who sees how you wrote the first occurrence can learn the same abbreviation. LZW does this automatically: it builds a dictionary of phrases while reading the text, and the decoder rebuilds the same dictionary while decoding.

Huffman and arithmetic coding need the symbol *probabilities*. Lempel–Ziv methods (LZ77 in 1977, LZ78 in 1978, LZW by Terry Welch in 1984) need nothing: they are **universal** — for long stationary sources their rate approaches the entropy rate, including the effect of memory (repeated words), which a zeroth-order Huffman code cannot see.

9.2 The LZW algorithm

Definition 9.1 • LZW encoder

1. Initialise the dictionary with all single symbols (codes $0, 1, \ldots$; for bytes, code = ASCII value, 0..255).
2. Set w = empty string.
3. For each input symbol c: if $w+c$ is in the dictionary, set $w \leftarrow w+c$. Otherwise output the code of w, add $w+c$ to the dictionary with the next free code, and set $w \leftarrow c$.
4. At the end, output the code of w.

Definition 9.2 • LZW decoder

1. Initialise the same dictionary. Read the first code, output its string, call it w.
2. For each next code k: if k is in the dictionary, entry = dict[k]; if k equals the next free code (not yet known), entry = $w + w[0]$. Output entry; add $w +$ entry[0] to the dictionary; set $w \leftarrow$ entry.

The decoder is always one dictionary entry behind the encoder; the special case covers the only situation in which this matters.

Fig. 9.1 shows the encoder as a flow chart; the decoder mirrors it and rebuilds the same dictionary one step later.

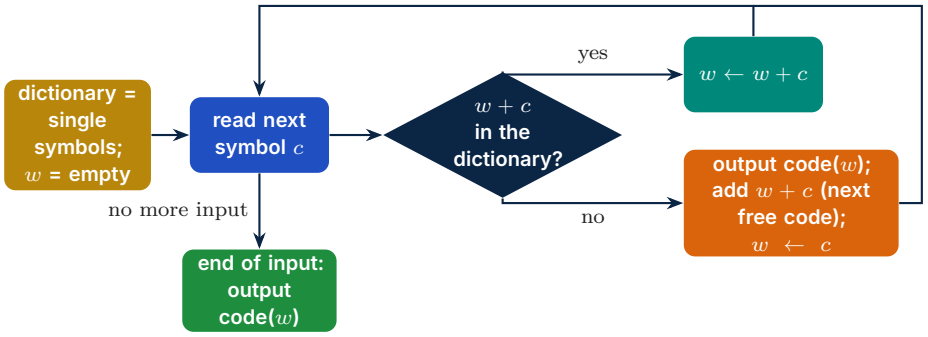


Figure 9.1. Block diagram. LZW encoder. No code table is sent: the decoder builds the same dictionary from the codes.

9.3 Example 1: ABABABA

Worked Example 9.1 • Encoding ABABABA with initial dictionary A = 0, B = 1

Step	Input read	Longest match w	Output	New entry	Code
1	A, B	A	**0**	AB	2
2	B, A	B	**1**	BA	3
3	A, B, A	AB	**2**	ABA	4
4	A, B, A (end)	ABA	**4**	—	

Output codes: **0 1 2 4**. Step 4 uses entry 4 = ABA, which was created only one step earlier.

Worked Example 9.2 • Decoding 0 1 2 4

Code	Known?	Entry	Output so far	New entry	Code
0	yes	A	A	—	
1	yes	B	AB	A+B = AB	2
2	yes	AB	ABAB	B+A = BA	3
4	**no**	$w + w[0]$ = AB+A = ABA	ABABABA	AB+A = ABA	4

Special case. When code 4 arrives, the decoder knows only codes 0–3. It must be the entry the encoder has just created: w followed by its own first letter. This happens for patterns like $cScSc$ (here A B A B A).
Size. With dynamic widths (dictionary sizes 2, 3, 4, 5 when the codes are written) the codes take $1+2+2+3=8$ bits. The text needs 7 bits with a 1-bit fixed code for {A,B} (56 bits in ASCII). **Short inputs do not compress** — the dictionary must first learn.

9.4 Example 2: TOBEORNOTTOBEORTOBEORNOT

Worked Example 9.3 • Encoding with the 256-entry ASCII dictionary

Step	Match w	Output	New entry	Code
1	T	84	TO	256
2	O	79	OB	257
3	B	66	BE	258
4	E	69	EO	259
5	O	79	OR	260
6	R	82	RN	261
7	N	78	NO	262
8	O	79	OT	263
9	T	84	TT	264
10	TO	**256**	TOB	265
11	BE	**258**	BEO	266
12	OR	**260**	ORT	267
13	TOB	**265**	TOBE	268
14	EO	**259**	EOR	269
15	RN	**261**	RNO	270
16	OT	**263**	(end)	

Output: 84 79 66 69 79 82 78 79 84 **256 258 260 265 259 261 263** — 16 codes for 24 characters. From step 10 on, every code stands for 2 or 3 letters.

Worked Example 9.4 • Decoding the 16 codes

Code	Entry output	New entry (w + first letter of entry)	Code
84	T	—	
79	O	TO	256
66	B	OB	257
69	E	BE	258
79	O	EO	259
82	R	OR	260
78	N	RN	261
79	O	NO	262
84	T	OT	263
256	TO	TT	264
258	BE	TOB	265
260	OR	BEO	266
265	TOB	ORT	267
259	EO	TOBE	268
261	RN	EOR	269
263	OT	RNO	270

Concatenating the entries gives TOBEORNOT TOBEORTOBEORNOT ✓. The dictionary was never transmitted.

9.5 Dynamic code widths and compression ratio

Information theory concept • Code width

When the k-th code ($k = 0, 1, \dots$) is written, the dictionary has $n_0 + k$ entries, so it is written with $\lceil \log_2(n_0 + k) \rceil$ bits. Encoder and decoder compute the same widths, so no extra information is needed. Practical formats cap the dictionary (for example at 2^{12} entries in GIF, 12-bit codes) and then freeze or reset it.

Worked Example 9.5 • Bits for TOBEORNOT…

The first code is written with $\lceil \log_2 256 \rceil = 8$ bits, the next 15 codes with $\lceil \log_2 257 \dots 271 \rceil = 9$ bits: $8 + 15 \times 9 = 143$ bits. Original: $24 \times 8 = 192$ bits. Compression ratio $192/143 = 1.34$. With fixed 12-bit codes: $16 \times 12 = 192$ bits — no gain at all. On 1080 characters of repeated TOBEORNOT (Chapter 10) LZW reaches ratio 7.1 because longer and longer phrases enter the dictionary.

9.6 Advantages, limitations and applications

Advantages	Limitations
No statistics or codebook needed; one pass	Poor on short or random data (expansion possible)
Captures repeated phrases (memory of the source)	Dictionary memory grows; must be capped or reset
Simple, fast decoding	Less efficient than modern LZ77+entropy coders

Applications: Unix `compress`, GIF images, TIFF (optional), early modem compression (V.42bis), PDF (LZWDecode filter). Modern formats (ZIP/DEFLATE, gzip, PNG) use LZ77 with Huffman coding instead.

9.7 Python implementation

Encoder and decoder (Lab 5 adds traces and bit costs)

```python
def lzw_encode(text, alphabet):
    d = {s: i for i, s in enumerate(alphabet)}
    w, out = "", []
    for c in text:
        if w + c in d:
            w += c
        else:
            out.append(d[w]); d[w + c] = len(d); w = c
    if w:
        out.append(d[w])
    return out

def lzw_decode(codes, alphabet):
    d = {i: s for i, s in enumerate(alphabet)}
    w = d[codes[0]]; out = [w]
    for k in codes[1:]:
        entry = d[k] if k in d else w + w[0]    # special case cScSc
        d[len(d)] = w + entry[0]
        out.append(entry); w = entry
    return "".join(out)

print(lzw_encode("ABABABA", "AB"))            # [0, 1, 2, 4]
```

Dictionaries remember content

The dictionary contains phrases of the input. Repetitive secrets (a token repeated in many requests) make the output shorter. This is the root of the compression side channels in Chapter 10. Also, an LZW decoder must validate codes: a malicious code larger than the next free code must be rejected (our decoder raises an error) to avoid crashes or memory abuse.

9.8 Summary

- Output the code of the longest known prefix; add prefix + next symbol to the dictionary.
- The decoder rebuilds the dictionary; unknown code $\Rightarrow w + w[0]$.
- Code width grows with the dictionary; gains appear only after the dictionary has learned repeated phrases.

Solved exercises

Problem 9.1. Encode AAAAAAA with dictionary A = 0.

Solution Steps: A→0 (add AA=1); AA→1 (add AAA=2); AAA→2 (add AAAA=3); remaining A → 0. Output 0 1 2 0 (1+1+2+2 = 6 bits with dynamic widths).

Problem 9.2. Decode 0 1 3 with dictionary A = 0, B = 1.

Solution 0 → A; 1 → B, add AB=2; 3 is unknown → $w + w[0]$ = BB, add BB=3. Output ABBB.

Problem 9.3. Why can LZW expand random data?

Solution Random text rarely repeats phrases, so most codes stand for one symbol but cost $\lceil \log_2(\text{dictionary size}) \rceil > 8$ bits after 256 entries. Chapter 10 measures 5.75 bits/symbol for a uniform 16-letter source with $H = 4$.

CHAPTER

10

Comparing Compressors; Compression and Cybersecurity

IN THIS CHAPTER YOU WILL LEARN

- a fair, measured comparison of the four algorithms;
- theoretical code length versus real file size;
- the cost of headers and dictionaries;
- complexity, memory, strengths and weaknesses;
- how compressed lengths leak information;
- traffic fingerprinting and compression side channels;
- defensive design rules.

10.1 How to compare fairly

A fair comparison uses the same input data and counts *everything the decoder needs*:

- **payload:** the coded bits of the message;
- **header:** side information — the code table for Shannon–Fano/Huffman, the frequency table for arithmetic coding. LZW needs no header because the decoder rebuilds the dictionary.

Header convention used in all measurements

16 bits for the number of distinct symbols, then for each symbol 8 bits (the symbol) + 16 bits (its count). From the counts, the decoder can rebuild exactly the same Shannon–Fano, Huffman or arithmetic coder. Real formats use smarter headers (canonical Huffman lengths), so our headers are a simple, honest upper estimate.

Definition 10.1 • Theoretical length versus file size

The entropy bound nH (with H the empirical zeroth-order entropy) and the average code length nL are *theoretical* payload sizes. A compressed *file* also contains headers, padding to whole bytes and format fields. Only payload + header can be compared with the raw size.

Fig. 10.1 shows the measurement set-up behind all results of this chapter.

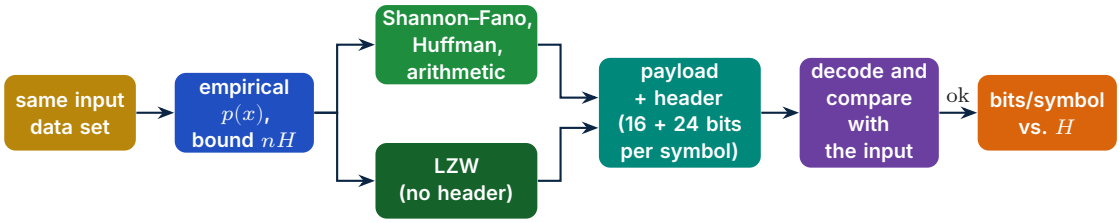


Figure 10.1. Block diagram. A fair comparison: same data, everything the decoder needs is counted, and every output is decoded and checked.

10.2 Measured results

Four data sets were compressed with the code of Labs 2–6 (every result was decoded and checked to be identical to the input). Raw size is 8 bits per character.

Data set (n, H bits/sym)	nH	Shannon–Fano	Huffman	Arithmetic	LZW	Raw
Skewed A–E (1000, 1.641)	1641	$1680 + 136$	$1680 + 136$	$1643 + 136$	2642	8000
English text (1020, 4.122)	4204	$4266 + 640$	$4240 + 640$	$4205 + 640$	4832	8160
Repetitive TOBEORNOT (1080, 2.419)	2613	$2640 + 160$	$2640 + 160$	$2614 + 160$	**1214**	8640
Uniform random 16 letters (1000, 3.994)	3994	$4033 + 400$	$4000 + 400$	$3995 + 400$	5752	8000

Entries "payload + header" in bits. LZW uses dynamic code widths and the 256-entry ASCII start dictionary.

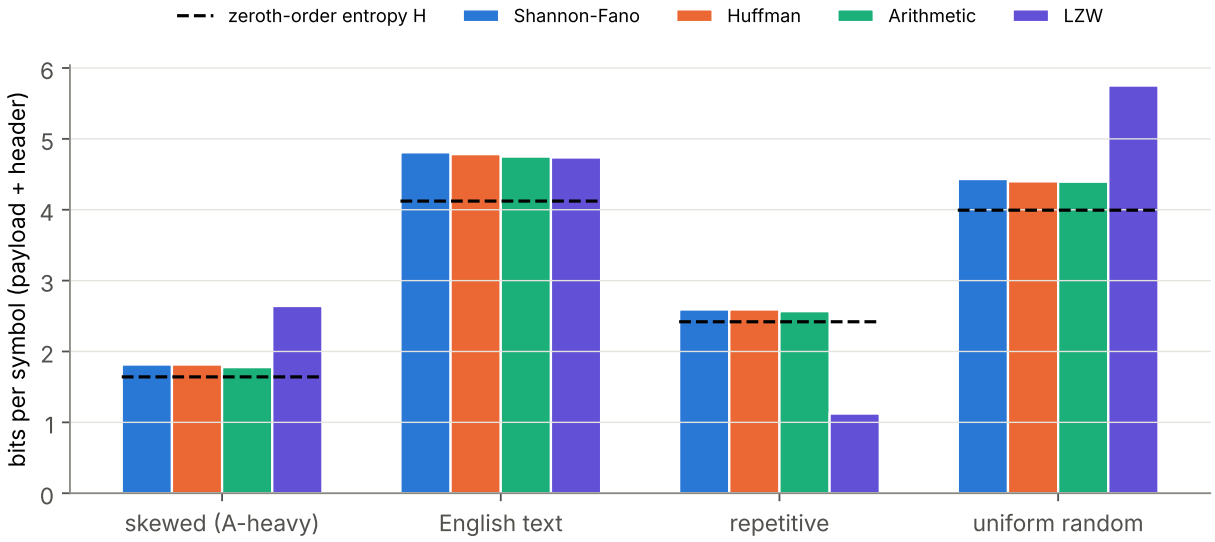


Figure 10.2. Total bits per symbol (payload + header). The dashed line is the zeroth-order entropy of each data set.

Worked Example 10.1 • Reading the results

1. **Arithmetic coding** is always within 2 bits of nH — it reaches the zeroth-order entropy.
2. **Huffman** is within 1 bit/symbol of H and never worse than Shannon–Fano (equal on two sets, better on two).

3. **Headers matter:** for English text the table costs 640 bits, 15% of the payload. For short messages the header can cancel the gain.
4. **LZW** loses on data without repeated phrases (skewed, random): it even expands random data to 5.75 bits/symbol. But on repetitive text it uses only 1.12 bits/symbol, *far below* the zeroth-order entropy 2.42, because it exploits memory (whole repeated words). The entropy that matters for a source with memory is the entropy *rate* (Chapter 11), which is much smaller here.

10.3 Overall comparison

	Shannon–Fano	Huffman	Arithmetic	LZW
Principle	top-down balanced splits	bottom-up merges	interval narrowing	dictionary of phrases
Needs statistics?	yes	yes	yes (or adaptive)	no
Efficiency	near-optimal	optimal symbol code	$\approx$ entropy	$\to$ entropy rate (long inputs)
Exploits memory?	no	no	with context models	yes
Encoding cost	$O(n \log n)$ build	$O(n \log n)$ build	$O(1)$ arithmetic/symbol	$O(1)$ hash lookup/symbol
Memory	code table	code table/tree	frequency table	dictionary (can be large)
Main weakness	not optimal	≥ 1 bit/symbol	slower; fragile	poor on short/random data
Typical use	teaching	DEFLATE, JPEG	video (CABAC), JPEG 2000	GIF, Unix compress

10.4 Compression and cybersecurity

Information theory concept • Compressed length is information

For a good compressor, the compressed length of a message m is about $-\log_2 P(m)$ bits. So the length tells how *probable* — how predictable, how repetitive — the content is. Encryption after compression hides the content but not the length.

10.4.1 Redundancy and predictability

Redundant data is predictable. Compression removes this redundancy before encryption, which is good practice: it reduces the size and removes known structure. But it also creates a new link between *content* and *length*.

10.4.2 Traffic fingerprinting

Each web page, video segment or app action has a characteristic sequence of compressed object sizes. An eavesdropper who sees only encrypted packet sizes can match them against a database of fingerprints (Worked Example 4.4 in Chapter 4 computed the information such sizes carry). Variable bit-rate compressed voice and video can even reveal spoken phrases or watched films.

10.4.3 Compression side channels

Worked Example 10.2 • Length reveals a secret byte (conceptual illustration)

A request contains a secret cookie value and also some text that an attacker can influence. If the attacker's text repeats the secret's beginning, an LZ compressor encodes it as a cheap back-reference, so the message gets shorter. Measured with Python's `zlib` (level 9) on the toy message `GET /?q={guess} HTTP/1.1 Cookie: sessionid=K7Q2XW9P`:

guess	compressed bytes	first new character right?
`sessionid=K`	54	yes
`sessionid=A`	55	no
`sessionid=K5`	55	no
`sessionid=K7`	54	yes
`sessionid=K7Z`	55	no
`sessionid=K7Q`	54	yes

One byte of length difference tells whether the guess extends the secret. This is the principle behind the published CRIME (2012) and BREACH (2013) attacks on compressed TLS/HTTP traffic. (Real attacks need many requests and careful handling of noise; the point here is only the information flow.)

Fig. 10.3 shows the information flow of this attack as a loop.

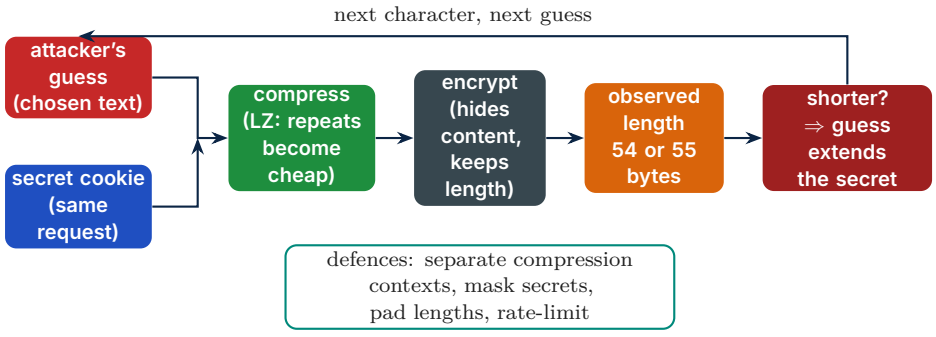


Figure 10.3. **Block diagram.** Compression side channel (the principle of CRIME and BREACH): the length is a channel from the secret to the attacker.

Defensive design considerations

- **Do not compress secrets together with attacker-controlled data** in the same compression context (TLS-level compression was disabled for this reason).
- **Mask or randomise secrets per response** (for example XOR CSRF tokens with a

fresh random value) so repeats do not match.

- **Pad lengths** to fixed buckets, or send constant-size records, to reduce $I(\text{content}; \text{length})$.
- **Rate-limit and monitor** requests that look like adaptive guessing.
- **Remember the trade-off:** padding wastes the bits that compression saved. Information theory quantifies both.

10.5 Summary

- Compare payload + header on the same data; verify lossless round trips.
- Arithmetic $\approx nH$; Huffman $\leq nH + n$; LZW wins only with repeated phrases (memory).
- Compressed length $\approx -\log_2 P(m)$ leaks information about content; separate secrets from attacker data and pad lengths.

Solved exercises

Problem 10.1. A 200-symbol message has $H = 3$ bits/symbol and 20 distinct symbols. Estimate the Huffman file size with our header convention.

Solution Payload between 600 and 800 bits; header $16 + 20 \times 24 = 496$ bits. Total about 1100–1300 bits versus 1600 raw: the header eats most of the gain.

Problem 10.2. Why does LZW beat the zeroth-order entropy on the repetitive data set?

Solution The entropy 2.42 bits treats letters as independent. In TOBEORNOT repeated, each letter is almost determined by the previous ones (entropy rate near 0). LZW's dictionary captures whole words, approaching the much smaller entropy rate.

Problem 10.3. Two encrypted answers "ACCEPTED" and "REJECTED BY POLICY" are equally likely. How much leaks through length, and how to fix it?

Solution Lengths differ, so the whole 1 bit leaks. Pad both to the same length (or to a common bucket) so $H(\text{answer} \mid \text{length}) = 1$ bit.

PART III

Information Sources with Memory

Markov sources, entropy rate, typical sequences, and information-theoretic intrusion and anomaly detection. Modules 6 and 7.

CHAPTER

11



Sources with Memory and Behavioural Security

IN THIS CHAPTER YOU WILL LEARN

- memoryless sources versus sources with memory;
- stationary processes and the entropy rate;
- the AEP and typical sequences;
- first- and higher-order Markov chains;
- transition matrices and stationary distributions;
- sequence probabilities and entropy-rate calculations;
- modelling user and malware behaviour.

11.1 Memoryless sources and sources with memory

Real-world analogy

In English, after the letter Q comes U almost always. After "th" comes a vowel or "r" very often. Letters are not independent: the text has *memory*. A login session is similar: after "login" usually comes "read", rarely "logout" at once. Knowing the past makes the future more predictable, so the true information per symbol is smaller than the single-letter entropy.

Definition 11.1 • Memoryless (i.i.d.) source

A source is **memoryless** if its symbols $X_1, X_2, \ldots$ are independent and identically distributed. Then $H(X_1, \ldots, X_n) = nH(X)$.

Definition 11.2 • Stationary process

A random process is **stationary** if its statistics do not change with time: $P(X_1 = x_1, \ldots, X_n = x_n) = P(X_{1+k} = x_1, \ldots, X_{n+k} = x_n)$ for all shifts k.

Information theory concept • Entropy rate

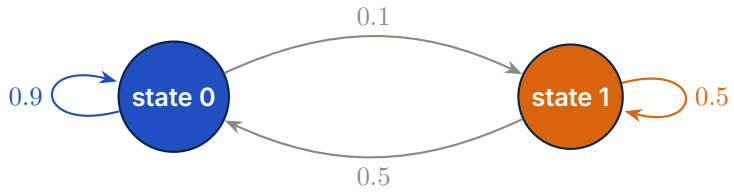
$$\mathcal{H} = \lim_{n\to\infty} \frac{1}{n} H(X_1, \dots, X_n) = \lim_{n\to\infty} H(X_n \mid X_1, \dots, X_{n-1})$$

(both limits exist and are equal for stationary processes). It is the true information per symbol, and the limit of lossless compression for the source. Always $\mathcal{H} \le H(X_1)$.

11.2 Markov chains

Definition 11.3 • First-order Markov chain

$X_1, X_2, \dots$ is a **Markov chain** if the future depends on the past only through the present: $P(X_{n+1} = j \mid X_n = i, X_{n-1}, \dots) = P(X_{n+1} = j \mid X_n = i) = P_{ij}$. The **transition matrix** $\mathbf{P} = [P_{ij}]$ has non-negative entries and every row sums to 1.



$$\mathbf{P} = \begin{bmatrix} 0.9 & 0.1 \\ 0.5 & 0.5 \end{bmatrix}$$

Figure 11.1. A two-state Markov chain (for example "normal" and "busy" periods of a server).

11.2.1 Stationary distribution

Information theory concept • Stationary distribution

A distribution π with $\pi\mathbf{P} = \pi$ and $\sum_i \pi_i = 1$. For an irreducible, aperiodic chain it exists, is unique, and $P(X_n = i) \to \pi_i$ from any start.

Worked Example 11.1 • Two-state chain

Equations: $\pi_0 = 0.9\pi_0 + 0.5\pi_1$ and $\pi_0 + \pi_1 = 1$. From the first, $0.1\pi_0 = 0.5\pi_1$, so $\pi_0 = 5\pi_1$. Then $\pi_1 = \frac{1}{6}$, $\pi_0 = \frac{5}{6}$.

11.2.2 Sequence probability

$$P(x_1, \dots, x_n) = \pi_{x_1} \prod_{k=2}^{n} P_{x_{k-1} x_k} \quad \text{(starting in the stationary distribution).}$$

Worked Example 11.2 • Probability of 0 0 0 1 1 0

$\frac{5}{6} \cdot P_{00}P_{00}P_{01}P_{11}P_{10} = \frac{5}{6} \cdot 0.9 \cdot 0.9 \cdot 0.1 \cdot 0.5 \cdot 0.5 = 0.016875.$

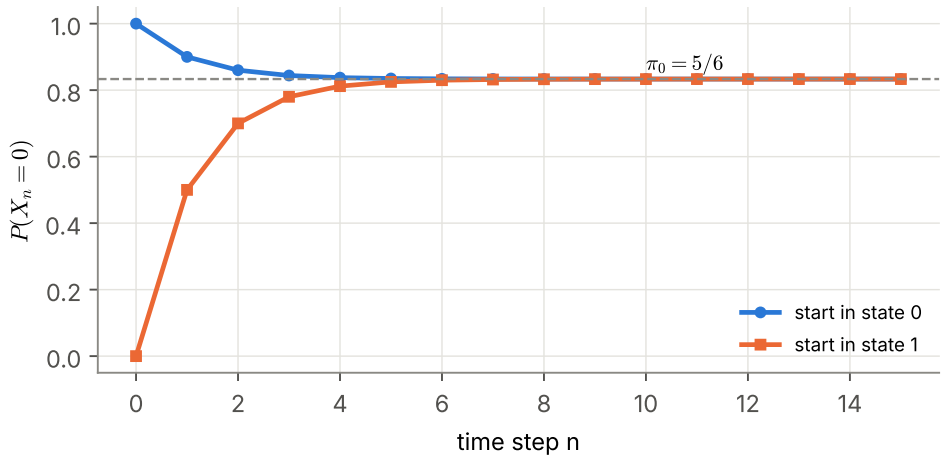


Figure 11.2. $P(X_n = 0)$ converges to $\pi_0 = 5/6$ from both starting states.

11.2.3 Entropy rate of a Markov chain

Derivation • Entropy rate formula

By the Markov property $H(X_n|X_1, \dots, X_{n-1}) = H(X_n|X_{n-1})$. In the stationary regime $P(X_{n-1} = i) = \pi_i$, so

$$\mathcal{H} = H(X_n|X_{n-1}) = \sum_i \pi_i \, H(\mathbf{P}_{i\cdot}) = -\sum_i \pi_i \sum_j P_{ij} \log_2 P_{ij},$$

the average of the row entropies, weighted by the stationary distribution.

Worked Example 11.3 • Entropy rate of the two-state chain

Row 0: $H_2(0.1) = 0.4690$. Row 1: $H_2(0.5) = 1$. $\mathcal{H} = \frac{5}{6}(0.4690) + \frac{1}{6}(1) = 0.3908 + 0.1667 = 0.5575$ bits/symbol.

If we ignored memory and coded each symbol with the stationary probabilities: $H(\pi) = H_2(\frac{1}{6}) = 0.6500$ bits. Memory saves 0.0925 bits/symbol (14%).

Worked Example 11.4 • A three-state weather chain

States Sunny, Cloudy, Rain; $\mathbf{P} = \begin{bmatrix} 0.7 & 0.2 & 0.1 \\ 0.3 & 0.4 & 0.3 \\ 0.2 & 0.3 & 0.5 \end{bmatrix}$.

Stationary distribution. Solving $\pi\mathbf{P} = \pi$, $\sum \pi = 1$ gives $\pi = (\frac{21}{46}, \frac{13}{46}, \frac{12}{46}) = (0.4565, 0.2826, 0.2609)$. Check first equation: $0.7(0.4565) + 0.3(0.2826) + 0.2(0.2609) = 0.3196 + 0.0848 + 0.0522 = 0.4565$ ✓.

Row entropies: 1.1568, 1.5710, 1.4855 bits.

Entropy rate: $\mathcal{H} = 0.4565(1.1568) + 0.2826(1.5710) + 0.2609(1.4855) = 1.3596$ bits/day.

Memoryless view: $H(\pi) = 1.5374$ bits. **Sequence** S S C R: $0.4565 \times 0.7 \times 0.2 \times 0.3 = 0.01917$.

11.2.4 Higher-order Markov chains

In an order-k chain the next symbol depends on the last k symbols: $P(X_{n+1} \mid X_n, \dots, X_{n-k+1})$. It is a first-order chain whose *states* are k-tuples, so all formulas above apply with $|\mathcal{X}|^k$ states. Text models with $k = 2..5$ capture spelling; the price is that $|\mathcal{X}|^k(|\mathcal{X}| - 1)$ probabilities must be estimated from data.

11.3 The Asymptotic Equipartition Property

Information theory concept • AEP (i.i.d. version)

For a memoryless source, $-\frac{1}{n}\log_2 P(X_1, \dots, X_n) \to H(X)$ with probability 1 as $n \to \infty$ (law of large numbers applied to $-\log_2 p(X_i)$). Long sequences are therefore almost always **typical**: their probability is close to 2^{-nH}.

Definition 11.4 • Typical set

$A_\varepsilon^{(n)} = \{x^n : 2^{-n(H+\varepsilon)} \le P(x^n) \le 2^{-n(H-\varepsilon)}\}$. For large n: $P(A_\varepsilon^{(n)}) > 1-\varepsilon$ and $|A_\varepsilon^{(n)}| \approx 2^{nH}$.

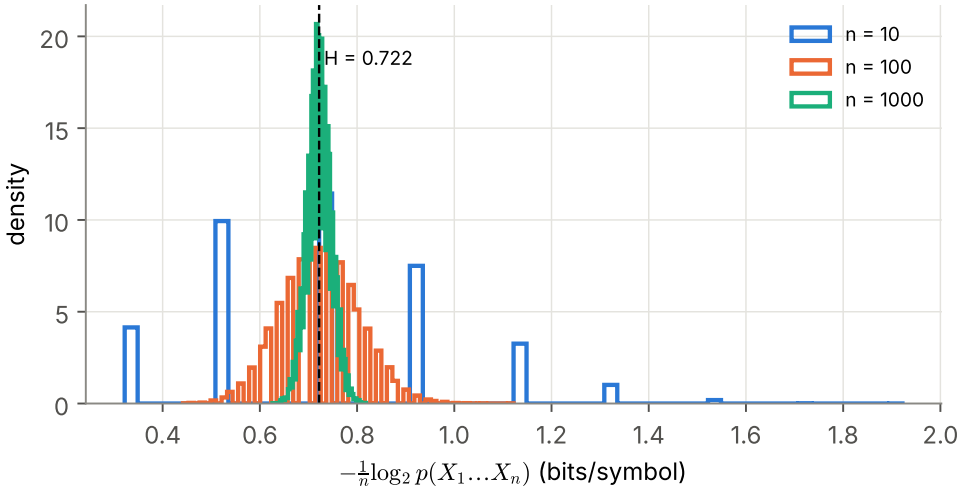


Figure 11.3. AEP for a binary source with $P(1) = 0.2$ ($H = 0.722$): the per-symbol log-probability concentrates around H as n grows (20 000 samples each).

Worked Example 11.5 • How small is the typical set?

Binary source, $P(1) = 0.2$, $n = 100$. All sequences: $2^{100} \approx 1.27 \times 10^{30}$. Typical ones: about $2^{100 \times 0.7219} = 2^{72.2} \approx 5.4 \times 10^{21}$ — a fraction $2^{-27.8}$ of all sequences, yet they carry almost all the probability. Indexing only typical sequences needs ≈ 72 bits instead of 100: this is the source coding theorem in another form. The AEP also holds for stationary ergodic sources with H replaced by the entropy rate.

11.4 Estimating a Markov model from data

Information theory concept • Maximum likelihood estimate

Count transitions N_{ij} (how often j follows i). Then $\hat{P}_{ij} = N_{ij} / \sum_k N_{ik}$. With **Laplace smoothing**, $\hat{P}_{ij} = (N_{ij} + \alpha)/(\sum_k N_{ik} + \alpha S)$, which avoids zero probabilities for unseen transitions.

Worked Example 11.6 • Estimating from a short sequence

Sequence 0 0 1 0 0 0 1 1 0 0 has 9 transitions: from 0: $0 \to 0$ four times, $0 \to 1$ twice; from 1: $1 \to 0$ twice, $1 \to 1$ once. $\hat{\mathbf{P}} = \begin{bmatrix} 4/6 & 2/6 \\ 2/3 & 1/3 \end{bmatrix}$. With 20 000 samples from the chain of Fig. 11.1, Lab 9 obtains $\begin{bmatrix} 0.898 & 0.102 \\ 0.495 & 0.505 \end{bmatrix}$, close to the truth.

Fig. 11.4 shows how a Markov model is learned from a log and what it is used for.

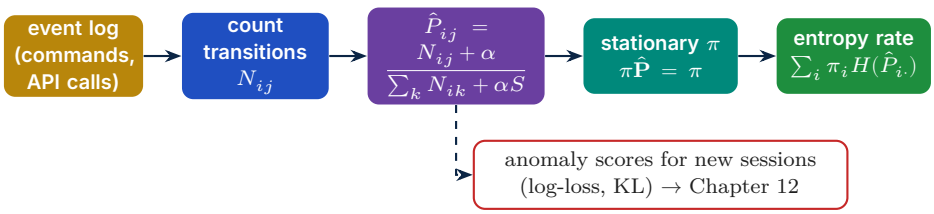


Figure 11.4. Block diagram. Learning a Markov model of behaviour from data.

11.5 Cybersecurity focus: behaviour as a source

User and malware behaviour

- **User behaviour modelling.** Commands, URLs or API calls form sequences with strong memory. A per-user Markov model describes "normal" sequences; the entropy rate tells how predictable the user is.
- **Malware behaviour.** Malware often has very regular (low-entropy) system-call loops, or unusual transitions (open → encrypt → write repeated thousands of times in ransomware).
- **Sequence analysis.** The probability of an observed session under the normal model, or its log-loss per symbol, measures how *surprising* it is. Surprise far above the entropy rate signals a statistical deviation (Chapter 12).

11.6 Module summary

Key points

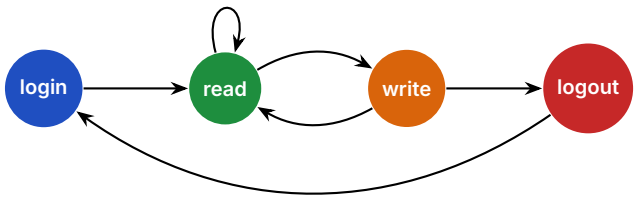
- Sources with memory have entropy rate $\mathcal{H} \leq H(X_1)$; compression and prediction should use it.

- Markov chain: $\pi\mathbf{P} = \pi$; $\mathcal{H} = \sum_i \pi_i H(\mathbf{P}_{i\cdot})$.
- AEP: long sequences have probability $\approx 2^{-n\mathcal{H}}$; about $2^{n\mathcal{H}}$ of them matter.
- Transition counts give maximum-likelihood models of behaviour.

Formula sheet

Entropy rate	$\mathcal{H} = \lim \frac{1}{n} H(X_1..X_n)$
Stationary	$\pi\mathbf{P} = \pi,\ \sum \pi_i = 1$
Markov entropy rate	$\mathcal{H} = -\sum_i \pi_i \sum_j P_{ij} \log_2 P_{ij}$
Sequence probability	$\pi_{x_1} \prod P_{x_{k-1} x_k}$
MLE	$\hat{P}_{ij} = N_{ij}/N_i$
AEP	$-\frac{1}{n}\log_2 P(X^n) \to H,\ \ \lvert A_\varepsilon^{(n)} \rvert \approx 2^{nH}$

Visual recap • Module 6



behaviour = Markov source
$\mathcal{H} = \sum \pi_i H(\text{row } i)$

Review questions

1. Why is the entropy rate never larger than the single-symbol entropy?
2. What does each row of a transition matrix represent?
3. Explain the AEP in one sentence.
4. Why is smoothing needed when estimating transition probabilities?

Solved exercises

Problem 11.1. Find the stationary distribution of $\mathbf{P} = \begin{bmatrix} 0.6 & 0.4 \\ 0.2 & 0.8 \end{bmatrix}$ and the entropy rate.

Solution $0.4\pi_0 = 0.2\pi_1 \Rightarrow \pi = (\frac{1}{3}, \frac{2}{3})$. $\mathcal{H} = \frac{1}{3}H_2(0.4) + \frac{2}{3}H_2(0.2) = \frac{1}{3}(0.9710) + \frac{2}{3}(0.7219) = 0.8049$ bits.

Problem 11.2. A chain alternates deterministically 0,1,0,1,…. What is its entropy rate?

Solution $P_{01} = P_{10} = 1$: every row has entropy 0, so $\mathcal{H} = 0$, although $H(\pi) = 1$ bit.

Problem 11.3. For a binary i.i.d. source with $P(1) = 0.1$ and $n = 1000$, how many typical sequences are there approximately?

Solution $2^{1000 \times 0.469} = 2^{469}$ of the 2^{1000} sequences.

CHAPTER

12



Intrusion Detection and Anomaly Detection

IN THIS CHAPTER YOU WILL LEARN

- statistical anomaly detection with empirical and model distributions;
- rebuilding Markov models from logs (maximum likelihood);
- entropy-, KL- and log-loss-based anomaly scores;
- thresholds, false positives and false negatives;
- a complete simulated detector with measured error rates;
- IDS/IPS, SIEM and SOC use.

12.1 Statistical anomaly detection

The bank teller

A bank teller does not know every fraud trick, but she knows what a *normal* day looks like. A customer who withdraws ten times in an hour is suspicious because the behaviour is *improbable* under the normal pattern. Anomaly detection learns "normal" as a probability model and flags what is surprising.

Two families of intrusion detection exist:

- **signature-based:** match known attack patterns (precise, but blind to new attacks);
- **anomaly-based:** learn normal behaviour and flag deviations (can find new attacks, but produces false alarms).

Information theory gives the anomaly-based family its natural scores: surprise, entropy and divergence.

12.2 Empirical and model distributions

Definition 12.1 • Empirical distribution

For a window of W observed symbols with counts n_x, the empirical distribution is $\hat{p}(x) = n_x/W$. With add-α smoothing, $\hat{p}(x) = (n_x + \alpha)/(W + \alpha|\mathcal{X}|)$, so no probability is zero.

Definition 12.2 • Model (baseline) distribution

A distribution q (or a Markov model $\hat{\mathbf{P}}$) learned from training data that is believed to be normal, using the maximum-likelihood estimates of Chapter 11.

12.3 Three information-theoretic scores

Information theory concept • Anomaly scores

1. **Entropy of a window:** $H(\hat{p})$. Very low entropy (one action repeated, e.g. a scan or brute force) or very high entropy (random-looking traffic, e.g. encrypted exfiltration or DGA domain names) is suspicious.
2. **KL divergence to the baseline:** $D(\hat{p}\,\|\,q)$ — how far the window's frequencies are from normal.
3. **Log-loss under a Markov model:** $\ell = -\frac{1}{W-1}\sum_k \log_2 \hat{P}_{x_{k-1}x_k}$ bits/action — the average surprise of the *sequence*. For normal data it is close to the entropy rate.

Derivation • Why KL is the right score

Suppose the window has W i.i.d. symbols. The probability of observing empirical distribution $\hat{p}$ when the true one is q is approximately $2^{-W\,D(\hat{p}\|q)}$ (the *method of types*, Sanov's theorem). A large D means the window is exponentially unlikely under normal behaviour. Similarly, the log-likelihood ratio test between "normal" and "attack" models compares cross-entropies, which is what the log-loss score measures.

Fig. 12.1 shows the complete detector. The upper path is done once (training on normal data); the lower path runs for every new window.

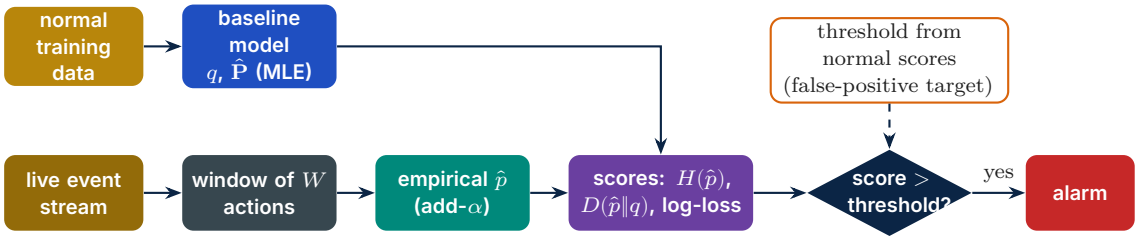


Figure 12.1. Block diagram. Information-theoretic anomaly detector: training path (top) and detection path (bottom).

12.4 A worked detector

Normal model. A user performs four actions: login (0), read (1), write (2), logout (3), following the Markov chain

$$\mathbf{P}_{\text{normal}} = \begin{bmatrix} 0.05 & 0.80 & 0.10 & 0.05 \\ 0.02 & 0.60 & 0.25 & 0.13 \\ 0.02 & 0.50 & 0.30 & 0.18 \\ 0.90 & 0.04 & 0.03 & 0.03 \end{bmatrix},$$

entropy rate 1.319 bits/action. From 20 000 training actions, the smoothed MLE baseline is close to the truth, and the stationary action frequencies are $q = (0.128,\ 0.541,\ 0.212,\ 0.118)$.

Worked Example 12.1 • KL score of a suspicious window

A 50-action window has 15 logins, 15 reads, 15 writes, 5 logouts. With add-0.5 smoothing: $\hat{p} = (15.5, 15.5, 15.5, 5.5)/52 = (0.2981, 0.2981, 0.2981, 0.1058)$.

$$D(\hat{p}\|q) = 0.2981 \log_2 \tfrac{0.2981}{0.128} + 0.2981 \log_2 \tfrac{0.2981}{0.541} + 0.2981 \log_2 \tfrac{0.2981}{0.212} + 0.1058 \log_2 \tfrac{0.1058}{0.118} = 0.235 \text{ bits.}$$

A typical normal window (6, 27, 11, 6) gives $D = 0.0008$ bits. With the threshold 0.200 chosen below, the first window is flagged.

Worked Example 12.2 • Log-loss of two short sessions

Normal-looking login, read, read, write, read, logout: surprises $-\log_2 P = 0.322,\ 0.737,\ 2.000,\ 1.000,\ 2.943$; average 1.40 bits/action, near the entropy rate 1.32.
Brute-force-like login, login, login, login, logout, login: surprises 4.322 (four times) and 0.152; average 3.49 bits/action — 2.6 times the normal level.

12.5 Thresholds, false positives and false negatives

Definition 12.3 • Detection errors

- **False positive** (false alarm): a normal window is flagged. Rate = FP/(all normal windows).
- **False negative** (miss): an attack window is not flagged. Rate = FN/(all attack windows).

Raising the threshold reduces false positives and increases misses.

A common rule: choose the threshold as a high percentile (here the 99th) of the scores of *normal validation* windows. Then about 1% false alarms are expected, without needing attack data.

Worked Example 12.3 • Measured performance (Lab 11)

Attack behaviour: repeated logins and unusually much writing (matrix in Lab 11). 200 normal and 200 attack test windows of 50 actions; thresholds from 200 normal validation windows.

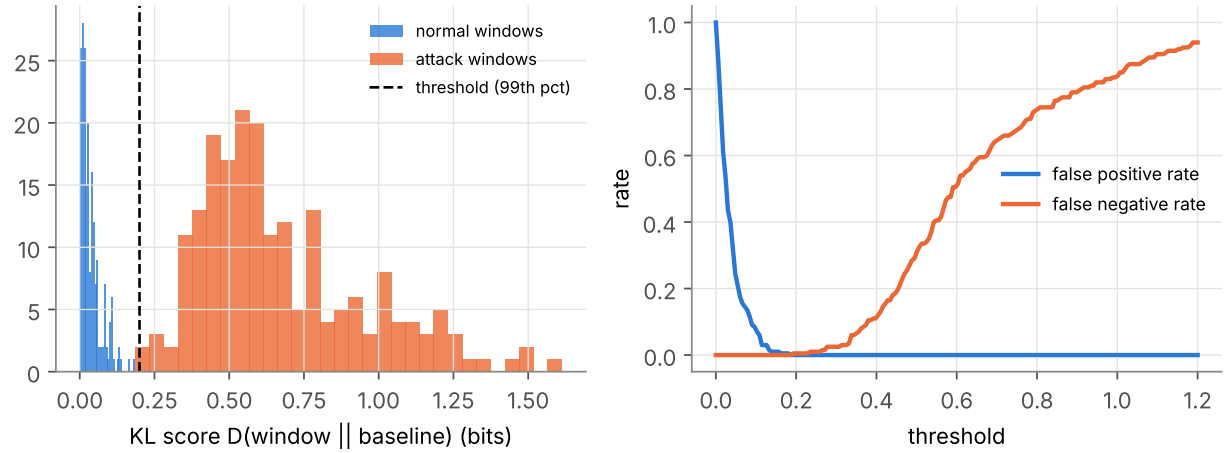


Figure 12.2. Left: KL scores of 200 normal and 200 attack windows ($W = 50$). Right: false-positive and false-negative rates versus threshold.

Score	threshold (99th pct)	false-positive rate	false-negative rate
KL of action frequencies	0.200 bits	0.000	0.005
Markov log-loss	1.695 bits/action	0.000	0.010

This attack is easy to see. Subtler attacks (mimicking normal frequencies) defeat the frequency-based KL score but still change *transitions*, which the Markov log-loss captures. In real systems, base rates matter: if 1 window in 10 000 is an attack, even a 0.5% false-positive rate means about 50 false alarms per real attack.

12.6 Information-theoretic behavioural analysis in practice

IDS, IPS, SIEM and SOC

- **IDS/IPS** (intrusion detection/prevention systems) can compute entropy of packet fields (source IPs, ports): a DDoS makes source entropy jump; a port scan makes destination-port entropy jump.
- **SIEM** platforms aggregate logs and can compute per-user baselines and KL/log-loss scores on sliding windows.
- **SOC** analysts receive alerts ranked by score; information-theoretic scores are explainable ("this session is 2.6 times more surprising than normal").
- Baselines drift: users change habits. Models must be re-estimated, and attackers may slowly poison the baseline.

Python: scoring a window (full simulation in Lab 11)

```python
from math import log2
def freq(window, S=4, alpha=0.5):
    c = [alpha] * S
    for a in window:
        c[a] += 1
    return [x / sum(c) for x in c]
def kl(p, q):
    return sum(pi * log2(pi / qi) for pi, qi in zip(p, q) if pi > 0)
q = [0.128, 0.541, 0.212, 0.118]
w = [0] * 15 + [1] * 15 + [2] * 15 + [3] * 5
print(round(kl(freq(w), q), 3))           # 0.235 -> alarm if > 0.200
```

12.7 Module summary

Key points

- Learn normal as a distribution or Markov model; flag what is improbable.
- Scores: window entropy, $D(\hat{p}\|q)$, and log-loss (cross-entropy rate).
- Thresholds from normal validation data control the false-positive rate; misses depend on the attack.

Formula sheet

Smoothed empirical	$\hat{p}(x) = (n_x + \alpha)/(W + \alpha\|\mathcal{X}\|)$
KL score	$D(\hat{p}\|q) = \sum \hat{p} \log_2(\hat{p}/q)$
Log-loss	$\ell = -\frac{1}{W-1} \sum \log_2 \hat{P}_{x_{k-1}x_k}$
Error rates	$\mathrm{FPR} = \mathrm{FP}/\mathrm{normal}, \;\; \mathrm{FNR} = \mathrm{FN}/\mathrm{attacks}$
Method of types	$P(\hat{p} \mid q) \approx 2^{-W D(\hat{p}\|q)}$

Review questions

1. Why is smoothing essential before computing $D(\hat{p}\|q)$?
2. Which score detects an attacker who keeps normal frequencies but changes the order of actions?
3. How does the base rate of attacks affect the usefulness of a detector?

Solved exercises

Problem 12.1. Baseline $q = (0.5, 0.5)$. A window has empirical $\hat{p} = (0.9, 0.1)$. Compute the KL score.

Solution $0.9 \log_2 1.8 + 0.1 \log_2 0.2 = 0.7632 - 0.2322 = 0.5310$ bits.

Problem 12.2. A detector has FPR 1% and FNR 10%. Out of 10 000 normal and 10 attack windows, how many alarms are true?

Solution False alarms 100, detected attacks 9: only $9/109 = 8.3\%$ of alarms are real.

Problem 12.3. The entropy of destination ports in one minute rises from 2 bits to 9 bits. Interpret.

Solution Traffic is spread over about $2^9 = 512$ equally used ports instead of about 4: typical of a port scan.

PART IV

Channels, Noise and Reliable Communication

Noisy channels, capacity, error detection and correction: Hamming, SECDED, repetition and BCH codes. Modules 8 and 9.

CHAPTER

13

Channels, Noise and Secure Communication

IN THIS CHAPTER YOU WILL LEARN

- the communication system model;
- discrete memoryless channels and transition matrices;
- BSC and BEC, worked in full;
- mutual information across a channel and capacity;
- Shannon's noisy-channel coding theorem;
- the data processing inequality;
- reliability versus confidentiality; the wiretap channel.

13.1 The communication system model

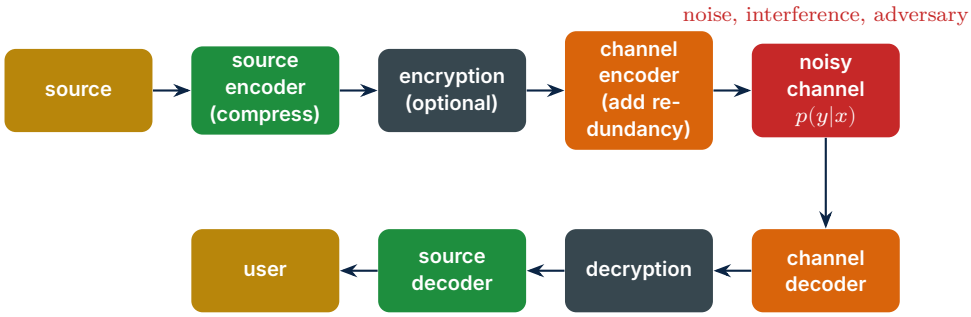


Figure 13.1. Shannon's model with security added. Source coding removes redundancy (Part II); channel coding adds controlled redundancy (Chapter 14).

13.2 Discrete memoryless channels

Definition 13.1 • Discrete memoryless channel (DMC)

A DMC has input alphabet $\mathcal{X}$, output alphabet $\mathcal{Y}$ and transition probabilities $p(y|x)$, written as a **channel matrix** $\mathbf{W}$ with rows x and columns y (each row sums to 1). *Memoryless*: each use is independent, $p(y_1..y_n|x_1..x_n) = \prod_i p(y_i|x_i)$.

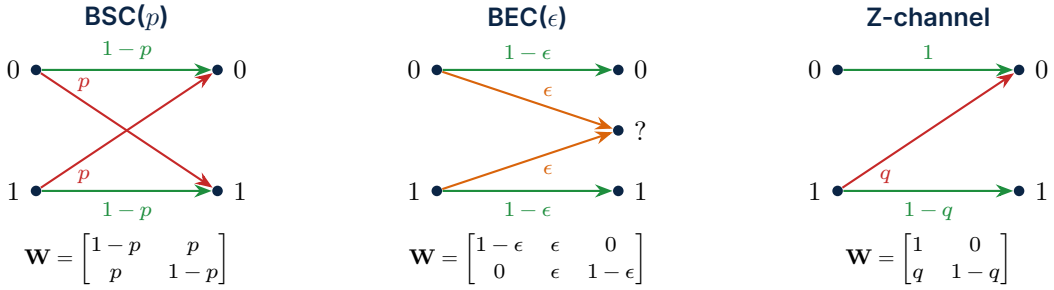


Figure 13.2. Three binary-input channels: symmetric errors, erasures, and one-sided errors.

13.3 Mutual information across a channel

For an input distribution $p(x)$, the output distribution is $p(y) = \sum_x p(x)p(y|x)$ and

$$I(X;Y) = H(Y) - H(Y|X), \qquad H(Y|X) = \sum_x p(x) H(\mathbf{W}_{x\cdot}).$$

$H(Y|X)$ is the *noise entropy*; $H(Y)$ is the uncertainty of the output. The difference is what gets through.

Worked Example 13.1 • BSC with a non-uniform input

BSC(0.1), $p(X = 1) = 0.2$. Output: $p(Y = 1) = 0.8(0.1) + 0.2(0.9) = 0.26$, $H(Y) = H_2(0.26) = 0.8267$. Noise: $H(Y|X) = H_2(0.1) = 0.4690$. $I(X;Y) = 0.8267 - 0.4690 = 0.3577$ bits per use. With a uniform input $H(Y) = 1$ and $I = 0.5310$: better.

13.4 Channel capacity

Definition 13.2 • Capacity

$$C = \max_{p(x)} I(X;Y) \quad \text{bits per channel use.}$$

Derivation • Capacities of the BSC and BEC

BSC. $I = H(Y) - H_2(p) \le 1 - H_2(p)$, with equality for a uniform input (then Y is uniform). So $C_{\text{BSC}} = 1 - H_2(p)$.

BEC. $H(X|Y)$: if Y is not erased, X is known; if erased (probability ϵ), X keeps its uncertainty $H(X)$. So $H(X|Y) = \epsilon H(X)$ and $I = (1-\epsilon)H(X) \le 1 - \epsilon$. Hence $C_{\text{BEC}} = 1 - \epsilon$.

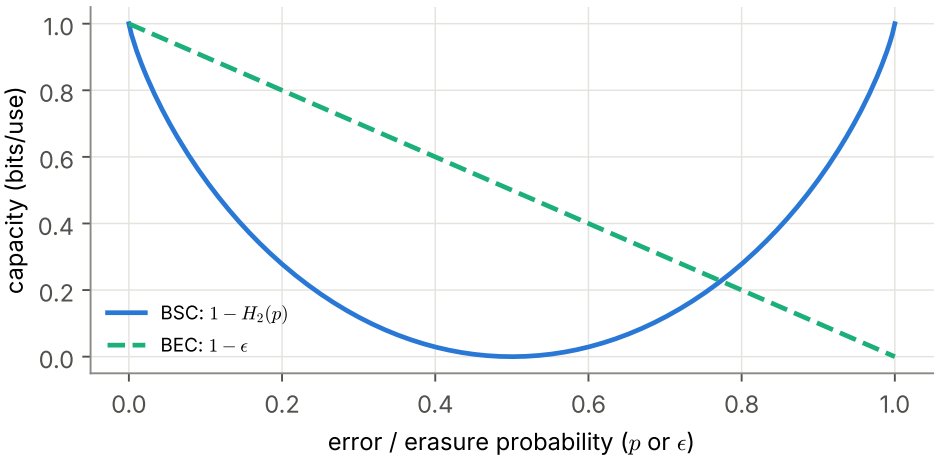


Figure 13.3. Capacity of the BSC and BEC. A BSC with $p = 0.5$ is useless; with $p > 0.5$ the receiver simply inverts the bits.

Worked Example 13.2 • Capacity calculations

Channel	formula	capacity
BSC, $p = 0.01$	$1 - H_2(0.01)$	0.9192
BSC, $p = 0.11$	$1 - H_2(0.11)$	0.5001
BEC, $\epsilon = 0.3$	$1 - 0.3$	0.7
Z-channel, $q = 0.5$	Blahut–Arimoto, best $P(X = 0) = 0.6$	$\log_2 \frac{5}{4} = 0.3219$
$\mathbf{W} = \left[\begin{smallmatrix} 0.9 & 0.1 \\ 0.2 & 0.8 \end{smallmatrix}\right]$	Blahut–Arimoto, best $P(X = 0) = 0.518$	0.3978

For the last channel, a uniform input gives $I = 0.3973$: very close, but the optimum is slightly non-uniform because errors are not symmetric. (The Blahut–Arimoto algorithm in Lab 10 computes the capacity of any DMC.)

Fig. 13.4 shows the Blahut–Arimoto algorithm used by Lab 10 to compute the capacity of any discrete memoryless channel.

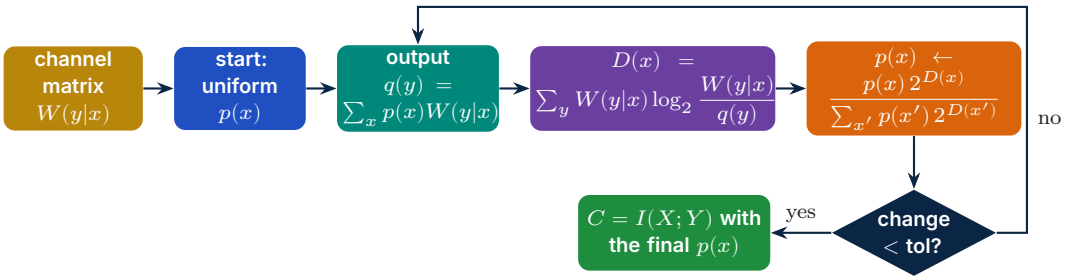


Figure 13.4. Block diagram. Blahut–Arimoto: inputs that "see" a more informative output distribution get more probability, until the input distribution stops changing.

13.5 Shannon's noisy-channel coding theorem

Information theory concept • Channel coding theorem

For every rate $R < C$ and every $\delta > 0$ there exist codes of rate R and long enough block length with error probability below δ. For $R > C$ the error probability cannot go to zero.

In words: noise does not limit *reliability*, it limits *speed*. Below capacity, clever redundancy can make errors as rare as we want. Chapter 14 shows simple codes; modern codes (LDPC, turbo, polar) come close to C.

13.6 The data processing inequality

Information theory concept • Data processing inequality (DPI)

If $X \to Y \to Z$ is a Markov chain (Z depends on X only through Y), then $I(X;Z) \le I(X;Y)$. Processing cannot create information about X.

Worked Example 13.3 • Two BSCs in a row

Two BSC(0.1) in cascade form a BSC with $p = 0.1(0.9) + 0.9(0.1) = 0.18$. Uniform input: $I(X;Y) = 0.5310$ after the first, $I(X;Z) = 1 - H_2(0.18) = 0.3199$ after both. Each relay without decoding loses information.

DPI for defenders

An attacker who only sees a processed version Z of a leak Y (aggregated logs, rounded timings, coarse sizes) cannot learn more than from Y itself: $I(S;Z) \le I(S;Y)$. Adding noise or coarsening observables is a provable way to reduce leakage.

13.7 Reliability versus confidentiality

Error correction and encryption have opposite goals: channel coding makes the message easy to recover *for everyone* who receives it; encryption makes it hard to recover for anyone without the key. A noisy channel helps nobody keep secrets by itself — unless the eavesdropper's channel is *worse* than the legitimate receiver's.

Information theory concept • Wiretap channel and secrecy capacity

Alice sends to Bob over a main channel; Eve listens through a noisier channel. For degraded binary symmetric channels with crossover $p_B < p_E$, the secrecy capacity is

$$C_s = C_B - C_E = H_2(p_E) - H_2(p_B).$$

Rates below C_s can be both reliable for Bob and secret from Eve, without a shared key (Wyner, 1975).

Worked Example 13.4 • Secrecy capacity

$p_B = 0.05$, $p_E = 0.20$: $C_s = H_2(0.2) - H_2(0.05) = 0.7219 - 0.2864 = 0.4355$ bits per use. If Eve's channel were as good as Bob's, $C_s = 0$.

VPNs and adversarial interference

- A VPN builds an encrypted tunnel over a noisy, untrusted network: lower layers provide reliability (error correction, retransmission); the VPN layer provides confidentiality and integrity (encryption and MACs). Packet sizes and timing still leak (Chapters 4 and 10).
- An **adversary** is not random noise: a jammer can target the moments or bits that hurt most, so capacity results for random noise are optimistic. Spread spectrum, interleaving and authenticated retransmissions help.
- Integrity needs more than error correction: an attacker can change a codeword into another *valid* codeword. Cryptographic MACs detect deliberate changes (Chapter 14).

13.8 Module summary

Key points

- A DMC is a matrix $p(y|x)$; $I(X;Y) = H(Y) - H(Y|X)$; $C = \max I$.
- $C_{\text{BSC}} = 1 - H_2(p)$, $C_{\text{BEC}} = 1 - \epsilon$; Blahut–Arimoto for general channels.
- Below capacity, reliable communication is possible; processing never increases information.
- Secrecy is possible from channel advantage: $C_s = H_2(p_E) - H_2(p_B)$ for BSCs.

Formula sheet

Output distribution	$p(y) = \sum_x p(x)p(y\|x)$
Information through channel	$I(X;Y) = H(Y) - H(Y\|X)$
Capacity	$C = \max_{p(x)} I(X;Y)$
BSC, BEC	$1 - H_2(p)$, $1 - \epsilon$
Cascade of BSCs	$p = p_1(1-p_2) + p_2(1-p_1)$
DPI	$X \to Y \to Z \Rightarrow I(X;Z) \le I(X;Y)$
Secrecy capacity (BSC)	$C_s = H_2(p_E) - H_2(p_B)$

Review questions

1. Why is a BSC with $p = 0.9$ as good as one with $p = 0.1$?
2. Which input distribution achieves capacity on symmetric channels?
3. State the DPI and give a security interpretation.

4. Why does error correction not provide integrity against an attacker?

Solved exercises

Problem 13.1. Find the capacity of a BSC with $p = 0.25$.

Solution $1 - H_2(0.25) = 1 - 0.8113 = 0.1887$ bits/use.

Problem 13.2. A BEC erases 40% of the bits. At most how many information bits can 1000 channel uses carry reliably?

Solution $1000 \times 0.6 = 600$ bits.

Problem 13.3. Is a code of rate 0.6 possible on a BSC with $p = 0.11$?

Solution No: $C = 0.5 < 0.6$, so errors cannot be made arbitrarily small.

CHAPTER

14



Error, Reliability and Integrity

IN THIS CHAPTER YOU WILL LEARN

- error detection versus error correction;
- Hamming distance, minimum distance, and $t = \lfloor (d_{\min} - 1)/2 \rfloor$;
- Hamming (7,4): parity positions, $\mathbf{G}$, $\mathbf{H}$, syndromes;
- extended Hamming (8,4) SECDED;
- the (5,1) repetition code: correcting two errors;
- BCH(15,7): two-error correction over GF(16);
- implementations, simulations and integrity.

14.1 Error detection and correction (Chapter 9.1 of the plan)

Real-world analogy

When you spell your name on the phone you add redundancy: "M as in Mary". If the listener hears "N as in Mary", she *detects* an error. If you spell every letter three times, she can even *correct* it by majority. Error-control codes add redundancy in the same spirit, but with mathematics that gets the most protection from the fewest extra bits.

Definition 14.1 • Detection, correction, code rate

An (n, k) block code maps k data bits to n code bits; $n - k$ bits are **redundancy** and $R = k/n$ is the **code rate**. **Error detection** recognises that a received word is not a codeword. **Error correction** also finds the most likely sent codeword.

Definition 14.2 • Hamming distance and minimum distance

$d(\mathbf{a}, \mathbf{b})$ = number of positions where $\mathbf{a}$ and $\mathbf{b}$ differ. $d_{\min}$ = smallest distance between two different codewords (for linear codes: smallest weight of a non-zero codeword).

Information theory concept • Capabilities

A code with minimum distance $d_{\min}$ can

$$\text{detect up to } d_{\min}-1 \text{ errors}, \qquad \text{correct up to } t=\left\lfloor \frac{d_{\min}-1}{2} \right\rfloor \text{ errors},$$

or, for example, correct t' and at the same time detect $t'+1$ errors if $2t'+1 < d_{\min}$.

Worked Example 14.1 • Using the formula

Code	(n,k)	R	$d_{\min}$	detects	corrects t
single parity bit	(8,7)	0.875	2	1	0
Hamming	(7,4)	0.571	3	2	1
extended Hamming	(8,4)	0.5	4	3	1 (+ detect 2)
repetition	(5,1)	0.2	5	4	2
BCH	(15,7)	0.467	5	4	2

Example for (5,1): $t=\lfloor (5-1)/2 \rfloor = 2$.

Error patterns. An error pattern $\mathbf{e}$ has a 1 where a bit flips: $\mathbf{r}=\mathbf{c}\oplus\mathbf{e}$. A *single-bit error* has weight 1; burst errors hit neighbouring bits. Codes are guaranteed only for patterns of weight $\le t$; heavier patterns may be corrected, detected, or silently miscorrected.

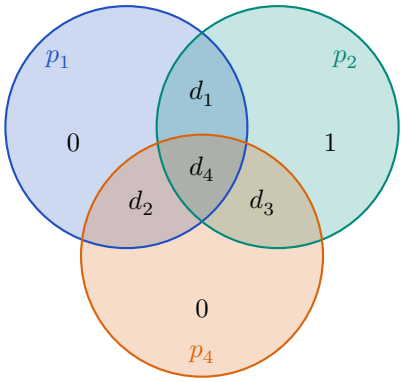
14.2 Hamming codes (Chapter 9.2)

14.2.1 Parity bits at power-of-two positions

Number the 7 positions 1 to 7. Positions 1, 2 and 4 (powers of two) hold parity bits; 3, 5, 6, 7 hold the data bits $d_1..d_4$. Parity bit p_j checks every position whose binary number contains j:

Position	1	2	3	4	5	6	7
Binary	001	010	011	100	101	110	111
Content	p_1	p_2	d_1	p_4	d_2	d_3	d_4
p_1 checks	✓		✓		✓		✓
p_2 checks		✓	✓			✓	✓
p_4 checks				✓	✓	✓	✓

With even parity: $p_1=d_1\oplus d_2\oplus d_4$, $p_2=d_1\oplus d_3\oplus d_4$, $p_4=d_2\oplus d_3\oplus d_4$.



Each circle must contain an even number of 1s. For data 1011 ($d_1 = 1, d_2 = 0, d_3 = 1, d_4 = 1$) the parity bits are $p_1 = 0$, $p_2 = 1$, $p_4 = 0$ (small numbers). A single flipped bit makes exactly the circles that contain it odd, and that pattern points to the bit.

Figure 14.1. Hamming (7,4) as three overlapping parity circles.

14.2.2 Generator and parity-check matrices

With the codeword order $(p_1, p_2, d_1, p_4, d_2, d_3, d_4)$ and row vectors, $\mathbf{c} = \mathbf{d}\,\mathbf{G}$ and $\mathbf{H}\mathbf{c}^T = \mathbf{0}$:

$$\mathbf{G} = \begin{bmatrix} 1 & 1 & 1 & 0 & 0 & 0 & 0 \\ 1 & 0 & 0 & 1 & 1 & 0 & 0 \\ 0 & 1 & 0 & 1 & 0 & 1 & 0 \\ 1 & 1 & 0 & 1 & 0 & 0 & 1 \end{bmatrix}, \qquad \mathbf{H} = \begin{bmatrix} 1 & 0 & 1 & 0 & 1 & 0 & 1 \\ 0 & 1 & 1 & 0 & 0 & 1 & 1 \\ 0 & 0 & 0 & 1 & 1 & 1 & 1 \end{bmatrix}.$$

Column j of $\mathbf{H}$ (read bottom-up as $s_4 s_2 s_1$) is the binary number j. Therefore, if only bit j is wrong, the syndrome $\mathbf{s} = \mathbf{H}\mathbf{r}^T$ equals column j, i.e. the number j.

Worked Example 14.2 • Encoding, one error, correction — every bit

Data 1011: $d_1 = 1, d_2 = 0, d_3 = 1, d_4 = 1$.
$p_1 = 1 \oplus 0 \oplus 1 = 0$, $p_2 = 1 \oplus 1 \oplus 1 = 1$, $p_4 = 0 \oplus 1 \oplus 1 = 0$. **Codeword** (positions 1–7): 0 1 1 0 0 1 1.
Channel flips position 5: received $\mathbf{r} = 0\,1\,1\,0\,\mathbf{1}\,1\,1$.
Syndrome: $s_1 = r_1 \oplus r_3 \oplus r_5 \oplus r_7 = 0 \oplus 1 \oplus 1 \oplus 1 = 1$; $s_2 = r_2 \oplus r_3 \oplus r_6 \oplus r_7 = 1 \oplus 1 \oplus 1 \oplus 1 = 0$; $s_4 = r_4 \oplus r_5 \oplus r_6 \oplus r_7 = 0 \oplus 1 \oplus 1 \oplus 1 = 1$.
$(s_4 s_2 s_1) = 101_2 = 5$: **flip position 5** $\Rightarrow$ 0110011, data 1011 ✓.

Fig. 14.2 puts the encoder, the channel and the SECDED decision rule into one block diagram.

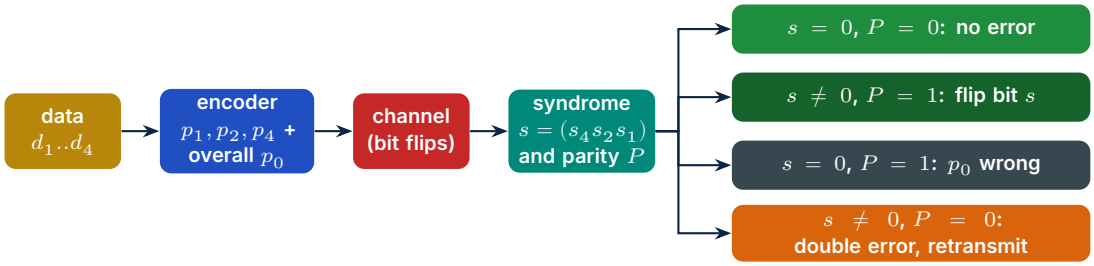


Figure 14.2. Block diagram. Extended Hamming (8,4) code with the SECDED decision rule. Without p_0 it is the Hamming (7,4) single-error corrector.

14.2.3 Limitations

! Hamming (7,4) does not correct two errors

$d_{\min} = 3$, so $t = 1$. Flip positions 2 and 6 of 0110011: $\mathbf{r} = 0\,\mathbf{0}\,1\,0\,0\,\mathbf{0}\,1$. Syndrome: $s_1 = 0 \oplus 1 \oplus 0 \oplus 1 = 0$, $s_2 = 0 \oplus 1 \oplus 0 \oplus 1 = 0$, $s_4 = 0 \oplus 0 \oplus 0 \oplus 1 = 1 \Rightarrow 4$. The decoder flips position 4 and outputs data 1001 — **wrong, without any warning**. Two errors look like a different single error.

14.2.4 Extended Hamming code and SECDED

Add an overall parity bit p_0 over all 7 bits ($d_{\min} = 4$). Decoding rule, with syndrome s and overall parity check P (0 = even):

s	P	decision
0	0	no error
$\neq 0$	1	single error at position s: correct it
0	1	error in p_0 itself: data are correct
$\neq 0$	0	**double error detected**: do not correct, request retransmission

Worked Example 14.3 • SECDED in action

Data 1011 $\rightarrow$ 0110011 | 0 ($p_0 = 0$ because the 7 bits contain four 1s). With the same two errors (positions 2 and 6): $s = 4 \neq 0$, overall parity even $\Rightarrow$ **double error detected**. With one error at position 5: $s = 5$, parity odd $\Rightarrow$ corrected. SECDED (single error correction, double error detection) protects most computer memory (ECC RAM, usually as a (72,64) code).

14.3 Simple multi-bit error correction (Chapter 9.3)

14.3.1 The (5,1) repetition code

Definition 14.3 • (5,1) repetition code

Each data bit is sent five times: $0 \rightarrow 00000$, $1 \rightarrow 11111$. The decoder counts the ones in each block of five and takes the **majority** (three or more ones $\Rightarrow$ 1). $d_{\min} = 5$, $t = 2$, $R = 1/5$.

Derivation • Why two errors are always corrected and three may not be

The two codewords differ in all 5 positions. With at most 2 flips, the received block has at least 3 bits equal to the sent bit, so the majority is right. With 3 flips, 3 bits show the wrong value and the majority is wrong: $t = \lfloor (5-1)/2 \rfloor = 2$ exactly.

Worked Example 14.4 • Two errors in each of two blocks, corrected

Message 1 0 1 $\rightarrow$ codeword 11111 00000 11111 (15 bits).
Channel flips positions 2, 4 (block 1) and 7, 10 (block 2) (counting from 1): received

10101 01001 11111.
Ones per block: 3, 2, 5 ⇒ majority 1, 0, 1 — **four bit errors corrected**.
Failure case: three flips in block 1 (**00**011): two ones ⇒ decodes 0, wrong.

Worked Example 14.5 • Comparison with Hamming (7,4) on a BSC

Probability that 4 data bits are received wrong, BSC crossover p:

Scheme	channel bits	$p = 0.01$	$p = 0.05$	$p = 0.1$
uncoded	4	3.9×10^{-2}	1.9×10^{-1}	3.4×10^{-1}
Hamming (7,4): $P(\geq 2$ errors in 7)	7	2.0×10^{-3}	4.4×10^{-2}	1.5×10^{-1}
(5,1) on each bit: $1-(1-P_3)^4$	20	3.9×10^{-5}	4.6×10^{-3}	3.4×10^{-2}

with $P_3 = \sum_{k=3}^{5} \binom{5}{k} p^k (1-p)^{5-k}$ ($= 9.85 \times 10^{-6}$ at $p = 0.01$). Repetition is far more reliable, but uses 20 channel bits instead of 7: a **redundancy–reliability trade-off**. It is educational and very wasteful; practical systems use stronger codes with much higher rates.

14.3.2 BCH(15,7): two-error correction at rate 0.47 (optional advanced)

BCH codes (Bose, Chaudhuri, Hocquenghem, 1959–60) correct t errors with algebra over finite fields.

Definition 14.4 • The field GF(16)

GF(16) has 16 elements, written as 4-bit vectors. Choose a root α of the primitive polynomial $x^4 + x + 1$, so $\alpha^4 = \alpha + 1$. The non-zero elements are $\alpha^0, \ldots, \alpha^{14}$; addition is XOR of the 4-bit vectors and multiplication adds exponents modulo 15:

i	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14
α^i	0001	0010	0100	1000	0011	0110	1100	1011	0101	1010	0111	1110	1111	1101	1001

Information theory concept • BCH(15,7) with $t = 2$

Codewords are the polynomials $c(x)$ of degree < 15 with roots $\alpha, \alpha^2, \alpha^3, \alpha^4$. This happens exactly when $c(x)$ is a multiple of

$$g(x) = m_1(x)\, m_3(x) = (x^4 + x + 1)(x^4 + x^3 + x^2 + x + 1) = x^8 + x^7 + x^6 + x^4 + 1.$$

$\deg g = 8$ parity bits, $k = 7$ data bits, and $d_{\min} = 5$ (the unit tests check all 128 codewords).

Worked Example 14.6 • Encoding and two-error correction

Systematic encoding. Message $m(x)$ with bits $(1, 0, 1, 1, 0, 0, 1)$ (coefficients of $x^0..x^6$). Compute $x^8 m(x) \bmod g(x)$ for the 8 parity bits: codeword (coefficients of $x^0..x^{14}$) = $\underbrace{0\,1\,0\,0\,0\,0\,1\,1}_{\text{parity}}\ \underbrace{1\,0\,1\,1\,0\,0\,1}_{\text{message}}$.
Errors at positions 2 and 11: $e(x) = x^2 + x^{11}$.
Syndromes (only the error matters, because $c(\alpha) = c(\alpha^3) = 0$): $S_1 = r(\alpha) = \alpha^2 + \alpha^{11} = 0100 \oplus 1110 = 1010 = \alpha^9$; $S_3 = r(\alpha^3) = \alpha^6 + \alpha^{33} = \alpha^6 + \alpha^3 = 1100 \oplus 1000 = 0100 = \alpha^2$.

Test for one error: $S_1^3 = \alpha^{27} = \alpha^{12} \neq S_3$, so two errors.
Error locator $\Lambda(x) = 1 + \sigma_1 x + \sigma_2 x^2$ with $\sigma_1 = S_1 = \alpha^9$ and $\sigma_2 = \dfrac{S_3 + S_1^3}{S_1} = \dfrac{\alpha^2 + \alpha^{12}}{\alpha^9} = \dfrac{\alpha^7}{\alpha^9} = \alpha^{13}$.
Chien search: test $x = \alpha^{-j}$ for $j = 0..14$; $\Lambda(\alpha^{-j}) = 0$ exactly for $j = 2$ and $j = 11$. Check: $\sigma_2 = \alpha^2\alpha^{11} = \alpha^{13}$ and $\sigma_1 = \alpha^2 + \alpha^{11}$ ✓. Flip bits 2 and 11: the codeword and the message 1011001 are recovered.

Fig. 14.3 shows the decoder as a chain of algebraic blocks.

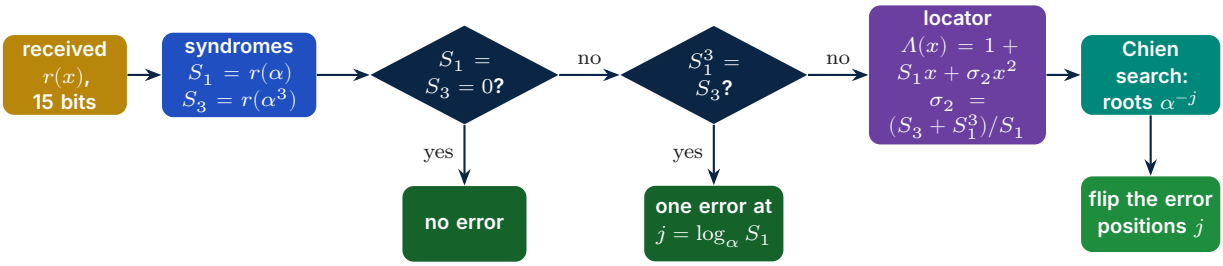


Figure 14.3. Block diagram. Two-error-correcting BCH(15,7) decoder over GF(16).

Important note

Why does this work? If errors are at positions j_1, j_2, write $X_1 = \alpha^{j_1}$, $X_2 = \alpha^{j_2}$. Then $S_1 = X_1 + X_2$ and $S_3 = X_1^3 + X_2^3$. Using $X_1^3 + X_2^3 = (X_1 + X_2)^3 + X_1X_2(X_1 + X_2)$ (characteristic 2) gives $X_1X_2 = (S_3 + S_1^3)/S_1$. The locator $(1 + X_1x)(1 + X_2x)$ then has roots at X_1^{-1}, X_2^{-1}. BCH(15,7) corrects any 2 errors at rate 0.467 — much better than (5,1) repetition at rate 0.2.

14.4 Implementations and simulation (Chapter 9.4)

Labs 7 and 8 contain complete programs for Hamming (7,4), SECDED, (5,1) repetition with controlled error injection, and BCH(15,7). The unit tests check *every* single-error pattern for Hamming, every 1- and 2-error pattern (120 patterns) for BCH on 30 random messages, and every error pattern of weight 0–5 for the repetition code.

Error correction is not integrity protection

Error-correcting codes and CRCs protect against *random* errors. An attacker can change a message and recompute the parity bits, because the codes are public and linear. Integrity against adversaries needs a *keyed* check: a message authentication code (HMAC) or authenticated encryption (AES-GCM). In practice: ECC for reliability below, MAC for integrity above.

14.5 Module summary

Key points

- $d_{\min}$ decides everything: detect $d_{\min} - 1$, correct $\lfloor (d_{\min} - 1)/2 \rfloor$.

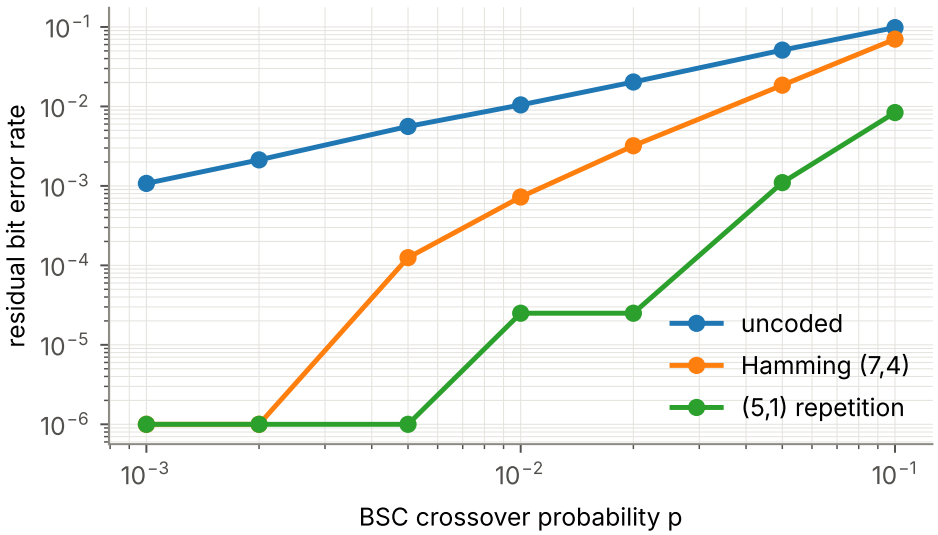


Figure 14.4. Simulated residual bit error rate after decoding on a BSC (40 000 data bits per point; points with no errors are drawn at 10^{-6}).

- Hamming (7,4): syndrome = error position; corrects exactly one error; two errors cause silent miscorrection.
- SECDED adds one parity bit to detect double errors.
- (5,1) repetition corrects two errors per bit at rate 1/5; BCH(15,7) corrects two errors per 15 bits at rate 7/15.
- Integrity against attackers requires cryptographic MACs.

Formula sheet

Rate	$R = k/n$
Capabilities	detect $d_{\min} - 1$; correct $t = \lfloor (d_{\min} - 1)/2 \rfloor$
Hamming parity	$p_1 = d_1 \oplus d_2 \oplus d_4$, $p_2 = d_1 \oplus d_3 \oplus d_4$, $p_4 = d_2 \oplus d_3 \oplus d_4$
Syndrome	$\mathbf{s} = \mathbf{H}\mathbf{r}^T$ = position of a single error
Block failure (BSC)	$P_{\text{fail}} = \sum_{k>t} \binom{n}{k} p^k (1-p)^{n-k}$
BCH(15,7)	$g = x^8 + x^7 + x^6 + x^4 + 1$; $\sigma_2 = (S_3 + S_1^3)/S_1$

Review questions

1. Why does a single parity bit detect but not correct errors?
2. Why is the syndrome of a single error equal to its position?
3. What does SECDED do with a double error?
4. Compare the rates of (5,1) repetition and BCH(15,7). Which is better, and why?

Solved exercises

Problem 14.1. Encode data 0110 with Hamming (7,4).

Solution $d_1 = 0, d_2 = 1, d_3 = 1, d_4 = 0$: $p_1 = 0 \oplus 1 \oplus 0 = 1$, $p_2 = 0 \oplus 1 \oplus 0 = 1$, $p_4 = 1 \oplus 1 \oplus 0 = 0$. Codeword $1\,1\,0\,0\,1\,1\,0$.

Problem 14.2. Received $1\,1\,0\,0\,1\,0\,0$ (Hamming (7,4)). Correct it.

Solution $s_1 = 1 \oplus 0 \oplus 1 \oplus 0 = 0$, $s_2 = 1 \oplus 0 \oplus 0 \oplus 0 = 1$, $s_4 = 0 \oplus 1 \oplus 0 \oplus 0 = 1$: syndrome $110_2 = 6$. Flip position 6: 1100110, data 0110.

Problem 14.3. A (5,1) block is received as 01101. Decode it. How many errors were corrected if 1 was sent?

Solution Three ones $\Rightarrow$ 1. Two zeros were errors: 2 errors corrected.

Problem 14.4. What is t for a code with $d_{\min} = 7$?

Solution $t = \lfloor 6/2 \rfloor = 3$.

PART V

Practical Python Laboratories

Eleven complete programming laboratories with tested Python code, outputs and exercises.

CHAPTER

15



Python Laboratories

IN THIS CHAPTER YOU WILL LEARN

Eleven laboratories. Each lab gives: objectives, background, algorithm, flowchart, libraries, the complete program (exactly the file in the `itlab` folder), a line-by-line explanation, sample input, real output, interpretation, common errors and exercises. Run every program with `python3 file.py` from inside the `itlab` folder. The test file `test_itlab.py` checks all labs automatically.

Lab	Experiment	File	Chapter
1	Entropy calculator	`entropy.py`	1–4
2	Shannon–Fano encoder/decoder	`shannon_fano.py`	6
3	Huffman encoder/decoder	`huffman.py`	7
4	Arithmetic encoder/decoder	`arithmetic.py`	8
5	LZW compressor/decompressor	`lzw.py`	9
6	Compression comparison	`compare.py`	10
7	Hamming detection and correction	`hamming.py`	14
8	Multi-bit correction: repetition and BCH	`repetition.py`, `bch.py`	14
9	Markov chains	`markov.py`	11
10	Channel capacity	`channel.py`	13
11	Entropy-based anomaly detection	`anomaly.py`	12

Requirements. Python 3.8 or newer; only the standard library (`math`, `collections`, `heapq`, `fractions`, `itertools`, `random`, `unittest`). Matplotlib is needed only for the figures.

15.1 Lab 1 — Entropy calculator

1. Objectives

Compute self-information, entropy, binary entropy, min-entropy, joint and conditional entropy, mutual information and KL divergence.

2. Background

$H = -\sum p \log_2 p$; $H(Y|X) = H(X,Y) - H(X)$; $I = H(X) + H(Y) - H(X,Y)$; $D(p\|q) = \sum p \log_2(p/q)$ (Chapters 1–4).

3. Algorithm and 4. Flowchart



5. Libraries

`math.log2`, `collections.Counter`.

6. Program

entropy.py

```python
"""entropy.py -- information measures (Labs 1 and 3/4 of the book).

All logarithms are base 2, so every result is in bits.
Probabilities equal to 0 are skipped, using the convention 0*log(0) = 0.
"""
from collections import Counter
from math import log2


def self_information(p):
    """I(x) = -log2 p(x): the surprise of an outcome with probability p."""
    if not 0 < p <= 1:
        raise ValueError("p must be in (0, 1]")
    return -log2(p)


def entropy(probs):
    """H(X) = -sum p log2 p for a list (or dict values) of probabilities."""
    if isinstance(probs, dict):
        probs = list(probs.values())
    if abs(sum(probs) - 1) > 1e-9:
        raise ValueError("probabilities must sum to 1")
    return -sum(p * log2(p) for p in probs if p > 0)


def binary_entropy(p):
    """H2(p) = -p log2 p - (1-p) log2 (1-p)."""
    return entropy([p, 1 - p])


def probs_from_text(text):
    """Relative frequency of every symbol, sorted by symbol."""
    counts = Counter(text)
    n = len(text)
    return {s: counts[s] / n for s in sorted(counts)}


# ------------------------------------------------------------ two variables
```

```python
def marginals(P):
    """P is a joint table P[x][y] (list of rows). Returns (p_x, p_y)."""
    px = [sum(row) for row in P]
    py = [sum(P[i][j] for i in range(len(P))) for j in range(len(P[0]))]
    return px, py


def joint_entropy(P):
    """H(X,Y) from a joint table."""
    return entropy([p for row in P for p in row])


def conditional_entropy(P):
    """H(Y|X) = H(X,Y) - H(X)  (rows = X, columns = Y)."""
    px, _ = marginals(P)
    return joint_entropy(P) - entropy(px)


def mutual_information(P):
    """I(X;Y) = H(X) + H(Y) - H(X,Y)."""
    px, py = marginals(P)
    return entropy(px) + entropy(py) - joint_entropy(P)


def kl_divergence(p, q):
    """D(p||q) = sum p log2(p/q). Infinite if q = 0 where p > 0."""
    d = 0.0
    for pi, qi in zip(p, q):
        if pi > 0:
            if qi == 0:
                return float("inf")
            d += pi * log2(pi / qi)
    return d


def min_entropy(probs):
    """H_inf(X) = -log2 max p: the best single guess of an attacker."""
    if isinstance(probs, dict):
        probs = list(probs.values())
    return -log2(max(probs))


if __name__ == "__main__":
    text = "ABRACADABRA"
    p = probs_from_text(text)
    print("text          :", text)
    print("probabilities :", {s: round(v, 4) for s, v in p.items()})
    print("H(X)          : %.4f bits/symbol" % entropy(p))
    print("max possible  : %.4f bits/symbol (log2 of %d symbols)" % (log2(len(p)), len(p)))
    print("H2(0.1)       : %.4f bits" % binary_entropy(0.1))
    P = [[0.25, 0.25], [0.0, 0.5]]            # joint table P[x][y]
    print("H(X,Y) = %.4f, H(Y|X) = %.4f, I(X;Y) = %.4f"
          % (joint_entropy(P), conditional_entropy(P), mutual_information(P)))
```

7. Line by line

- Lines 17–23: `entropy` accepts a list or dict, checks that probabilities sum to 1, and skips zero probabilities ($0 \log 0 = 0$).
- Lines 31–35: `probs_from_text` counts symbols with `Counter` and divides by the length.
- Lines 39–60: a joint table is a list of rows (rows = X, columns = Y); marginals are row and column sums; $H(Y|X)$ and I use the identities.
- Lines 63–71: KL returns infinity when $q = 0$ where $p > 0$.

8. Sample input and 9. Output

Input: the text ABRACADABRA and the joint table $\left[\begin{smallmatrix} 0.25 & 0.25 \\ 0 & 0.5 \end{smallmatrix}\right]$.

OUTPUT (real run)

```
text          : ABRACADABRA
probabilities : {'A': 0.4545, 'B': 0.1818, 'C': 0.0909, 'D': 0.0909, 'R': 0.1818}
H(X)          : 2.0404 bits/symbol
max possible  : 2.3219 bits/symbol (log2 of 5 symbols)
H2(0.1)       : 0.4690 bits
H(X,Y) = 1.5000, H(Y|X) = 0.5000, I(X;Y) = 0.3113
```

10. Interpretation

ABRACADABRA has 2.04 bits/letter against a maximum of 2.32: the text is somewhat predictable. In the joint table Y reveals 0.31 bits about X.

11. Common errors

Using `math.log` (natural log, nats) instead of `log2`; forgetting to skip $p = 0$ (`log2(0)` raises an error); passing counts instead of probabilities.

12. Exercises

(a) Compute the entropy of your full name. (b) Add a function for Rényi entropy $H_\alpha = \frac{1}{1-\alpha} \log_2 \sum p^\alpha$ and check $H_2 \geq H_\infty$. (c) Verify $I(X;Y) = D(p_{XY} \| p_X p_Y)$ numerically.

15.2 Lab 2 — Shannon–Fano encoder and decoder

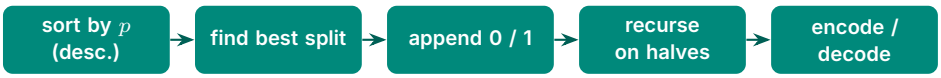
1. Objectives

Build a Shannon–Fano code, encode and decode a message, compute L, efficiency and redundancy.

2. Background

Chapter 6: sort, split into balanced halves, assign 0/1, recurse.

3. Algorithm and 4. Flowchart



5. Libraries

`math.log2`.

6. Program

shannon_fano.py

```python
"""shannon_fano.py -- Shannon-Fano coding (Lab 2).

Conventions
  * symbols are sorted by decreasing probability; ties are broken by symbol;
  * a list is split at the point where |P(left) - P(right)| is smallest
    (the first such point if several are equal);
  * the left (more probable) group gets bit '0', the right group bit '1'.
"""
from math import log2


def shannon_fano(probs):
    """probs: dict symbol -> probability. Returns dict symbol -> codeword."""
    items = sorted(probs.items(), key=lambda kv: (-kv[1], kv[0]))
    codes = {s: "" for s, _ in items}
    steps = []                                  # (group, left, right) for tables

    def split(group):
        if len(group) == 1:
            return
        total = sum(p for _, p in group)
        best_k, best_diff, run = 1, float("inf"), 0.0
        for k in range(1, len(group)):          # left = group[:k]
            run += group[k - 1][1]
            diff = abs(run - (total - run))
            if diff < best_diff - 1e-12:
                best_k, best_diff = k, diff
        left, right = group[:best_k], group[best_k:]
        steps.append(([s for s, _ in left], [s for s, _ in right]))
        for s, _ in left:
            codes[s] += "0"
        for s, _ in right:
            codes[s] += "1"
        split(left)
        split(right)

    if len(items) == 1:                         # one symbol still needs one bit
        return {items[0][0]: "0"}
    split(items)
    shannon_fano.steps = steps
    return codes


def encode(message, codes):
    return "".join(codes[s] for s in message)


def decode(bits, codes):
    """Prefix-free decoding: read bits until they match a codeword."""
    inverse = {c: s for s, c in codes.items()}
    out, cur = [], ""
```

```
    for b in bits:
        cur += b
        if cur in inverse:
            out.append(inverse[cur])
            cur = ""
    if cur:
        raise ValueError("bitstream ended inside a codeword")
    return "".join(out)


def average_length(probs, codes):
    """L = sum p(x) * len(code(x)) in bits/symbol."""
    return sum(p * len(codes[s]) for s, p in probs.items())


def report(probs, codes):
    H = -sum(p * log2(p) for p in probs.values() if p > 0)
    L = average_length(probs, codes)
    return {"H": H, "L": L, "efficiency": H / L, "redundancy": 1 - H / L}


if __name__ == "__main__":
    probs = {"A": 0.30, "B": 0.25, "C": 0.15, "D": 0.12, "E": 0.10, "F": 0.08}
    codes = shannon_fano(probs)
    for s in sorted(codes, key=lambda s: -probs[s]):
        print(f"{s}  p = {probs[s]:.2f}  code = {codes[s]}")
    r = report(probs, codes)
    print("H = %.4f  L = %.4f  efficiency = %.2f%%" % (r["H"], r["L"], 100 * r["efficiency"
    ]))
    msg = "ABACADEF"
    bits = encode(msg, codes)
    print("encode", msg, "->", bits, "->", decode(bits, codes))
```

7. Line by line

- Line 14: sort by decreasing probability, ties by symbol name (the book's convention).
- Lines 18–38: `split` tries every cut k, keeps the cut with the smallest $|P_{\text{left}} - P_{\text{right}}|$ (first one on ties), appends 0 to the left group and 1 to the right group, then recurses. The cuts are stored in `steps` for printing tables.
- Lines 48–59: the decoder accumulates bits until they form a codeword; this works because the code is prefix-free. Leftover bits raise an error.

8. Sample input and 9. Output

Input: A .30, B .25, C .15, D .12, E .10, F .08 and the message ABACADEF.

OUTPUT (real run)

```
A  p = 0.30  code = 00
B  p = 0.25  code = 01
C  p = 0.15  code = 100
D  p = 0.12  code = 101
E  p = 0.10  code = 110
F  p = 0.08  code = 111
```

```
H = 2.4224  L = 2.4500  efficiency = 98.87%
encode ABACADEF -> 0001001000010111011 -> ABACADEF
```

10. Interpretation

$L = 2.45$ bits vs $H = 2.4224$: 98.9% efficient. The 8-symbol message takes 20 bits instead of 64 in ASCII.

11. Common errors

Sorting in increasing order; choosing the split by number of symbols instead of probability; decoding a stream that was cut in the middle of a codeword.

12. Exercises

(a) Run the seven-symbol example of Chapter 6 and check the codes. (b) Find a source where Shannon–Fano is worse than Huffman (hint: one symbol with $p \approx 0.4$).

15.3 Lab 3 — Huffman encoder and decoder

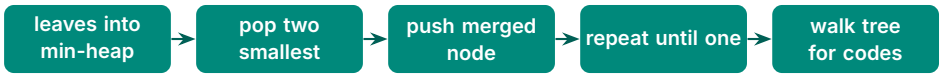
1. Objectives

Build Huffman codes with a priority queue, print the merge table, encode and decode, verify optimality.

2. Background

Chapter 7.

3. Algorithm and 4. Flowchart



5. Libraries

`heapq`, `itertools.count`.

6. Program

huffman.py

```python
"""huffman.py -- Huffman coding (Lab 3).

Conventions (needed to get the same codewords every time)
  * the priority queue holds (probability, order, node); 'order' is a counter
    that breaks ties: older nodes come first, leaves are entered in sorted
    symbol order;
  * at each step the two least probable nodes are removed; the FIRST removed
    becomes the child with bit '0', the second the child with bit '1'.
"""
import heapq
from itertools import count
from math import log2


```

```python
def huffman(probs):
    """probs: dict symbol -> probability (or count). Returns dict symbol -> code."""
    if len(probs) == 1:
        return {next(iter(probs)): "0"}
    tick = count()
    heap = [(p, next(tick), s) for s, p in sorted(probs.items())]
    heapq.heapify(heap)
    merges = []                                     # for the step tables
    while len(heap) > 1:
        p0, _, n0 = heapq.heappop(heap)             # least probable  -> '0'
        p1, _, n1 = heapq.heappop(heap)             # second least    -> '1'
        merges.append((n0, p0, n1, p1, p0 + p1))
        heapq.heappush(heap, (p0 + p1, next(tick), (n0, n1)))
    huffman.merges = merges
    codes = {}

    def walk(node, prefix):
        if isinstance(node, tuple):                 # internal node
            walk(node[0], prefix + "0")
            walk(node[1], prefix + "1")
        else:
            codes[node] = prefix
    walk(heap[0][2], "")
    return codes


def encode(message, codes):
    return "".join(codes[s] for s in message)


def decode(bits, codes):
    inverse = {c: s for s, c in codes.items()}
    out, cur = [], ""
    for b in bits:
        cur += b
        if cur in inverse:
            out.append(inverse[cur])
            cur = ""
    if cur:
        raise ValueError("bitstream ended inside a codeword")
    return "".join(out)


def average_length(probs, codes):
    total = sum(probs.values())
    return sum(p * len(codes[s]) for s, p in probs.items()) / total


def leaves(node):
    return [node] if not isinstance(node, tuple) else leaves(node[0]) + leaves(node[1])


if __name__ == "__main__":
    probs = {"A": 0.30, "B": 0.25, "C": 0.15, "D": 0.12, "E": 0.10, "F": 0.08}
```

```
    codes = huffman(probs)
    for step, (n0, p0, n1, p1, p) in enumerate(huffman.merges, 1):
        print(f"step {step}: merge {''.join(leaves(n0))}({p0:.2f}) + "
              f"{''.join(leaves(n1))}({p1:.2f}) = {p:.2f}")
    for s in sorted(codes):
        print(s, codes[s])
    H = -sum(p * log2(p) for p in probs.values())
    L = average_length(probs, codes)
    print("H = %.4f  L = %.4f  efficiency = %.2f%%" % (H, L, 100 * H / L))
    msg = "ABACADEF"
    bits = encode(msg, codes)
    print("encode", msg, "->", bits, "->", decode(bits, codes))
```

7. Line by line

- Lines 19–21: heap entries are (probability, age, node); the counter `tick` gives the age, so ties are deterministic and Python never compares nodes.
- Lines 23–27: pop the two smallest (first gets bit 0), record the merge, push the parent (a tuple of the two children).
- Lines 31–36: a recursive walk appends 0/1 at each level; leaves are symbols.
- Lines 58–60: average length works with probabilities or counts (divides by the total).

8. Sample input and 9. Output

OUTPUT (real run)

```
step 1: merge F(0.08) + E(0.10) = 0.18
step 2: merge D(0.12) + C(0.15) = 0.27
step 3: merge FE(0.18) + B(0.25) = 0.43
step 4: merge DC(0.27) + A(0.30) = 0.57
step 5: merge FEB(0.43) + DCA(0.57) = 1.00
A 11
B 01
C 101
D 100
E 001
F 000
H = 2.4224  L = 2.4500  efficiency = 98.87%
encode ABACADEF -> 11011110111100001000 -> ABACADEF
```

10. Interpretation

The merge table is exactly the table of Worked Example 7.1. For this source Huffman and Shannon–Fano have the same L (2.45); the unit tests check on 200 random sources that Huffman is never worse.

11. Common errors

Putting nodes (tuples) as the second heap field (comparisons fail on ties); forgetting the one-symbol case (empty codeword); assuming codewords are unique across implementations (only lengths are).

12. Exercises

(a) Implement canonical Huffman codes from the lengths. (b) Huffman-code blocks of two symbols of a binary source with $P(0) = 0.9$ and reproduce $L_2/2 = 0.645$.

15.4 Lab 4 — Arithmetic encoder and decoder

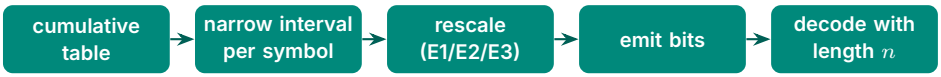
1. Objectives

Implement exact (fraction) arithmetic coding for hand-checkable examples and a 32-bit integer coder for real data; verify round trips.

2. Background

Chapter 8: interval narrowing, shortest tag, E1/E2/E3 scaling.

3. Algorithm and 4. Flowchart



5. Libraries

`fractions.Fraction`, `math`.

6. Program

arithmetic.py

```python
"""arithmetic.py -- arithmetic coding (Lab 4).

Two versions are given.

1. encode_exact / decode_exact  (teaching version)
   Uses exact fractions, so every interval in the book can be checked by hand.
   The decoder is told the message length n (or stops at a terminator symbol).

2. encode_int / decode_int  (practical finite-precision version)
   32-bit integer arithmetic with the classic scaling rules
   E1 (both in lower half), E2 (both in upper half) and E3 (middle half,
   'pending' bits) of Witten, Neal and Cleary (1987). Symbol frequencies are
   integers; the decoder is told the message length n.

Conventions: symbols are ordered by sorted symbol name in the cumulative table;
symbol s occupies [cum_low(s), cum_high(s)) inside the current interval.
"""
from fractions import Fraction
from math import ceil, log2


# ---------------------------------------------------------- cumulative table
def cumulative(probs):
    """dict symbol -> (low, high) as exact Fractions, symbols in sorted order."""
    table, c = {}, Fraction(0)
    for s in sorted(probs):
        p = Fraction(probs[s]).limit_denominator(10**9)
        table[s] = (c, c + p)
        c += p
    if c != 1:
        raise ValueError("probabilities must sum to 1 exactly")
    return table
```

```python


# ------------------------------------------------------------ exact version
def encode_exact(message, probs, trace=False):
    """Returns (low, high, bits): the final interval and the shortest binary
    string whose dyadic interval [0.bits, 0.bits + 2^-len) lies inside it."""
    table = cumulative(probs)
    low, high = Fraction(0), Fraction(1)
    steps = []
    for s in message:
        width = high - low
        c_lo, c_hi = table[s]
        low, high = low + width * c_lo, low + width * c_hi
        steps.append((s, low, high))
    if trace:
        encode_exact.steps = steps
    # shortest tag: try lengths k = 1, 2, ... (k <= ceil(-log2(width)) + 1)
    k = 1
    while True:
        m = ceil(low * 2**k)                    # smallest k-bit number >= low
        if Fraction(m + 1, 2**k) <= high:
            bits = format(m, "0%db" % k)
            return low, high, bits
        k += 1


def decode_exact(bits, probs, n):
    """Decode n symbols from the binary fraction 0.bits."""
    table = cumulative(probs)
    value = Fraction(int(bits, 2), 2**len(bits))
    low, high = Fraction(0), Fraction(1)
    out = []
    for _ in range(n):
        width = high - low
        t = (value - low) / width               # position inside the interval
        for s, (c_lo, c_hi) in table.items():
            if c_lo <= t < c_hi:
                out.append(s)
                low, high = low + width * c_lo, low + width * c_hi
                break
    return "".join(out)


# ---------------------------------------------------------- integer version
PREC = 32
FULL = (1 << PREC) - 1
HALF = 1 << (PREC - 1)
QUARTER = 1 << (PREC - 2)


def _freq_table(counts):
    syms = sorted(counts)
    cum, c = {}, 0
    for s in syms:
```

```
        cum[s] = (c, c + counts[s])
        c += counts[s]
    return syms, cum, c


def encode_int(message, counts):
    """counts: dict symbol -> positive integer frequency. Returns a bit string."""
    syms, cum, total = _freq_table(counts)
    if total > QUARTER:
        raise ValueError("total frequency too large for 32-bit precision")
    low, high, pending = 0, FULL, 0
    out = []

    def emit(bit):
        nonlocal pending
        out.append(bit)
        out.extend([1 - bit] * pending)          # resolve pending E3 bits
        pending = 0

    for s in message:
        rng = high - low + 1
        c_lo, c_hi = cum[s]
        high = low + rng * c_hi // total - 1
        low = low + rng * c_lo // total
        while True:
            if high < HALF:                      # E1: interval in lower half
                emit(0)
            elif low >= HALF:                    # E2: interval in upper half
                emit(1)
                low -= HALF; high -= HALF
            elif low >= QUARTER and high < 3 * QUARTER:   # E3: middle half
                pending += 1
                low -= QUARTER; high -= QUARTER
            else:
                break
            low = 2 * low
            high = 2 * high + 1
    pending += 1                                 # flush: two final bits
    emit(0 if low < QUARTER else 1)
    return "".join(map(str, out))


def decode_int(bits, counts, n):
    syms, cum, total = _freq_table(counts)
    stream = [int(b) for b in bits]
    pos = 0

    def next_bit():
        nonlocal pos
        b = stream[pos] if pos < len(stream) else 0   # pad with zeros
        pos += 1
        return b

    low, high, code = 0, FULL, 0
```

```
    for _ in range(PREC):
        code = (code << 1) | next_bit()
    out = []
    for _ in range(n):
        rng = high - low + 1
        target = ((code - low + 1) * total - 1) // rng
        for s in syms:                              # find the symbol slot
            c_lo, c_hi = cum[s]
            if c_lo <= target < c_hi:
                break
        out.append(s)
        high = low + rng * c_hi // total - 1
        low = low + rng * c_lo // total
        while True:
            if high < HALF:
                pass
            elif low >= HALF:
                low -= HALF; high -= HALF; code -= HALF
            elif low >= QUARTER and high < 3 * QUARTER:
                low -= QUARTER; high -= QUARTER; code -= QUARTER
            else:
                break
            low = 2 * low
            high = 2 * high + 1
            code = (code << 1) | next_bit()
    return "".join(out)


if __name__ == "__main__":
    probs = {"A": Fraction(1, 2), "B": Fraction(1, 4), "C": Fraction(1, 4)}
    msg = "ABAC"
    low, high, bits = encode_exact(msg, probs, trace=True)
    for s, lo, hi in encode_exact.steps:
        print(f"after '{s}': [{lo}, {hi})  width {hi - lo}")
    print("tag bits:", bits, " -> decoded:", decode_exact(bits, probs, len(msg)))
    text = "TOBEORNOTTOBEORTOBEORNOT"
    from collections import Counter
    counts = Counter(text)
    b = encode_int(text, counts)
    print("integer coder:", len(b), "bits; round trip OK:", decode_int(b, counts, len(text)
    ) == text)
```

7. Line by line

- Lines 23–33: the cumulative table with exact fractions; an error is raised if probabilities do not sum to exactly 1.
- Lines 36–56: exact encoder; the tag search tries lengths $k = 1, 2, \ldots$ until a whole dyadic interval fits inside $[\text{low}, \text{high})$.
- Lines 59–74: exact decoder; it rescales the received value into the current interval and finds the slot.
- Lines 77–80: 32-bit constants; `HALF` and `QUARTER` define the scaling regions.
- Lines 92–126: integer encoder; `emit` writes a bit and then the pending E3 bits (inverted);

two final bits finish the code.

- Lines 129–166: integer decoder; it mirrors the encoder's scaling on `low`, `high` and the received `code` window.

8. Sample input and 9. Output

OUTPUT (real run)

```
after 'A': [0, 1/2)  width 1/2
after 'B': [1/4, 3/8)  width 1/8
after 'A': [1/4, 5/16)  width 1/16
after 'C': [19/64, 5/16)  width 1/64
tag bits: 010011  -> decoded: ABAC
integer coder: 60 bits; round trip OK: True
```

10. Interpretation

The exact intervals are those of Worked Example 8.1. The integer coder needs 60 bits for TOBEORNOTTOBEORTOBEORNOT; with the text's own symbol statistics $nH = 24 \times 2.4235 = 58.2$ bits, so the coder is within 2 bits of the entropy bound (the frequency table is not counted here).

11. Common errors

Using floating point (precision runs out after about 50 symbols); forgetting to tell the decoder when to stop; frequency totals larger than 2^{30}.

12. Exercises

(a) Add an EOF symbol and stop at it. (b) Make the model adaptive (update counts after every symbol) in both encoder and decoder.

15.5 Lab 5 — LZW compressor and decompressor

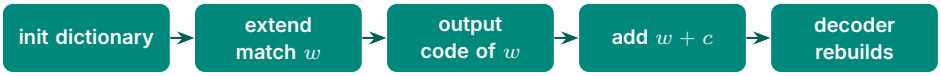
1. Objectives

Implement LZW with traces, handle the special case, count bits with dynamic widths.

2. Background

Chapter 9.

3. Algorithm and 4. Flowchart



5. Libraries

`math`.

6. Program

lzw.py

```
"""lzw.py -- Lempel-Ziv-Welch compression (Lab 5).

```

```
Conventions
  * the dictionary starts with the single symbols of 'alphabet' with codes
    0, 1, 2, ... in the given order (use alphabet=None for all 256 byte values,
    i.e. code = ASCII value);
  * every output code is followed by one new dictionary entry;
  * with dynamic widths, a code is written with ceil(log2(dictionary size))
    bits, where 'dictionary size' is the size at the moment of writing.
"""
from math import ceil, log2


def initial_dictionary(alphabet=None):
    if alphabet is None:
        return {chr(i): i for i in range(256)}
    return {s: i for i, s in enumerate(alphabet)}


def encode(text, alphabet=None, trace=False):
    """Returns the list of output codes."""
    d = initial_dictionary(alphabet)
    w, out, steps = "", [], []
    for c in text:
        if w + c in d:
            w = w + c                       # keep extending the match
        else:
            out.append(d[w])                # output the longest match
            d[w + c] = len(d)               # add match + next symbol
            steps.append((w, d[w], w + c, len(d) - 1))
            w = c
    if w:
        out.append(d[w])
        steps.append((w, d[w], None, None))
    if trace:
        encode.steps = steps
    return out


def decode(codes, alphabet=None, trace=False):
    """Rebuilds the dictionary while decoding (no dictionary is transmitted)."""
    d = {i: s for s, i in initial_dictionary(alphabet).items()}
    w = d[codes[0]]
    out, steps = [w], [(codes[0], w, None, None)]
    for k in codes[1:]:
        if k in d:
            entry = d[k]
        elif k == len(d):                   # special case: code not yet known
            entry = w + w[0]                # (pattern cScSc)
        else:
            raise ValueError("bad code %d" % k)
        d[len(d)] = w + entry[0]
        steps.append((k, entry, w + entry[0], len(d) - 1))
        out.append(entry)
        w = entry
    if trace:
```

```
57          decode.steps = steps
58      return "".join(out)
59
60
61  def bit_cost(codes, initial_size, dynamic=True, fixed_width=12):
62      """Total bits to store the code sequence."""
63      if not dynamic:
64          return len(codes) * fixed_width
65      total, size = 0, initial_size
66      for i, _ in enumerate(codes):
67          total += max(1, ceil(log2(size)))
68          size += 1 if i < len(codes) - 1 else 0   # one new entry per code
69      return total
70
71
72  if __name__ == "__main__":
73      for text, alpha in [("ABABABA", "AB"), ("TOBEORNOTTOBEORTOBEORNOT", None)]:
74          codes = encode(text, alpha, trace=True)
75          back = decode(codes, alpha)
76          n0 = len(initial_dictionary(alpha))
77          print(text)
78          print("  codes:", codes)
79          print("  round trip OK:", back == text)
80          print("  bits (dynamic width):", bit_cost(codes, n0),
81                " original:", len(text) * 8, "bits (8-bit ASCII)")
```

7. Line by line

- Lines 14–17: `alphabet=None` gives the 256 byte values (code = ASCII); otherwise codes $0, 1, \ldots$ in the given order.
- Lines 20–37: the encoder keeps the longest known match `w`; on a miss it outputs `d[w]` and adds `w+c`.
- Lines 40–58: the decoder; `k == len(d)` is the special case $w + w[0]$; any other unknown code is rejected (security).
- Lines 61–69: dynamic widths $\lceil \log_2(n_0 + k) \rceil$ for the k-th code.

8. Sample input and 9. Output

OUTPUT (real run)

```
ABABABA
  codes: [0, 1, 2, 4]
  round trip OK: True
  bits (dynamic width): 8  original: 56 bits (8-bit ASCII)
TOBEORNOTTOBEORTOBEORNOT
  codes: [84, 79, 66, 69, 79, 82, 78, 79, 84, 256, 258, 260, 265, 259, 261, 263]
  round trip OK: True
  bits (dynamic width): 143  original: 192 bits (8-bit ASCII)
```

10. Interpretation

Both examples round-trip. TOBEORNOT… takes 143 bits instead of 192.

11. Common errors

Forgetting the special case (decoder crashes on ABABABA); adding the dictionary entry before outputting the code; different initial dictionaries at both ends.

12. Exercises

(a) Limit the dictionary to 4096 entries and reset it when full. (b) Compress a 10 kB English text and report the ratio.

15.6 Lab 6 — Compression algorithm comparison

1. Objectives

Compare the four compressors fairly on common data, including header costs, and verify every round trip.

2. Background

Chapter 10.

3. Algorithm and 4. Flowchart



5. Libraries

The Lab 2–5 modules, `collections`, `random`.

6. Program

compare.py

```python
"""compare.py -- fair comparison of the four compressors (Lab 6).

For a text of n symbols:
  raw size            = 8 n bits (8-bit ASCII)
  Shannon-Fano/Huffman = payload bits + header (code table)
  arithmetic (integer) = payload bits + header (frequency table)
  LZW (dynamic widths, 256-entry ASCII start dictionary) = payload bits, no header
Header convention (explicit and simple, used for all table-based coders):
  16 bits for the number of distinct symbols
  + for each symbol: 8 bits (symbol) + 16 bits (its count)
The decoder rebuilds the identical code from the counts, so this header is
sufficient for Shannon-Fano, Huffman and arithmetic coding alike.
"""
from collections import Counter
from math import log2
import shannon_fano as sf
import huffman as hf
import arithmetic as ac
import lzw


def header_bits(counts):
```

```
    return 16 + len(counts) * (8 + 16)


def compare(text):
    n = len(text)
    counts = Counter(text)
    probs = {s: c / n for s, c in counts.items()}
    H = -sum(p * log2(p) for p in probs.values())
    rows = {}
    c_sf = sf.shannon_fano(probs)
    b = sf.encode(text, c_sf); assert sf.decode(b, c_sf) == text
    rows["Shannon-Fano"] = (len(b), header_bits(counts))
    c_hf = hf.huffman(probs)
    b = hf.encode(text, c_hf); assert hf.decode(b, c_hf) == text
    rows["Huffman"] = (len(b), header_bits(counts))
    b = ac.encode_int(text, counts); assert ac.decode_int(b, counts, n) == text
    rows["Arithmetic"] = (len(b), header_bits(counts))
    codes = lzw.encode(text); assert lzw.decode(codes) == text
    rows["LZW"] = (lzw.bit_cost(codes, 256), 0)
    return {"n": n, "H": H, "entropy_bits": n * H, "raw": 8 * n, "rows": rows}


DATASETS = {
    "skewed (A-heavy)": "A" * 600 + "B" * 200 + "C" * 120 + "D" * 50 + "E" * 30,
    "English text": None,          # filled in below
    "repetitive": "TOBEORNOT" * 120,
    "uniform random": None,
}

ENGLISH = ("information theory studies how much information a message carries and how "
           "efficiently it can be stored and sent. a source with predictable symbols has "
           "low entropy and can be compressed well. a source with random symbols has high "
           "entropy and cannot be compressed much. compression removes redundancy, and "
           "error correcting codes add redundancy in a controlled way so that a receiver "
           "can find and fix errors. security engineers use the same ideas to measure the "
           "strength of keys and the leakage of side channels. ") * 2


def datasets():
    import random
    rng = random.Random(2026)
    d = dict(DATASETS)
    skew = list(d["skewed (A-heavy)"]); rng.shuffle(skew)
    d["skewed (A-heavy)"] = "".join(skew)
    d["English text"] = ENGLISH
    d["uniform random"] = "".join(rng.choice("ABCDEFGHIJKLMNOP") for _ in range(1000))
    return d


if __name__ == "__main__":
    for name, text in datasets().items():
        r = compare(text)
        print(f"{name}: n = {r['n']}, H = {r['H']:.3f} bits/sym, n*H = {r['entropy_bits
    ']:.0f} bits, raw = {r['raw']}")
```

```
        for k, (pay, hdr) in r["rows"].items():
            print(f"   {k:13s} payload {pay:6d}  header {hdr:4d}  total {pay+hdr:6d}  "
                  f"ratio {r['raw']/(pay+hdr):5.2f}  bits/sym {pay/r['n']:.3f}")
```

7. Line by line

- Lines 22–23: the header convention (16 bits + 24 bits per symbol).
- Lines 26–42: every coder runs on the same text; `assert` statements guarantee lossless recovery.
- Lines 61–69: the four data sets; random ones use a fixed seed (2026) so that results are reproducible.

8. Sample input and 9. Output

OUTPUT (real run)

```
skewed (A-heavy): n = 1000, H = 1.641 bits/sym, n*H = 1641 bits, raw = 8000
   Shannon-Fano  payload   1680  header  136  total   1816  ratio  4.41  bits/sym 1.680
   Huffman       payload   1680  header  136  total   1816  ratio  4.41  bits/sym 1.680
   Arithmetic    payload   1643  header  136  total   1779  ratio  4.50  bits/sym 1.643
   LZW           payload   2642  header    0  total   2642  ratio  3.03  bits/sym 2.642
English text: n = 1020, H = 4.122 bits/sym, n*H = 4204 bits, raw = 8160
   Shannon-Fano  payload   4266  header  640  total   4906  ratio  1.66  bits/sym 4.182
   Huffman       payload   4240  header  640  total   4880  ratio  1.67  bits/sym 4.157
   Arithmetic    payload   4205  header  640  total   4845  ratio  1.68  bits/sym 4.123
   LZW           payload   4832  header    0  total   4832  ratio  1.69  bits/sym 4.737
repetitive: n = 1080, H = 2.419 bits/sym, n*H = 2613 bits, raw = 8640
   Shannon-Fano  payload   2640  header  160  total   2800  ratio  3.09  bits/sym 2.444
   Huffman       payload   2640  header  160  total   2800  ratio  3.09  bits/sym 2.444
   Arithmetic    payload   2614  header  160  total   2774  ratio  3.11  bits/sym 2.420
   LZW           payload   1214  header    0  total   1214  ratio  7.12  bits/sym 1.124
uniform random: n = 1000, H = 3.994 bits/sym, n*H = 3994 bits, raw = 8000
   Shannon-Fano  payload   4033  header  400  total   4433  ratio  1.80  bits/sym 4.033
   Huffman       payload   4000  header  400  total   4400  ratio  1.82  bits/sym 4.000
   Arithmetic    payload   3995  header  400  total   4395  ratio  1.82  bits/sym 3.995
   LZW           payload   5752  header    0  total   5752  ratio  1.39  bits/sym 5.752
```

10. Interpretation

See Chapter 10: arithmetic $\approx nH$, Huffman close, LZW wins only on repetitive data.

11. Common errors

Comparing payloads without headers; comparing a compressor's bits with nH computed from a *different* model; forgetting that the empirical H ignores memory.

12. Exercises

(a) Add Python's `zlib` as a fifth method. (b) Plot the compression ratio of each method against text length $100, \ldots, 10\,000$.

15.7 Lab 7 — Hamming error detection and correction

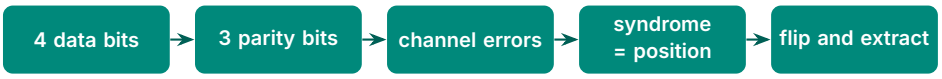
1. Objectives

Encode with Hamming (7,4), compute syndromes, correct single errors, show the failure with two errors, and detect doubles with SECDED.

2. Background

Chapter 14.

3. Algorithm and 4. Flowchart



5. Libraries

none.

6. Program

hamming.py

```python
"""hamming.py -- Hamming (7,4) and extended Hamming (8,4) SECDED (Lab 7).

Bit positions 1..7 (position-numbered layout of the book):
      position : 1   2   3   4   5   6   7
      content  : p1  p2  d1  p4  d2  d3  d4
  p1 checks positions 1,3,5,7   (positions whose binary form has bit 1 set)
  p2 checks positions 2,3,6,7   (bit 2 set)
  p4 checks positions 4,5,6,7   (bit 4 set)
Even parity is used. The syndrome (s4 s2 s1) read as a binary number is the
position of a single error (0 = no error detected).
"""

def encode74(d):
    """d = [d1, d2, d3, d4] -> 7-bit codeword [p1, p2, d1, p4, d2, d3, d4]."""
    d1, d2, d3, d4 = d
    p1 = d1 ^ d2 ^ d4          # positions 3,5,7
    p2 = d1 ^ d3 ^ d4          # positions 3,6,7
    p4 = d2 ^ d3 ^ d4          # positions 5,6,7
    return [p1, p2, d1, p4, d2, d3, d4]


def syndrome74(r):
    """Returns the error position 0..7 computed from the 7 received bits."""
    s1 = r[0] ^ r[2] ^ r[4] ^ r[6]   # positions 1,3,5,7
    s2 = r[1] ^ r[2] ^ r[5] ^ r[6]   # positions 2,3,6,7
    s4 = r[3] ^ r[4] ^ r[5] ^ r[6]   # positions 4,5,6,7
    return 4 * s4 + 2 * s2 + s1


def decode74(r):
    """Corrects up to one error. Returns (data bits, corrected word, syndrome)."""
    r = list(r)
    s = syndrome74(r)
    if s:
        r[s - 1] ^= 1                # flip the bit at position s
```

```
    return [r[2], r[4], r[5], r[6]], r, s


# Matrix form (same code): codeword c = d G (mod 2), H c^T = 0
G74 = [[1, 1, 1, 0, 0, 0, 0],
       [1, 0, 0, 1, 1, 0, 0],
       [0, 1, 0, 1, 0, 1, 0],
       [1, 1, 0, 1, 0, 0, 1]]
H74 = [[1, 0, 1, 0, 1, 0, 1],     # s1 : column j is the binary form of position j
       [0, 1, 1, 0, 0, 1, 1],     # s2
       [0, 0, 0, 1, 1, 1, 1]]     # s4


# ------------------------------------------------------------------ SECDED (8,4)
def encode84(d):
    """Hamming (7,4) codeword plus an overall parity bit p0 at the end."""
    c = encode74(d)
    return c + [sum(c) % 2]


def decode84(r):
    """Single error correction, double error detection.
    Returns (data or None, status) with status in
    'no error', 'corrected single error', 'parity bit error', 'double error detected'."""
    s = syndrome74(r[:7])
    overall = sum(r) % 2                # 0 if an even number of bits flipped
    if s == 0 and overall == 0:
        return [r[2], r[4], r[5], r[6]], "no error"
    if overall == 1:                    # odd number of errors: assume one
        if s == 0:
            return [r[2], r[4], r[5], r[6]], "parity bit error"
        d, _, _ = decode74(r[:7])
        return d, "corrected single error"
    return None, "double error detected"  # s != 0 and even parity


if __name__ == "__main__":
    d = [1, 0, 1, 1]
    c = encode74(d)
    print("data", d, "-> codeword", c)
    r = c.copy(); r[4] ^= 1             # error at position 5
    dd, fixed, s = decode74(r)
    print("received", r, "syndrome", s, "-> corrected", fixed, "data", dd)
    r2 = c.copy(); r2[1] ^= 1; r2[5] ^= 1   # two errors (positions 2 and 6)
    dd2, _, s2 = decode74(r2)
    print("two errors -> syndrome", s2, "data", dd2, "(WRONG: Hamming(7,4) corrects only
    one)")
    c8 = encode84(d)
    r8 = c8.copy(); r8[1] ^= 1; r8[5] ^= 1   # the same two errors in SECDED
    print("SECDED (8,4) codeword", c8, "with two errors ->", decode84(r8)[1])
```

7. Line by line

- Lines 13–19: parity equations with XOR (^); codeword in the order $p_1 p_2 d_1 p_4 d_2 d_3 d_4$.
- Lines 22–27: the three checks; the syndrome is $4s_4 + 2s_2 + s_1$.
- Lines 30–35: flip the bit at position s (index $s-1$) and read the data positions 3, 5, 6, 7.
- Lines 39–46: the same code as matrices $\mathbf{G}$ and $\mathbf{H}$ (tested against the equations).
- Lines 50–69: SECDED adds an overall parity bit and applies the decision table of Chapter 14.

8. Sample input and 9. Output

OUTPUT (real run)

```
data [1, 0, 1, 1] -> codeword [0, 1, 1, 0, 0, 1, 1]
received [0, 1, 1, 0, 1, 1, 1] syndrome 5 -> corrected [0, 1, 1, 0, 0, 1, 1] data [1, 0, 1, 1]
two errors -> syndrome 4 data [1, 0, 0, 1] (WRONG: Hamming(7,4) corrects only one)
SECDED (8,4) codeword [0, 1, 1, 0, 0, 1, 1, 0] with two errors -> double error detected
```

10. Interpretation

One error is corrected; two errors are silently miscorrected by (7,4) (wrong data 1001), while SECDED detects them.

11. Common errors

Off-by-one between position s and list index $s-1$; mixing the positional layout with the systematic $[\mathbf{I}|\mathbf{P}]$ layout.

12. Exercises

(a) Encode a text: split bytes into 4-bit nibbles. (b) Measure the residual BER on a BSC with $p = 0.01$.

15.8 Lab 8 — Multi-bit correction: repetition and BCH

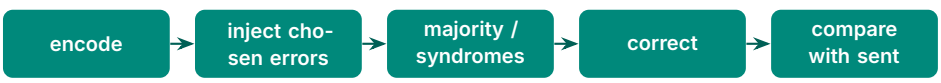
1. Objectives

Correct two errors per block with the (5,1) repetition code and with BCH(15,7); inject controlled errors; test success and failure.

2. Background

Chapter 14.

3. Algorithm and 4. Flowchart



5. Libraries

none.

6. Programs

repetition.py

```python
"""repetition.py -- the (5,1) repetition code (Lab 8).

Each message bit is sent five times. The decoder takes a majority vote of each
5-bit block: d_min = 5, so up to t = floor((5-1)/2) = 2 errors per block are
corrected. Three or more errors in one block cause a decoding error.
Rate R = 1/5: educational and very redundant, not an efficient practical code.
"""
N_REP = 5


def encode(bits, n=N_REP):
    return [b for b in bits for _ in range(n)]


def decode(received, n=N_REP):
    """Majority vote per block. Returns (bits, number of 1s in each block)."""
    out, votes = [], []
    for i in range(0, len(received), n):
        ones = sum(received[i:i + n])
        votes.append(ones)
        out.append(1 if ones > n // 2 else 0)
    return out, votes


def flip(bits, positions):
    """Return a copy of bits with the given 0-based positions inverted."""
    r = list(bits)
    for p in positions:
        r[p] ^= 1
    return r


if __name__ == "__main__":
    msg = [1, 0, 1]
    c = encode(msg)
    print("message", msg, "-> codeword", c)
    r = flip(c, [1, 3, 6, 9])        # 2 errors in block 1, 2 errors in block 2
    print("received", r)
    dec, votes = decode(r)
    print("ones per block", votes, "-> decoded", dec, "OK" if dec == msg else "FAIL")
    r3 = flip(c, [0, 1, 2])          # 3 errors in block 1
    print("3 errors in one block ->", decode(r3)[0], "(failure expected)")
```

bch.py

```python
"""bch.py -- binary BCH (15,7) code, corrects up to t = 2 errors (optional
advanced extension of Chapter 14).

Field GF(16) built from the primitive polynomial p(x) = x^4 + x + 1, alpha = x.
Generator polynomial g(x) = m1(x) m3(x)
       m1(x) = x^4 + x + 1                (minimal polynomial of alpha)
       m3(x) = x^4 + x^3 + x^2 + x + 1    (minimal polynomial of alpha^3)
```

```
       g(x)  = x^8 + x^7 + x^6 + x^4 + 1
Bit lists hold polynomial coefficients, index i = coefficient of x^i.
Systematic encoding: c(x) = x^8 m(x) + [x^8 m(x) mod g(x)],
so c[0..7] are parity bits and c[8..14] are the 7 message bits.
Decoding (Peterson): syndromes S1 = r(alpha), S3 = r(alpha^3);
  no error  : S1 = 0 (then S3 = 0)
  one error : S3 = S1^3, error at position j with alpha^j = S1
  two errors: error-locator  1 + S1 x + ((S3 + S1^3)/S1) x^2, roots by Chien search.
"""
N, K = 15, 7
G_POLY = [1, 0, 0, 0, 1, 0, 1, 1, 1]          # g(x) = 1 + x^4 + x^6 + x^7 + x^8

# ---------------------------------------------------------------- GF(16)
EXP = [0] * 30
LOG = [0] * 16
v = 1
for i in range(15):
    EXP[i] = v
    LOG[v] = i
    v <<= 1
    if v & 0x10:
        v ^= 0b10011                           # reduce with x^4 + x + 1
for i in range(15, 30):
    EXP[i] = EXP[i - 15]


def gmul(a, b):
    if a == 0 or b == 0:
        return 0
    return EXP[LOG[a] + LOG[b]]


def gdiv(a, b):
    if a == 0:
        return 0
    return EXP[(LOG[a] - LOG[b]) % 15]


def gpow(a, k):
    if a == 0:
        return 0
    return EXP[(LOG[a] * k) % 15]


# ---------------------------------------------------------------- encoder
def poly_mod(dividend, divisor):
    r = list(dividend)
    for i in range(len(r) - 1, len(divisor) - 2, -1):
        if r[i]:
            shift = i - (len(divisor) - 1)
            for j, gj in enumerate(divisor):
                r[shift + j] ^= gj
    return r[:len(divisor) - 1]

```

```
62
63  def encode(m):
64      """m: 7 message bits (m[0] = coefficient of x^0). Returns 15 code bits."""
65      shifted = [0] * 8 + list(m)                 # x^8 m(x)
66      parity = poly_mod(shifted, G_POLY)
67      return parity + list(m)
68
69
70  def syndromes(r):
71      s1 = s3 = 0
72      for i, bit in enumerate(r):
73          if bit:
74              s1 ^= EXP[i % 15]
75              s3 ^= EXP[(3 * i) % 15]
76      return s1, s3
77
78
79  def decode(r):
80      """Returns (message bits, corrected codeword, error positions found)."""
81      r = list(r)
82      s1, s3 = syndromes(r)
83      if s1 == 0 and s3 == 0:
84          return r[8:], r, []
85      if s1 != 0 and s3 == gpow(s1, 3):           # single error
86          pos = [LOG[s1]]
87      elif s1 == 0:                                # >2 errors: cannot correct
88          return r[8:], r, None
89      else:
90          sigma2 = gdiv(s3 ^ gpow(s1, 3), s1)
91          pos = []
92          for j in range(15):                      # Chien search: test x = alpha^-j
93              x = EXP[(15 - j) % 15]
94              if (1 ^ gmul(s1, x) ^ gmul(sigma2, gmul(x, x))) == 0:
95                  pos.append(j)
96          if len(pos) != 2:
97              return r[8:], r, None                # decoding failure detected
98      for j in pos:
99          r[j] ^= 1
100     return r[8:], r, pos
101
102
103 if __name__ == "__main__":
104     m = [1, 0, 1, 1, 0, 0, 1]
105     c = encode(m)
106     print("message ", m)
107     print("codeword", c, " syndromes", syndromes(c))
108     r = list(c); r[2] ^= 1; r[11] ^= 1
109     mh, ch, pos = decode(r)
110     print("received with errors at positions 2 and 11 -> found", pos,
111           "message OK:", mh == m)
```

7. Line by line

- `repetition.py` lines 15–22: count ones in each block of 5; majority > 2.
- `bch.py` lines 21–31: build the GF(16) tables `EXP` (powers of α) and `LOG`; multiplication adds logarithms.
- lines 53–67: systematic encoding by polynomial division by $g(x)$.
- lines 70–76: syndromes $S_1 = r(\alpha)$, $S_3 = r(\alpha^3)$ as XOR sums of powers.
- lines 79–100: Peterson decoding: no error, single error ($S_3 = S_1^3$), or two errors via $\sigma_2 = (S_3 + S_1^3)/S_1$ and Chien search.

8. Sample input and 9. Output

OUTPUT (real run)

```
message [1, 0, 1] -> codeword [1, 1, 1, 1, 1, 0, 0, 0, 0, 0, 1, 1, 1, 1, 1]
received [1, 0, 1, 0, 1, 0, 1, 0, 0, 1, 1, 1, 1, 1, 1]
ones per block [3, 2, 5] -> decoded [1, 0, 1] OK
3 errors in one block -> [0, 0, 1] (failure expected)
```

OUTPUT (real run)

```
message  [1, 0, 1, 1, 0, 0, 1]
codeword [0, 1, 0, 0, 0, 0, 1, 1, 1, 0, 1, 1, 0, 0, 1]  syndromes (0, 0)
received with errors at positions 2 and 11 -> found [2, 11] message OK: True
```

10. Interpretation

Repetition corrects two errors in each block but fails with three; BCH finds the exact error positions 2 and 11.

11. Common errors

Using an even repetition length (ties); doing ordinary arithmetic instead of GF(16) arithmetic; forgetting that $\alpha^{-j} = \alpha^{15-j}$.

12. Exercises

(a) Show by simulation that (5,1) fails with probability $\approx 10\,p^3$ for small p. (b) Extend BCH to report a failure when 3 errors happen.

15.9 Lab 9 — Markov chains

1. Objectives

Compute stationary distributions and entropy rates, simulate a chain, and estimate a transition matrix from data.

2. Background

Chapter 11.

3. Algorithm and 4. Flowchart



5. Libraries

`random`, `math`.

6. Program

markov.py

```python
"""markov.py -- Markov sources, entropy rate, estimation (Lab 9).

A first-order chain on states 0..S-1 has transition matrix P with
P[i][j] = Pr(next = j | current = i); every row sums to 1.
"""
import random
from math import log2


def stationary(P, iters=10000, tol=1e-14):
    """Stationary distribution pi = pi P by power iteration."""
    S = len(P)
    pi = [1.0 / S] * S
    for _ in range(iters):
        new = [sum(pi[i] * P[i][j] for i in range(S)) for j in range(S)]
        if max(abs(a - b) for a, b in zip(new, pi)) < tol:
            return new
        pi = new
    return pi


def row_entropy(row):
    return -sum(p * log2(p) for p in row if p > 0)


def entropy_rate(P, pi=None):
    """H = sum_i pi_i H(P[i]) bits per symbol (stationary first-order chain)."""
    if pi is None:
        pi = stationary(P)
    return sum(pi[i] * row_entropy(P[i]) for i in range(len(P)))


def sequence_probability(seq, P, p0):
    """P(x1..xn) = p0(x1) * prod P[x_{k-1}][x_k]."""
    pr = p0[seq[0]]
    for a, b in zip(seq, seq[1:]):
        pr *= P[a][b]
    return pr


def estimate(seq, S, alpha=0.0):
    """Maximum-likelihood transition matrix from counts N_ij / N_i
    (alpha > 0 adds Laplace smoothing)."""
```

```
    N = [[alpha] * S for _ in range(S)]
    for a, b in zip(seq, seq[1:]):
        N[a][b] += 1
    P = []
    for row in N:
        t = sum(row)
        P.append([x / t if t else 1.0 / S for x in row])
    return P


def simulate(P, n, start=0, seed=None):
    rng = random.Random(seed)
    s, out = start, [start]
    for _ in range(n - 1):
        u, c = rng.random(), 0.0
        for j, p in enumerate(P[s]):
            c += p
            if u < c:
                s = j
                break
        out.append(s)
    return out


if __name__ == "__main__":
    P = [[0.9, 0.1], [0.5, 0.5]]
    pi = stationary(P)
    print("stationary distribution:", [round(x, 4) for x in pi])
    print("entropy rate: %.4f bits/symbol" % entropy_rate(P))
    print("entropy of pi (memoryless view): %.4f" % row_entropy(pi))
    seq = simulate(P, 20000, seed=1)
    print("estimated P from 20000 samples:", [[round(x, 3) for x in r] for r in estimate(
    seq, 2)])
```

7. Line by line

- Lines 10–19: power iteration $\pi \leftarrow \pi \mathbf{P}$ until it stops changing.
- Lines 26–30: entropy rate formula.
- Lines 41–51: transition counts and maximum-likelihood rows (with optional smoothing α).
- Lines 54–65: simulation with a seeded generator; inverse-CDF sampling of the next state.

8. Sample input and 9. Output

OUTPUT (real run)

```
stationary distribution: [0.8333, 0.1667]
entropy rate: 0.5575 bits/symbol
entropy of pi (memoryless view): 0.6500
estimated P from 20000 samples: [[0.898, 0.102], [0.495, 0.505]]
```

10. Interpretation

$\pi = (5/6, 1/6)$; memory reduces 0.65 bits to 0.5575 bits/symbol; 20 000 samples estimate $\mathbf{P}$ within about 0.005.

11. Common errors

Using columns instead of rows; normalising counts over the whole matrix instead of per row; not seeding the generator.

12. Exercises

(a) Estimate an order-1 letter model from a text and compute its entropy rate. (b) Generate text from it.

15.10 Lab 10 — Channel capacity

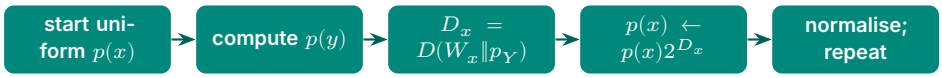
1. Objectives

Compute BSC/BEC capacity, $I(X;Y)$ for any input, and the capacity of any DMC with Blahut–Arimoto.

2. Background

Chapter 13.

3. Algorithm and 4. Flowchart



5. Libraries

`math`, `random`.

6. Program

channel.py

```python
"""channel.py -- discrete memoryless channels and capacity (Lab 10)."""
import random
from math import log2


def h2(p):
    return 0.0 if p in (0, 1) else -p * log2(p) - (1 - p) * log2(1 - p)


def bsc_capacity(p):
    return 1 - h2(p)


def bec_capacity(eps):
    return 1 - eps


def mutual_information(px, W):
```

```
    """I(X;Y) for input distribution px and channel matrix W[x][y] = P(y|x)."""
    ny = len(W[0])
    py = [sum(px[x] * W[x][y] for x in range(len(px))) for y in range(ny)]
    I = 0.0
    for x, pxv in enumerate(px):
        for y in range(ny):
            if pxv > 0 and W[x][y] > 0:
                I += pxv * W[x][y] * log2(W[x][y] / py[y])
    return I


def blahut_arimoto(W, iters=2000, tol=1e-12):
    """Capacity of any DMC: returns (C, best input distribution)."""
    nx, ny = len(W), len(W[0])
    px = [1.0 / nx] * nx
    for _ in range(iters):
        py = [sum(px[x] * W[x][y] for x in range(nx)) for y in range(ny)]
        D = [sum(W[x][y] * log2(W[x][y] / py[y]) for y in range(ny) if W[x][y] > 0)
             for x in range(nx)]
        new = [px[x] * 2 ** D[x] for x in range(nx)]
        s = sum(new)
        new = [v / s for v in new]
        if max(abs(a - b) for a, b in zip(new, px)) < tol:
            px = new
            break
        px = new
    return mutual_information(px, W), px


def bsc(bits, p, seed=None):
    """Flip each bit independently with probability p."""
    rng = random.Random(seed)
    return [b ^ (rng.random() < p) for b in bits]


if __name__ == "__main__":
    for p in (0.0, 0.01, 0.11, 0.5):
        print("BSC p = %.2f  C = %.4f bits/use" % (p, bsc_capacity(p)))
    W = [[0.9, 0.1], [0.2, 0.8]]           # a non-symmetric binary channel
    C, px = blahut_arimoto(W)
    print("Z-like channel W =", W, " C = %.4f, best P(X) = %s" % (C, [round(v, 4) for v in
    px]))
```

7. Line by line

- Lines 18–27: $I(X;Y) = \sum p(x)p(y|x)\log_2 \frac{p(y|x)}{p(y)}$.
- Lines 30–45: Blahut–Arimoto: inputs whose output distributions are far from the average get more weight; converges to capacity.
- Lines 48–51: a BSC simulator for later experiments.

8. Sample input and 9. Output

OUTPUT (real run)

```
BSC p = 0.00  C = 1.0000 bits/use
BSC p = 0.01  C = 0.9192 bits/use
BSC p = 0.11  C = 0.5001 bits/use
BSC p = 0.50  C = 0.0000 bits/use
Z-like channel W = [[0.9, 0.1], [0.2, 0.8]]  C = 0.3978, best P(X) = [0.5176, 0.4824]
```

10. Interpretation

The asymmetric channel's capacity 0.3978 needs a slightly non-uniform input (0.518, 0.482).

11. Common errors

Using channel matrices with columns summing to 1; taking log of zero transition probabilities.

12. Exercises

(a) Compute the Z-channel capacity for $q = 0.1..0.9$. (b) Verify $C_{\text{BEC}} = 1 - \epsilon$ with Blahut–Arimoto.

15.11 Lab 11 — Entropy-based anomaly detection

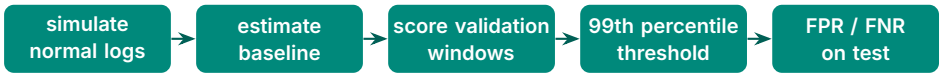
1. Objectives

Learn a normal behaviour model from synthetic logs, score windows with KL divergence and Markov log-loss, choose thresholds and measure error rates.

2. Background

Chapter 12.

3. Algorithm and 4. Flowchart



5. Libraries

`random`, `math`, Lab 9's `markov`.

6. Program

anomaly.py

```python
"""anomaly.py -- entropy- and KL-based anomaly detection (Lab 11).

A 'normal' user is modelled by a Markov chain over 4 actions
  0 = login, 1 = read, 2 = write, 3 = logout.
A baseline model is learned from training data. Each new window of W actions
is scored by the KL divergence D(empirical || baseline) of its action
frequencies and by the average log-loss under the Markov baseline.
A window is flagged when its score exceeds a threshold chosen from normal
validation windows (e.g. the 99th percentile).
"""
import random
from math import log2
from markov import estimate, simulate, stationary
```

```
14
15 ACTIONS = ["login", "read", "write", "logout"]
16 P_NORMAL = [[0.05, 0.80, 0.10, 0.05],
17             [0.02, 0.60, 0.25, 0.13],
18             [0.02, 0.50, 0.30, 0.18],
19             [0.90, 0.04, 0.03, 0.03]]
20 P_ATTACK = [[0.60, 0.10, 0.05, 0.25],     # repeated login attempts
21             [0.05, 0.30, 0.55, 0.10],     # unusual amount of writing
22             [0.05, 0.20, 0.65, 0.10],
23             [0.85, 0.05, 0.05, 0.05]]
24
25
26 def freq(window, S=4, alpha=0.5):
27     c = [alpha] * S                         # add-alpha smoothing avoids log(0)
28     for a in window:
29         c[a] += 1
30     t = sum(c)
31     return [x / t for x in c]
32
33
34 def kl(p, q):
35     return sum(pi * log2(pi / qi) for pi, qi in zip(p, q) if pi > 0)
36
37
38 def log_loss(window, P):
39     """Average -log2 P(x_k | x_{k-1}) in bits per action (cross-entropy rate)."""
40     return -sum(log2(P[a][b]) for a, b in zip(window, window[1:])) / (len(window) - 1)
41
42
43 def windows(seq, W):
44     return [seq[i:i + W] for i in range(0, len(seq) - W + 1, W)]
45
46
47 def percentile(values, q):
48     v = sorted(values)
49     return v[min(len(v) - 1, int(q * len(v)))]
50
51
52 def run(W=50, seed=7):
53     train = simulate(P_NORMAL, 20000, seed=seed)
54     P_hat = estimate(train, 4, alpha=1.0)           # smoothed MLE baseline
55     base = stationary(P_hat)
56     valid = windows(simulate(P_NORMAL, 50 * W * 4, seed=seed + 1), W)
57     test_norm = windows(simulate(P_NORMAL, 200 * W, seed=seed + 2), W)
58     test_att = windows(simulate(P_ATTACK, 200 * W, seed=seed + 3), W)
59     res = {}
60     for name, score in [("KL", lambda w: kl(freq(w), base)),
61                         ("log-loss", lambda w: log_loss(w, P_hat))]:
62         thr = percentile([score(w) for w in valid], 0.99)
63         fp = sum(score(w) > thr for w in test_norm) / len(test_norm)
64         fn = sum(score(w) <= thr for w in test_att) / len(test_att)
65         res[name] = (thr, fp, fn)
66     return P_hat, base, res
67
```

```

if __name__ == "__main__":
    P_hat, base, res = run()
    print("baseline action frequencies:", [round(x, 3) for x in base])
    for name, (thr, fp, fn) in res.items():
        print("%-8s threshold %.3f  false positive rate %.3f  false negative rate %.3f"
              % (name, thr, fp, fn))
```

7. Line by line

- Lines 16–24: the normal and attack behaviour matrices (actions login, read, write, logout).
- Lines 26–31: smoothed window frequencies (add-0.5).
- Lines 38–40: log-loss in bits per action under the estimated Markov model.
- Lines 52–66: train, validate (threshold = 99th percentile of normal scores), and test on 200 normal and 200 attack windows.

8. Sample input and 9. Output

OUTPUT (real run)

```
baseline action frequencies: [0.128, 0.541, 0.212, 0.118]
KL       threshold 0.200  false positive rate 0.000  false negative rate 0.005
log-loss threshold 1.695  false positive rate 0.000  false negative rate 0.010
```

10. Interpretation

No false alarm on 200 normal windows and 1–2 misses out of 200 attack windows. Fig. 12.2 shows the score distributions and error curves.

11. Common errors

Choosing the threshold on test data (optimistic results); no smoothing (infinite KL); windows too short to estimate frequencies.

12. Exercises

(a) Design an attack that keeps normal frequencies but changes transitions; which score detects it? (b) Plot FPR and FNR against the window length W.

15.12 Running all tests

test_itlab.py

```
"""test_itlab.py -- unit tests for every lab.  Run:  python test_itlab.py"""
import itertools, random, unittest
from fractions import Fraction
from math import log2
import entropy as E, shannon_fano as sf, huffman as hf, arithmetic as ac, lzw
import hamming as hm, repetition as rp, bch, markov as mk, channel as ch

rng = random.Random(1)

```

```

def random_text(n, alphabet):
    return "".join(rng.choice(alphabet) for _ in range(n))


def is_prefix_free(codes):
    cw = list(codes.values())
    return not any(a != b and b.startswith(a) for a in cw for b in cw)


class TestEntropy(unittest.TestCase):
    def test_values(self):
        self.assertAlmostEqual(E.entropy([0.5, 0.5]), 1.0)
        self.assertAlmostEqual(E.entropy([0.25] * 4), 2.0)
        self.assertAlmostEqual(E.entropy([1.0]), 0.0)
        self.assertAlmostEqual(E.binary_entropy(0.11), 0.4999, places=3)

    def test_identities(self):
        P = [[0.1, 0.2, 0.1], [0.3, 0.05, 0.25]]
        px, py = E.marginals(P)
        I = E.mutual_information(P)
        self.assertGreaterEqual(I, 0)
        self.assertAlmostEqual(E.joint_entropy(P), E.entropy(px) + E.conditional_entropy(P)
    )
        # independence -> I = 0
        Q = [[a * b for b in [0.3, 0.7]] for a in [0.4, 0.6]]
        self.assertAlmostEqual(E.mutual_information(Q), 0.0)
        # I = D(p_xy || p_x p_y)
        flat = [p for r in P for p in r]
        prod = [a * b for a in px for b in py]
        self.assertAlmostEqual(I, E.kl_divergence(flat, prod))


class TestSourceCodes(unittest.TestCase):
    def test_prefix_kraft_bounds(self):
        for _ in range(200):
            k = rng.randint(2, 12)
            w = [rng.random() + 0.01 for _ in range(k)]
            probs = {chr(65 + i): x / sum(w) for i, x in enumerate(w)}
            H = E.entropy(probs)
            for coder in (sf.shannon_fano, hf.huffman):
                codes = coder(probs)
                self.assertTrue(is_prefix_free(codes))
                self.assertLessEqual(sum(2 ** -len(c) for c in codes.values()), 1 + 1e-12)
                L = sum(probs[s] * len(codes[s]) for s in probs)
                self.assertGreaterEqual(L, H - 1e-9)
                self.assertLess(L, H + 1)
            Lh = sum(probs[s] * len(hf.huffman(probs)[s]) for s in probs)
            Ls = sum(probs[s] * len(sf.shannon_fano(probs)[s]) for s in probs)
            self.assertLessEqual(Lh, Ls + 1e-12)            # Huffman is optimal

    def test_round_trip(self):
        for _ in range(50):
            t = random_text(rng.randint(1, 300), "ABCDEFG")
```

```
            p = E.probs_from_text(t)
            for m in (sf, hf):
                c = (sf.shannon_fano if m is sf else hf.huffman)(p)
                self.assertEqual(m.decode(m.encode(t, c), c), t)


class TestArithmetic(unittest.TestCase):
    def test_exact(self):
        probs = {"A": Fraction(1, 2), "B": Fraction(1, 4), "C": Fraction(1, 4)}
        for _ in range(100):
            t = random_text(rng.randint(1, 20), "ABC")
            lo, hi, bits = ac.encode_exact(t, probs)
            self.assertEqual(ac.decode_exact(bits, probs, len(t)), t)
            self.assertLessEqual(len(bits), -log2(hi - lo) + 2)

    def test_integer(self):
        from collections import Counter
        for _ in range(100):
            t = random_text(rng.randint(1, 2000), "ABCDE" if rng.random() < .5 else "
    AAAAAAB")
            c = Counter(t)
            self.assertEqual(ac.decode_int(ac.encode_int(t, c), c, len(t)), t)


class TestLZW(unittest.TestCase):
    def test_round_trip(self):
        for _ in range(200):
            t = random_text(rng.randint(1, 500), "AB" if rng.random() < .5 else "ABCD")
            self.assertEqual(lzw.decode(lzw.encode(t, "ABCD"), "ABCD"), t)
        self.assertEqual(lzw.encode("ABABABA", "AB"), [0, 1, 2, 4])   # special case
        t = "TOBEORNOTTOBEORTOBEORNOT"
        self.assertEqual(lzw.decode(lzw.encode(t)), t)


class TestChannelCodes(unittest.TestCase):
    def test_hamming(self):
        for d in itertools.product([0, 1], repeat=4):
            c = hm.encode74(list(d))
            Gc = [sum(d[i] * hm.G74[i][j] for i in range(4)) % 2 for j in range(7)]
            self.assertEqual(c, Gc)
            self.assertEqual(hm.syndrome74(c), 0)
            for e in range(7):                      # every single error corrected
                r = list(c); r[e] ^= 1
                self.assertEqual(hm.decode74(r)[0], list(d))
            for e1, e2 in itertools.combinations(range(8), 2):   # SECDED detects doubles
                r = hm.encode84(list(d)); r[e1] ^= 1; r[e2] ^= 1
                self.assertEqual(hm.decode84(r)[1], "double error detected")

    def test_hamming_two_errors_fail(self):
        c = hm.encode74([1, 0, 1, 1])
        r = list(c); r[1] ^= 1; r[5] ^= 1
        self.assertNotEqual(hm.decode74(r)[0], [1, 0, 1, 1])

    def test_repetition(self):
```

```
        for b in (0, 1):
            c = rp.encode([b])
            for k in range(6):
                for pos in itertools.combinations(range(5), k):
                    ok = rp.decode(rp.flip(c, pos))[0] == [b]
                    self.assertEqual(ok, k <= 2)

    def test_bch(self):
        for _ in range(30):
            m = [rng.randint(0, 1) for _ in range(7)]
            c = bch.encode(m)
            self.assertEqual(bch.syndromes(c), (0, 0))
            for k in (1, 2):
                for pos in itertools.combinations(range(15), k):
                    r = list(c)
                    for p in pos:
                        r[p] ^= 1
                    mh, chat, found = bch.decode(r)
                    self.assertEqual(chat, c)
                    self.assertEqual(sorted(found), list(pos))
        # minimum distance = 5: check all 128 codewords
        w = [sum(bch.encode(list(m))) for m in itertools.product([0, 1], repeat=7)]
        self.assertEqual(min(x for x in w if x > 0), 5)


class TestMarkovChannel(unittest.TestCase):
    def test_markov(self):
        P = [[0.9, 0.1], [0.5, 0.5]]
        pi = mk.stationary(P)
        self.assertAlmostEqual(pi[0], 5 / 6, places=9)
        self.assertAlmostEqual(mk.entropy_rate(P), 5 / 6 * E.binary_entropy(.1) + 1 / 6,
    places=9)

    def test_capacity(self):
        self.assertAlmostEqual(ch.bsc_capacity(0.11), 1 - E.binary_entropy(0.11))
        C, px = ch.blahut_arimoto([[0.89, 0.11], [0.11, 0.89]])
        self.assertAlmostEqual(C, ch.bsc_capacity(0.11), places=8)
        C, px = ch.blahut_arimoto([[1 - .3, .3, 0], [0, .3, .7]])     # BEC(0.3)
        self.assertAlmostEqual(C, 0.7, places=8)


if __name__ == "__main__":
    unittest.main(verbosity=1)
```

OUTPUT (real run)

```
.............
----------------------------------------------------------------------
Ran 13 tests in 0.672s

OK
```

The 13 tests include: Kraft inequality and $H \le L < H + 1$ for 200 random sources, Huffman never worse than Shannon–Fano, 300 arithmetic and 200 LZW round trips, every single error of

Hamming (7,4), every double error detected by SECDED, every error pattern for the (5,1) code, all 1- and 2-error patterns of BCH(15,7) on 30 messages, its $d_{\min} = 5$, the Markov stationary distribution, and Blahut–Arimoto on the BSC and BEC.

PART VI

Solved Problems, Exams and Final Project

Fifty fully solved problems, an examination question bank with answer keys, and an integrated final project.

CHAPTER

16



Fully Solved Problems

IN THIS CHAPTER YOU WILL LEARN

Fifty-one problems covering every module. Each solution shows the given data, what is required, the equations, the substitutions and steps, the final answer, its meaning, and a typical mistake. All numbers were checked with the `itlab` code.

16.1 Entropy and self-information

Problem 16.1 • Self-information

Given: An event with $p = 1/32$.
Required: I.
Equations: $I = -\log_2 p$.
Solution: $I = -\log_2(1/32) = \log_2 32 = 5$.
Answer: 5 bits.
Interpretation: As surprising as five fair coin tosses all coming up heads.
Common mistake: Using $\log_{10}$ or ln and reporting the result as bits.

Problem 16.2 • Entropy of four symbols

Given: $p = (0.5, 0.2, 0.2, 0.1)$.
Required: H.
Equations: $H = -\sum p \log_2 p$.
Solution: $0.5(1) + 2 \times 0.2(2.3219) + 0.1(3.3219) = 0.5 + 0.9288 + 0.3322$.
Answer: $H = 1.7610$ bits.
Interpretation: Less than $\log_2 4 = 2$: the source is predictable.
Common mistake: Forgetting to multiply each $-\log_2 p$ by p.

Problem 16.3 • Binary entropy

Given: Bit source with $P(1) = 0.3$.
Required: $H_2(0.3)$.
Equations: $H_2(p) = -p\log_2 p - (1-p)\log_2(1-p)$.
Solution: $0.3(1.7370) + 0.7(0.5146) = 0.5211 + 0.3602$.
Answer: 0.8813 bits.
Interpretation: Each output bit carries 0.88 bits of information.
Common mistake: Using only $-p\log_2 p$ and forgetting the $(1-p)$ term.

Problem 16.4 • Uniform source

Given: 8 equally likely messages.
Required: H and the minimum number of yes/no questions.
Equations: $H = \log_2 N$.
Solution: $\log_2 8 = 3$; halving questions find the message in exactly 3.
Answer: 3 bits; 3 questions.
Interpretation: Uniform $\Rightarrow$ maximum entropy for 8 values.
Common mistake: Thinking 8 messages need 8 bits.

Problem 16.5 • PIN strength

Given: A 6-digit PIN chosen uniformly, versus 4 lowercase letters.
Required: Bits of each.
Equations: $H = n\log_2 |\mathcal{A}|$.
Solution: PIN: $6\log_2 10 = 19.93$. Letters: $4\log_2 26 = 18.80$.
Answer: PIN 19.93 bits; letters 18.80 bits.
Interpretation: The 6-digit PIN is slightly stronger (if truly uniform).
Common mistake: Ignoring that human choices are not uniform.

Problem 16.6 • Entropy versus min-entropy

Given: Password distribution $p = (0.6, 0.2, 0.1, 0.1)$.
Required: H and H_∞.
Equations: $H = -\sum p\log_2 p$; $H_\infty = -\log_2 \max p$.
Solution: $H = 0.6(0.7370) + 0.2(2.3219) + 2(0.1)(3.3219) = 0.4422 + 0.4644 + 0.6644 = 1.5710$. $H_\infty = -\log_2 0.6 = 0.7370$.
Answer: $H = 1.571$ bits; $H_\infty = 0.737$ bits.
Interpretation: One guess succeeds 60% of the time: the worst case is much weaker than the average.
Common mistake: Rating a password policy only by Shannon entropy.

Problem 16.7 • Sum of two dice

Given: Two fair dice; S = their sum (2–12, probabilities $1, 2, \ldots, 6, \ldots, 1$ over 36).
Required: $H(S)$.
Equations: $H = -\sum p\log_2 p$.
Solution: $\frac{2}{36}\log_2 36 + \frac{4}{36}\log_2 18 + \frac{6}{36}\log_2 12 + \frac{8}{36}\log_2 9 + \frac{10}{36}\log_2 \frac{36}{5} + \frac{6}{36}\log_2 6$.
Answer: $H(S) = 3.2744$ bits.
Interpretation: Less than $\log_2 11 = 3.459$ (values not equally likely) and less than H(two dice)$= 5.17$ (the sum loses information).
Common mistake: Assuming the 11 sums are equally likely.

Problem 16.8 • Bits per character of a uniform alphabet

Given: 26 letters, uniform.
Required: H.
Equations: $\log_2 N$.
Solution: $\log_2 26 = 4.700$.
Answer: 4.70 bits/letter.
Interpretation: Real English has about 4.1–4.2 bits/letter from single-letter frequencies and much less with context.
Common mistake: Using 8 bits (ASCII size) as the information content.

16.2 Joint, conditional entropy and leakage

Problem 16.9 • Joint and conditional entropy

Given: $p(x, y)$: row $x = 0$: $(0.2, 0.1, 0.1)$; row $x = 1$: $(0.1, 0.3, 0.2)$.
Required: $H(X, Y)$, $H(X)$, $H(Y)$, $H(Y|X)$, $H(X|Y)$.
Equations: Marginals by sums; $H(Y|X) = H(X, Y) - H(X)$; $H(X|Y) = H(X, Y) - H(Y)$.
Solution: $p(x) = (0.4, 0.6)$, $p(y) = (0.3, 0.4, 0.3)$. $H(X, Y) = 2.4464$, $H(X) = 0.9710$, $H(Y) = 1.5710$.
Answer: $H(Y|X) = 1.4755$; $H(X|Y) = 0.8755$ bits.
Interpretation: Knowing Y reduces uncertainty about X by only 0.0955 bits.
Common mistake: Computing $H(X|Y)$ as $H(X)/H(Y)$.

Problem 16.10 • Chain rule check

Given: Same table as Problem 16.9.
Required: Verify $H(X, Y) = H(Y) + H(X|Y)$.
Equations: Chain rule.
Solution: $1.5710 + 0.8755 = 2.4465 \approx 2.4464$.
Answer: Verified (rounding).
Interpretation: Joint uncertainty = uncertainty of one variable + remaining uncertainty of the other.
Common mistake: Adding $H(X) + H(Y)$ (true only for independence).

Problem 16.11 • Weight leak of a 3-bit key

Given: Uniform 3-bit key; the attacker learns its Hamming weight W.
Required: $H(K|W)$ and the leakage.
Equations: $H(K|W) = \sum_w p(w) \log_2(\#\text{keys with weight } w)$.
Solution: Weights 0,1,2,3 occur for 1,3,3,1 keys. $H(K|W) = \frac{3}{8}\log_2 3 + \frac{3}{8}\log_2 3 = 1.1887$.
Answer: $H(K|W) = 1.189$ bits; leakage $3 - 1.189 = 1.811$ bits.
Interpretation: Over 60% of the key's uncertainty leaks.
Common mistake: Thinking a weight leak reveals at most 1 bit.

Problem 16.12 • Parity leak

Given: Uniform 8-bit key; its parity bit leaks.
Required: Remaining uncertainty.
Equations: Each parity value leaves 2^7 keys.
Solution: $H(K|P) = \log_2 128 = 7$.
Answer: 7 bits remain; 1 bit leaked.
Interpretation: Parity halves the key space.
Common mistake: Saying nothing leaks because the parity is "only one bit".

Problem 16.13 • Length metadata

Given: Answers yes/no/maybe with probabilities $0.5, 0.3, 0.2$ and different lengths (3, 2, 5 bytes), encrypted.
Required: Leakage.
Equations: $\mathcal{L} = H(A) - H(A|\text{length})$.
Solution: Length identifies the answer: $H(A|\text{length}) = 0$; $H(A) = 1.4855$.
Answer: All 1.486 bits leak.
Interpretation: Encryption without padding protects nothing here.
Common mistake: Believing encryption hides message size.

Problem 16.14 • Conditional entropy of a BSC

Given: Uniform X, $Y = X \oplus E$ with $P(E = 1) = p$.
Required: $H(X|Y)$.
Equations: $H(X|Y) = H(X) - I(X;Y)$, $I = 1 - H_2(p)$.
Solution: $H(X|Y) = 1 - (1 - H_2(p)) = H_2(p)$.
Answer: $H(X|Y) = H_2(p)$; for $p = 0.1$, 0.469 bits.
Interpretation: The noise entropy is what remains unknown.
Common mistake: Writing $H(X|Y) = p$.

16.3 Mutual information and KL divergence

Problem 16.15 • Mutual information from a table

Given: Table of Problem 16.9.
Required: $I(X;Y)$.
Equations: $I = H(X) + H(Y) - H(X,Y)$.
Solution: $0.9710 + 1.5710 - 2.4464$.
Answer: 0.0955 bits.
Interpretation: Weak dependence.
Common mistake: Getting a negative result from rounding errors; keep 4 decimals.

Problem 16.16 • KL in both directions

Given: $p = (0.7, 0.2, 0.1)$, $q = (\frac{1}{3}, \frac{1}{3}, \frac{1}{3})$.
Required: $D(p\|q)$, $D(q\|p)$.
Equations: $D(p\|q) = \sum p \log_2(p/q)$.
Solution: $D(p\|q) = \log_2 3 - H(p) = 1.5850 - 1.1568 = 0.4282$. $D(q\|p) = \frac{1}{3}[\log_2 \frac{1/3}{0.7} + \log_2 \frac{1/3}{0.2} + \log_2 \frac{1/3}{0.1}] = 0.4678$.
Answer: 0.4282 and 0.4678 bits.
Interpretation: KL is not symmetric.
Common mistake: Treating KL as a distance.

Problem 16.17 • Information through a BSC

Given: BSC with $p = 0.2$, uniform input.
Required: $I(X;Y)$.
Equations: $I = H(Y) - H(Y|X) = 1 - H_2(p)$.
Solution: $1 - 0.7219$.
Answer: 0.2781 bits/use.
Interpretation: This is also the capacity of BSC(0.2).
Common mistake: Using $1 - p$.

Problem 16.18 • Traffic analysis with two sites

Given: Site A (0.6) or B (0.4). Size small with prob. 0.9 for A and 0.3 for B.
Required: $I(W;S)$ and the attacker's best success rate.
Equations: Joint table, $I = H(W) + H(S) - H(W,S)$; MAP guess per observation.
Solution: Joint: A $(0.54, 0.06)$, B $(0.12, 0.28)$; $p(s) = (0.66, 0.34)$. $I = 0.2909$. Small $\to$ A $(0.54/0.66 = 0.82)$, large $\to$ B $(0.28/0.34 = 0.82)$.
Answer: $I = 0.291$ bits; success $0.54 + 0.28 = 0.82$ (vs 0.6 without observation).
Interpretation: Sizes raise the attacker's success by 22 points.
Common mistake: Comparing I with 1 bit instead of $H(W) = 0.971$.

Problem 16.19 • Infinite KL

Given: $p = (0.5, 0.5)$, $q = (1, 0)$.
Required: $D(p\|q)$.
Equations: $p_2 \log_2(p_2/q_2)$ with $q_2 = 0$.
Solution: Term $= 0.5 \log_2(0.5/0) = \infty$.
Answer: $D = \infty$.
Interpretation: A model that forbids an observed event is infinitely wrong; smooth models.
Common mistake: Ignoring the term because $q = 0$.

Problem 16.20 • MI as KL

Given: Table of Problem 16.9.
Required: Check $I = D(p_{XY}\|p_X p_Y)$.
Equations: $D = \sum p(x,y) \log_2 \frac{p(x,y)}{p(x)p(y)}$.
Solution: Product table: $(0.12, 0.16, 0.12)$, $(0.18, 0.24, 0.18)$. Terms: $0.2 \log_2 \frac{0.2}{0.12} + 0.1 \log_2 \frac{0.1}{0.16} + \ldots$
Answer: 0.0955 bits — same as Problem 16.15.
Interpretation: Mutual information is divergence from independence.
Common mistake: Using $p(x) + p(y)$ instead of the product.

16.4 Source coding basics

Problem 16.21 • Kraft check

Given: Lengths $(1, 2, 3, 4, 4)$.
Required: Can a prefix code have them?
Equations: $\sum 2^{-\ell} \le 1$.
Solution: $\frac{1}{2} + \frac{1}{4} + \frac{1}{8} + \frac{1}{16} + \frac{1}{16} = 1$.
Answer: Yes; the tree is complete.
Interpretation: Codes: 0, 10, 110, 1110, 1111.
Common mistake: Checking only that lengths are different.

Problem 16.22 • Average length and efficiency

Given: $p = (0.4, 0.25, 0.15, 0.1, 0.1)$, lengths $(1, 2, 3, 4, 4)$.
Required: L, η.
Equations: $L = \sum p\ell$; $\eta = H/L$.
Solution: $L = 0.4 + 0.5 + 0.45 + 0.4 + 0.4 = 2.15$; $H = 2.1037$.
Answer: $L = 2.15$ bits; $\eta = 97.85\%$.
Interpretation: Very good code.
Common mistake: Dividing L by H.

Problem 16.23 • Shannon code lengths

Given: $p = (0.35, 0.25, 0.2, 0.12, 0.08)$.
Required: Lengths, L, bound check.
Equations: $\ell = \lceil -\log_2 p \rceil$; $H \le L < H + 1$.
Solution: $-\log_2 p = (1.51, 2, 2.32, 3.06, 3.64) \Rightarrow (2, 2, 3, 4, 4)$. $L = 0.7 + 0.5 + 0.6 + 0.48 + 0.32 = 2.6$; $H = 2.1531$; Kraft $= 0.75$.
Answer: $L = 2.6 \in [2.153, 3.153)$.
Interpretation: Valid but not optimal (Kraft sum < 1 wastes branches).
Common mistake: Rounding down ($\lfloor \cdot \rfloor$) and breaking Kraft.

Problem 16.24 • Block coding

Given: Binary source $P(0) = 0.8$; Huffman on pairs.
Required: Bits/symbol.
Equations: Pairs $(0.64, 0.16, 0.16, 0.04)$; Huffman lengths $(1, 2, 3, 3)$.
Solution: $L_2 = 0.64 + 0.32 + 0.48 + 0.12 = 1.56$; per symbol 0.78. $H = 0.7219$.
Answer: 0.78 bits/symbol ($\eta = 92.6\%$, versus 72.2% for single symbols).
Interpretation: Blocks approach the entropy.
Common mistake: Forgetting to divide by the block length.

16.5 Shannon–Fano

Problem 16.25 • Shannon–Fano construction

Given: A .30, B .25, C .20, D .15, E .10.
Required: Code, L, η.
Equations: Balanced splits.
Solution: Split {A,B} (0.55) | {C,D,E} (0.45); A|B; C|{D,E}; D|E. Codes A 00, B 01, C 10, D 110, E 111.
Answer: $L = 2 \cdot 0.75 + 3 \cdot 0.25 = 2.25$; $H = 2.2282$; $\eta = 99.0\%$.
Interpretation: Near-optimal here.
Common mistake: Splitting by symbol count (2|3) rather than probability.

Problem 16.26 • Shannon–Fano decoding

Given: Code of Problem 16.25; bits `011111100010`.
Required: Message.
Equations: Prefix decoding.
Solution: 01 = B, 111 = E, 110 = D, 00 = A, 10 = C.
Answer: BEDAC.
Interpretation: No separators are needed.
Common mistake: Trying to read fixed 2- or 3-bit groups.

Problem 16.27 • Redundancy

Given: Problem 16.25.
Required: Redundancy in percent and bits.
Equations: $1-\eta$; $L-H$.
Solution: $1-0.990=1.0\%$; $2.25-2.2282=0.0218$ bits.
Answer: 1.0%, 0.022 bits/symbol.
Interpretation: Almost nothing to gain from a better code.
Common mistake: Using H/L as the redundancy.

16.6 Huffman

Problem 16.28 • Huffman construction

Given: A .35, B .20, C .20, D .15, E .10.
Required: Merges, code, L.
Equations: Merge two smallest repeatedly.
Solution: E+D = 0.25; B+C = 0.40; ED+A = 0.60; BC+EDA = 1. Codes B 00, C 01, E 100, D 101, A 11.
Answer: $L=2(0.35+0.2+0.2)+3(0.15+0.1)=2.25$; $H=2.2016$; $\eta=97.8\%$.
Interpretation: The most probable symbol A gets 2 bits like B and C — still optimal.
Common mistake: Expecting the most probable symbol always to get the shortest unique code.

Problem 16.29 • Huffman of a word

Given: MISSISSIPPI: I 4, S 4, P 2, M 1.
Required: Code and bit count.
Equations: Merge counts.
Solution: M1+P2=3; MP3+I4=7; S4+MPI7=11. Codes S 0, M 100, P 101, I 11.
Answer: Encoded length $4(1)+4(2)+2(3)+1(3)=21$ bits ($L=1.909$; $H=1.823$).
Interpretation: 88 ASCII bits $\rightarrow$ 21 bits.
Common mistake: Merging I and S first (they are not the two smallest).

Problem 16.30 • Compression ratio

Given: Problem 16.29.
Required: Ratio versus 8-bit ASCII and versus a 2-bit fixed code.
Equations: ratio = original / compressed.
Solution: $88/21=4.19$; $22/21=1.05$.
Answer: 4.19 and 1.05.
Interpretation: Most of the gain against ASCII comes from the small alphabet, not from Huffman.
Common mistake: Comparing with ASCII only.

Problem 16.31 • Ties

Given: Four symbols, each 0.25.
Required: Huffman lengths.
Equations: Merges.
Solution: 0.25+0.25, 0.25+0.25, 0.5+0.5.
Answer: All lengths 2; $L=H=2$.
Interpretation: For uniform dyadic sources Huffman = fixed length.
Common mistake: Thinking ties give different average lengths (they do not).

Problem 16.32 • Huffman encoding

Given: Code of Problem 16.28; message BEAD.
Required: Bit string.
Equations: Concatenate.
Solution: B 00, E 100, A 11, D 101.
Answer: `0010011101` (10 bits).
Interpretation: Average 2.5 bits/letter for this message.
Common mistake: Inserting spaces or separators.

16.7 Arithmetic coding

Problem 16.33 • Interval of a message

Given: A $[0, 0.7)$, B $[0.7, 0.9)$, C $[0.9, 1)$; message AAB.
Required: Final interval.
Equations: low $\leftarrow$ low $+ wF(s^-)$, high $\leftarrow$ low $+ wF(s)$.
Solution: A: $[0, 0.7)$; A: $[0, 0.49)$; B: $[0 + 0.49(0.7),\ 0.49(0.9)) = [0.343, 0.441)$.
Answer: $[0.343, 0.441)$, width $0.098 = 0.7 \cdot 0.7 \cdot 0.2$.
Interpretation: Information $-\log_2 0.098 = 3.35$ bits.
Common mistake: Updating low before computing high with the old low.

Problem 16.34 • Tag bits

Given: Same table; message CAB.
Required: Interval and shortest tag.
Equations: Tag rule of Chapter 8.
Solution: C: $[0.9, 1)$; A: $[0.9, 0.97)$; B: $[0.949, 0.963)$. $k = 7$: $0.1111010_2 = 0.953125$, and $0.953125 + \frac{1}{128} = 0.9609 \leq 0.963$.
Answer: Tag `1111010` (7 bits).
Interpretation: $-\log_2(0.014) = 6.16$, so 7 bits is within the bound $\lceil 6.16 \rceil + 1$.
Common mistake: Choosing a number at the edge of the interval.

Problem 16.35 • Arithmetic decoding

Given: Same table; tag `0110` ($v = 0.375$); 3 symbols.
Required: Message.
Equations: $t = (v - \text{low})/w$.
Solution: $t = 0.375 \to$ A, $[0, 0.7)$; $t = 0.536 \to$ A, $[0, 0.49)$; $t = 0.765 \to$ B.
Answer: AAB.
Interpretation: Consistent with Problem 16.33.
Common mistake: Not rescaling v into the current interval.

Problem 16.36 • Bit budget

Given: 1000 symbols from $p = (0.7, 0.2, 0.1)$.
Required: Approximate arithmetic-coded size.
Equations: $\approx nH + 2$.
Solution: $H = 1.1568$; $1000 \times 1.1568 + 2$.
Answer: About 1159 bits.
Interpretation: Huffman would need $1000 \times 1.3 = 1300$ bits (codes 0, 10, 11).
Common mistake: Forgetting the model/table overhead in file sizes.

16.8 LZW

Problem 16.37 • LZW encoding

Given: AABAABAAB, dictionary A=0, B=1.
Required: Codes and bits.
Equations: LZW rules; widths $\lceil \log_2(n_0 + k) \rceil$.
Solution: A→0 (AA=2); A→0 (AB=3); B→1 (BA=4); AA→2 (AAB=5); BA→4 (BAA=6); AB→3.
Answer: Codes 0 0 1 2 4 3; $1+2+2+3+3+3 = 14$ bits.
Interpretation: 9 symbols in 14 bits: no gain yet (1-bit raw code needs 9).
Common mistake: Adding entries for the last match.

Problem 16.38 • LZW special case

Given: Codes 1 0 2 4, dictionary A=0, B=1.
Required: Decoded text.
Equations: Unknown code $\Rightarrow w + w[0]$.
Solution: 1→B; 0→A (BA=2); 2→BA (AB=3); 4 unknown → BA+B = BAB (BAB=4).
Answer: BABABAB.
Interpretation: The special case appears in alternating patterns.
Common mistake: Rejecting code 4 as invalid.

Problem 16.39 • Code widths

Given: ASCII start (256 entries), 100 output codes.
Required: Bits with dynamic widths.
Equations: Width of code k: $\lceil \log_2(256 + k) \rceil$.
Solution: $k = 0$: 8 bits; $k = 1..99$: 9 bits.
Answer: $8 + 99 \times 9 = 899$ bits.
Interpretation: Fixed 12-bit codes would need 1200 bits.
Common mistake: Starting with 9 bits for the first code.

16.9 Markov sources

Problem 16.40 • Stationary distribution

Given: $\mathbf{P} = \left[\begin{smallmatrix} 0.8 & 0.2 \\ 0.4 & 0.6 \end{smallmatrix}\right]$.
Required: π.
Equations: $\pi\mathbf{P} = \pi$.
Solution: $0.2\pi_0 = 0.4\pi_1 \Rightarrow \pi_0 = 2\pi_1$; $\pi = (\frac{2}{3}, \frac{1}{3})$.
Answer: $(0.667, 0.333)$.
Interpretation: Long-run fraction of time in each state.
Common mistake: Using columns of $\mathbf{P}$.

Problem 16.41 • Entropy rate

Given: $\mathbf{P}$ of Problem 16.40.
Required: $\mathcal{H}$.
Equations: $\sum \pi_i H(\text{row } i)$.
Solution: $\frac{2}{3}H_2(0.2) + \frac{1}{3}H_2(0.4) = \frac{2}{3}(0.7219) + \frac{1}{3}(0.9710)$.
Answer: 0.8049 bits/symbol.
Interpretation: Less than $H(\pi) = 0.918$: memory helps prediction.
Common mistake: Using $H(\pi)$ as the entropy rate.

Problem 16.42 • Three-state chain

Given: $\mathbf{P} = \left[\begin{smallmatrix} 0.5 & 0.5 & 0 \\ 0.25 & 0.5 & 0.25 \\ 0 & 0.5 & 0.5 \end{smallmatrix}\right]$.
Required: π and $\mathcal{H}$.
Equations: As above.
Solution: $\pi = (0.25, 0.5, 0.25)$ (check: $0.5(0.25) + 0.25(0.5) = 0.25$). Row entropies $1, 1.5, 1$.
Answer: $\mathcal{H} = 0.25 + 0.75 + 0.25 = 1.25$ bits.
Interpretation: A random walk on a line.
Common mistake: Not checking that π sums to 1.

Problem 16.43 • Sequence probability

Given: Chain of Problem 16.40, stationary start; sequence 0 0 1 1 0.
Required: Probability.
Equations: $\pi_{x_1} \prod P$.
Solution: $\frac{2}{3} \times 0.8 \times 0.2 \times 0.6 \times 0.4$.
Answer: 0.0256.
Interpretation: Surprise $-\log_2 = 5.29$ bits.
Common mistake: Multiplying by π at every step.

16.10 Channels

Problem 16.44 • BSC capacity

Given: BSC, $p = 0.2$.
Required: C.
Equations: $1 - H_2(p)$.
Solution: $1 - 0.7219$.
Answer: 0.2781 bits/use.
Interpretation: Need ≥ 3.6 channel bits per data bit.
Common mistake: Using $1 - p = 0.8$.

Problem 16.45 • BEC capacity

Given: BEC, $\epsilon = 0.25$; 2000 uses.
Required: Maximum reliable data.
Equations: $C = 1 - \epsilon$.
Solution: 0.75×2000.
Answer: 1500 bits.
Interpretation: Erasures are easier than errors.
Common mistake: Using $1 - H_2(0.25)$.

Problem 16.46 • Cascade and DPI

Given: Two BSC(0.05) in series.
Required: Equivalent crossover and capacity.
Equations: $p = p_1(1 - p_2) + p_2(1 - p_1)$.
Solution: $p = 2(0.05)(0.95) = 0.095$; $C = 1 - H_2(0.095) = 0.5471$ (single link: 0.7136).
Answer: 0.547 bits/use.
Interpretation: Processing loses information (DPI).
Common mistake: Adding $p_1 + p_2$.

16.11 Error control

Problem 16.47 • Hamming encoding

Given: Data 1101 $(d_1..d_4)$.
Required: Codeword.
Equations: $p_1 = d_1 \oplus d_2 \oplus d_4$, $p_2 = d_1 \oplus d_3 \oplus d_4$, $p_4 = d_2 \oplus d_3 \oplus d_4$.
Solution: $p_1 = 1 \oplus 1 \oplus 1 = 1$; $p_2 = 1 \oplus 0 \oplus 1 = 0$; $p_4 = 1 \oplus 0 \oplus 1 = 0$.
Answer: 1 0 1 0 1 0 1.
Interpretation: Positions 1,2,4 are parity.
Common mistake: Putting data in positions 1–4.

Problem 16.48 • Hamming syndrome decoding

Given: Data 0101 sent as 0100101; received 0110101.
Required: Syndrome and correction.
Equations: s_1, s_2, s_4 checks.
Solution: $s_1 = 0 \oplus 1 \oplus 1 \oplus 1 = 1$; $s_2 = 1 \oplus 1 \oplus 0 \oplus 1 = 1$; $s_4 = 0 \oplus 1 \oplus 0 \oplus 1 = 0 \Rightarrow 011_2 = 3$.
Answer: Flip position 3: 0100101; data 0101.
Interpretation: Single error corrected.
Common mistake: Reading the syndrome bits in the wrong order $(s_1 s_2 s_4)$.

Problem 16.49 • SECDED

Given: Data 1110 $\rightarrow$ 0010110 | 1. Bits 1 and 7 flipped: 1010111 | 1.
Required: Decision.
Equations: Decision table.
Solution: Syndrome of the first 7 bits $= 6 \neq 0$; overall parity of 8 bits is even.
Answer: Double error detected (no correction).
Interpretation: Plain (7,4) would wrongly flip bit 6.
Common mistake: Correcting whenever $s \neq 0$.

Problem 16.50 • Two errors with (5,1)

Given: Bit 0 sent as 00000; received 01001.
Required: Decoded bit.
Equations: Majority.
Solution: Two ones, three zeros.
Answer: 0 (correct).
Interpretation: Two errors corrected.
Common mistake: Decoding by the first bit only.

Problem 16.51 • Repetition versus Hamming

Given: BSC $p = 0.02$; protect 4 data bits.
Required: Failure probability for uncoded, Hamming (7,4), and (5,1) per bit.
Equations: $1 - (1-p)^4$; $P(\geq 2 \text{ in } 7)$; $1 - (1 - P_3)^4$, $P_3 = \sum_{k \geq 3} \binom{5}{k} p^k (1-p)^{5-k}$.
Solution: Uncoded 0.0776; Hamming 0.00786; $P_3 = 7.76 \times 10^{-5}$, repetition 3.10×10^{-4}.
Answer: 7.8×10^{-2}, 7.9×10^{-3}, 3.1×10^{-4} using 4, 7 and 20 channel bits.
Interpretation: More redundancy buys reliability.
Common mistake: Comparing codes at different numbers of channel bits without saying so.

CHAPTER

17



Examination Question Bank

IN THIS CHAPTER YOU WILL LEARN

40 multiple-choice questions, 25 short-answer questions, 20 numerical exercises and 10 programming challenges. Answer keys and full solutions are in Section 17.5. Instructors can draw midterm and final papers from this bank.

17.1 Multiple-choice questions

M1. The self-information of an event with probability $1/4$ is (a) 0.25 bits (b) 4 bits (c) 2 bits (d) -2 bits.

M2. Entropy is maximum when the distribution is (a) binary (b) uniform (c) deterministic (d) skewed.

M3. $H_2(0.5)$ equals (a) 0.5 (b) 1 (c) 2 (d) 0.

M4. The entropy of a source with N equally likely symbols is (a) N (b) $\log_2 N$ (c) $1/N$ (d) $N \log_2 N$.

M5. Which is *always* true? (a) $H(X|Y) \geq H(X)$ (b) $H(X|Y) \leq H(X)$ (c) $I(X;Y) < 0$ is possible (d) $H(X,Y) > H(X) + H(Y)$.

M6. $I(X;Y) = 0$ exactly when (a) X, Y independent (b) $H(X) = H(Y)$ (c) $X = Y$ (d) Y is binary.

M7. The chain rule states (a) $I = H(X,Y)$ (b) $H(X|Y) = H(X) + H(Y)$ (c) $H(X,Y) = H(X)H(Y|X)$ (d) $H(X,Y) = H(X) + H(Y|X)$.

M8. KL divergence $D(p\|q)$ is (a) non-negative (b) symmetric (c) a metric (d) always ≤ 1.

M9. For security against guessing, the best single measure is (a) KL divergence (b) joint entropy (c) min-entropy (d) Shannon entropy.

M10. Leakage of an observation O about a secret S is (a) $H(S) + H(O)$ (b) $H(S) - H(S|O)$ (c) $H(O)$ (d) $H(S|O)$.

M11. A prefix-free code is (a) always fixed-length (b) lossy (c) never optimal (d) instantaneously decodable.

M12. Kraft's inequality is (a) $\sum p_i \ell_i \geq 1$ (b) $\sum 2^{-\ell_i} \leq 1$ (c) $\sum 2^{\ell_i} \leq 1$ (d) $\sum \ell_i \leq H$.

M13. For the best symbol code, L satisfies (a) $H \leq L < H+1$ (b) $L < H$ (c) $L \geq H+1$ (d) $L = 2H$.

M14. Shannon–Fano coding builds the tree (a) from intervals (b) top-down by balanced splits (c) bottom-up (d) from a dictionary.

M15. Huffman coding first merges (a) the two least probable symbols (b) the two most probable symbols (c) the first two symbols alphabetically (d) random symbols.

M16. Huffman codes are optimal among (a) symbol-by-symbol prefix codes (b) dictionary codes (c) all compressors (d) lossy codes.

M17. Building a Huffman code for n symbols with a heap costs (a) $O(n^2)$ (b) $O(2^n)$ (c) $O(n)$ (d) $O(n \log n)$.

M18. Arithmetic coding represents a message by (a) a number in an interval (b) a parity bit (c) a dictionary index (d) a tree.

M19. After encoding a message, the interval width equals (a) the message probability (b) 1/2 (c) the message length (d) the entropy.

M20. An arithmetic decoder knows when to stop by (a) the interval width (b) a length or terminator symbol (c) it cannot stop (d) parity.

M21. LZW sends (a) Huffman tables (b) the dictionary (c) only codes (d) probabilities.

M22. The LZW decoder's special case happens when (a) the dictionary is full (b) a code equals the next free code (c) a code is 0 (d) the input is empty.

M23. LZW compresses well when data (a) has repeated phrases (b) is encrypted (c) is random (d) is short.

M24. Encrypted data compresses (a) like text (b) hardly at all (c) very well (d) to zero.

M25. CRIME/BREACH attacks exploit (a) channel capacity (b) parity bits (c) compressed length (d) Hamming distance.

M26. The entropy rate of a stationary source is (a) $\geq H(X_1)$ (b) always 1 (c) undefined with memory (d) $\leq H(X_1)$.

M27. The stationary distribution satisfies (a) $\sum \pi_i = 0$ (b) $\pi \mathbf{P} = \pi$ (c) $\pi = \mathbf{P}^{-1}$ (d) $\mathbf{P}\pi = 0$.

M28. The AEP says typical sequences have probability about (a) 2^{-nH} (b) H/n (c) $1/n$ (d) 2^{-n}.

M29. In anomaly detection, a false positive is (a) an attack flagged (b) an attack missed (c) a normal event flagged (d) a correct rejection.

M30. Smoothing is used before KL scoring to avoid (a) slow code (b) symmetric results (c) negative values (d) division by zero / infinite KL.

M31. Capacity of a BSC with crossover p is (a) p (b) $1-p$ (c) $H_2(p)$ (d) $1-H_2(p)$.

M32. Capacity of a BEC with erasure ϵ is (a) ϵ (b) 1 (c) $1-\epsilon$ (d) $1-H_2(\epsilon)$.

M33. The data processing inequality says (a) C increases (b) processing increases information (c) $I(X;Z) \le I(X;Y)$ for $X \to Y \to Z$ (d) $H(Z) \le H(X)$ always.

M34. A code with $d_{\min} = 3$ corrects (a) 2 (b) 0 (c) 3 errors (d) 1.

M35. Hamming (7,4) has rate (a) 4/7 (b) 1/2 (c) 3/7 (d) 7/4.

M36. With two errors, Hamming (7,4) (a) may silently miscorrect (b) always detects them (c) corrects both (d) stops.

M37. SECDED adds (a) a key (b) three parity bits (c) one overall parity bit (d) a CRC.

M38. The (5,1) repetition code corrects up to (a) 3 (b) 1 (c) 5 errors per block (d) 2.

M39. BCH(15,7) uses arithmetic in (a) integers mod 15 (b) GF(16) (c) real numbers (d) GF(15).

M40. Integrity against an active attacker requires (a) a MAC (b) a CRC (c) a Hamming code (d) repetition.

17.2 Short-answer questions

S1. Define information and explain how it differs from data.

S2. State the three axioms that characterise Shannon entropy.

S3. Why is a certain event worth 0 bits?

S4. Explain the difference between Shannon entropy and min-entropy with a password example.

S5. Explain why a single observation can increase uncertainty although $H(X|Y) \le H(X)$.

S6. Give two examples of side channels and model one as a channel $p(o|s)$.

S7. Why is KL divergence not a distance?

S8. Interpret the entropy Venn diagram.

S9. What is the difference between uniquely decodable and instantaneous codes?

S10. Prove that no uniquely decodable code has $L < H$ (sketch).

S11. Why can Shannon–Fano be suboptimal?

S12. State the two lemmas behind Huffman optimality.

S13. Why can Huffman not compress a binary source, and how can it be fixed?

S14. Explain E1, E2 and E3 scaling in arithmetic coding.

S15. Explain the LZW special case with an example.

S16. Why does LZW sometimes expand data?

S17. Why must headers be counted when comparing compressors?

S18. Describe how compression can leak secrets in encrypted traffic, and two defences.

S19. Define entropy rate and explain why it is smaller for sources with memory.

S20. State the AEP and its meaning for compression.

S21. How do you choose an anomaly threshold without attack data?

S22. State Shannon's noisy-channel coding theorem.

S23. What is the secrecy capacity of a wiretap channel? Give the BSC formula.

S24. Explain why error correction does not provide integrity against attackers.

S25. Compare (5,1) repetition, Hamming (7,4) and BCH(15,7) in rate and correction power.

17.3 Numerical exercises

N1. Compute H for $(0.25, 0.25, 0.25, 0.125, 0.125)$.

N2. Compute $H_2(0.05)$.

N3. Self-information of an event with probability 0.001.

N4. Bits of an 8-character password from 62 symbols (uniform).

N5. For $p(x, y)$: row 0 $(0.25, 0.25)$, row 1 $(0.5, 0)$, compute $H(X, Y)$, $H(Y)$, $H(Y|X)$ and $I(X; Y)$.

N6. Compute $D(p\|q)$ for $p = (0.5, 0.5)$, $q = (0.75, 0.25)$.

N7. Kraft sum of lengths $(2, 2, 2, 3, 3)$. Is the code complete?

N8. Huffman code and L for $(0.5, 0.2, 0.15, 0.1, 0.05)$.

N9. Shannon–Fano code and L for the same source.

N10. Efficiency of the code in N8.

N11. Arithmetic interval of BA with A $[0, 0.6)$, B $[0.6, 1)$.

N12. LZW codes and dynamic-width bits for ABBABBA (A=0, B=1).

N13. Stationary distribution of $\left[\begin{smallmatrix} 0.7 & 0.3 \\ 0.6 & 0.4 \end{smallmatrix}\right]$.

N14. Entropy rate of the chain in N13.

N15. Capacity of a BSC with $p = 0.15$.

N16. Reliable bits over 5000 uses of BEC(0.1).

N17. Hamming (7,4) codeword for data 1001.

N18. Decode the Hamming (7,4) word 1011110.

N19. Correction capability of a code with $d_{\min} = 9$.

N20. Failure probability of one (5,1) block on BSC(0.1).

17.4 Programming challenges

P1. Write a function that returns the empirical entropy of any file (bytes) and test it on a text file, a ZIP file and an encrypted file.

P2. Implement canonical Huffman codes: from code lengths only, produce codewords, and decode.

P3. Add an adaptive model to the integer arithmetic coder (counts updated after every symbol).

P4. Implement LZW with a 4096-entry limit and dictionary reset; measure the ratio on a 100 kB text.

P5. Write a tool that estimates an order-1 Markov model of letters from a book and computes its entropy rate.

P6. Simulate the KL-based detector with window sizes 10–200 and plot FPR/FNR.

P7. Implement Blahut–Arimoto for a 3-input, 3-output channel and verify C for a symmetric example.

P8. Write a Hamming (15,11) encoder/decoder and test all single errors.

P9. Simulate a BSC with $p = 0.01..0.1$ and compare residual BER of (3,1), (5,1), Hamming (7,4) and BCH(15,7).

P10. Build a pipeline: Huffman-compress a text, protect with Hamming (7,4), send over BSC, decode, and report symbol errors (compare with Lab outputs of Chapter 18).

17.5 Answer keys and solutions

Multiple-choice key

M1 c	M2 b	M3 b	M4 b	M5 b	M6 a	M7 d	M8 a	M9 c	M10 b
M11 d	M12 b	M13 a	M14 b	M15 a	M16 a	M17 d	M18 a	M19 a	M20 b
M21 c	M22 b	M23 a	M24 b	M25 c	M26 d	M27 b	M28 a	M29 c	M30 d
M31 d	M32 c	M33 c	M34 d	M35 a	M36 a	M37 c	M38 d	M39 b	M40 a

Short-answer model answers

S1. Data is representation; information is reduction of uncertainty, measured by probabilities (Ch. 1).

S2. Continuity; monotonic in N for uniform sources; grouping/chain rule (Ch. 2).

S3. $-\log_2 1 = 0$: nothing new is learned.

S4. H averages; $H_\infty = -\log_2 \max p$ is the one-guess success. A 50% "123456" policy has $H = 21$ bits but $H_\infty = 1$ bit (Ch. 2).

S5. $H(X|Y = y)$ can exceed $H(X)$ for some y (a rare alarm makes the state uncertain); the average over y cannot (Ch. 3).

S6. Timing (key weight), power, packet size. Example: S = key, O = execution time, $p(o|s)$ from measurements (Ch. 3).

S7. Not symmetric, no triangle inequality, can be infinite (Ch. 4).

S8. Circles $H(X)$, $H(Y)$; overlap I; non-overlapping parts $H(X|Y)$, $H(Y|X)$; union $H(X,Y)$ (Ch. 4).

S9. Uniquely decodable: one parse exists; instantaneous: decided at the last bit of each codeword. Code $\{0, 01\}$ is UD but not instantaneous (Ch. 5).

S10. $L - H = D(p\|q) - \log_2 \sum 2^{-\ell_i} \geq 0$ by Gibbs and Kraft (Ch. 5).

S11. Its first balanced split is fixed and may put a very probable symbol at depth 2 (Ch. 6).

S12. More probable symbols are not longer; the two least probable are siblings at maximum depth (Ch. 7).

S13. Two symbols get 1 bit each. Code blocks of symbols or use arithmetic coding (Ch. 5, 7, 8).

S14. E1: lower half, output 0; E2: upper half, output 1; E3: middle half, remember a pending opposite bit (Ch. 8).

S15. Code equals next free entry; output $w + w[0]$; e.g. ABABABA $\to$ 0 1 2 4 (Ch. 9).

S16. Few repeated phrases; each code costs $\lceil \log_2(\text{dict size}) \rceil$ bits $>$ symbol information (Ch. 9).

S17. The decoder needs the table; for short data the header can exceed the gain (Ch. 10).

S18. Secret and attacker text compressed together: correct guesses shorten output. Separate contexts; mask secrets; pad lengths (Ch. 10).

S19. $\lim \frac{1}{n} H(X^n)$; memory makes the next symbol predictable (Ch. 11).

S20. $-\frac{1}{n} \log_2 P(X^n) \to H$; about 2^{nH} typical sequences need nH bits (Ch. 11).

S21. Use a high percentile (e.g. 99th) of scores on normal validation data (Ch. 12).

S22. Reliable communication possible iff $R < C$ (Ch. 13).

S23. Max secret rate without a key; $C_s = H_2(p_E) - H_2(p_B)$ (Ch. 13).

S24. Codes are public and linear: an attacker can create another valid codeword; use a keyed MAC (Ch. 14).

S25. Rates 0.2, 0.571, 0.467; corrections 2, 1, 2 per block (Ch. 14).

Numerical solutions

N1. $3 \times 0.25 \times 2 + 2 \times 0.125 \times 3 = 1.5 + 0.75 = 2.25$ bits.

N2. $0.05(4.3219) + 0.95(0.0740) = 0.2161 + 0.0703 = 0.2864$ bits.

N3. $\log_2 1000 = 9.966$ bits.

N4. $8 \log_2 62 = 47.63$ bits.

N5. $H(X,Y) = 1.5$; $p(y) = (0.75, 0.25)$, $H(Y) = 0.8113$; $H(X) = 1$, $H(Y|X) = 0.5$; $I = 1 + 0.8113 - 1.5 = 0.3113$ bits.

N6. $0.5 \log_2 \frac{0.5}{0.75} + 0.5 \log_2 \frac{0.5}{0.25} = -0.2925 + 0.5 = 0.2075$ bits.

N7. $3/4 + 2/8 = 1$: complete.

N8. Merges: 0.05+0.1, 0.15(C)+0.15, 0.2+0.3, 0.5+0.5. Codes A 0, B 10, C 110, E 1110, D 1111; $L = 0.5 + 0.4 + 0.45 + 0.4 + 0.2 = 1.95$ bits.

N9. A 0, B 10, C 110, D 1110, E 1111; $L = 1.95$ bits (same as Huffman here).

N10. $H = 1.9232$; $\eta = 1.9232/1.95 = 98.6\%$.

N11. B: $[0.6, 1)$; A: $[0.6,\ 0.6 + 0.4 \times 0.6) = [0.6, 0.84)$.

N12. A$\to$0 (AB=2), B$\to$1 (BB=3), B$\to$1 (BA=4), AB$\to$2 (ABB=5), BA$\to$4. Codes 0 1 1 2 4; bits $1 + 2 + 2 + 3 + 3 = 11$.

N13. $0.3\pi_0 = 0.6\pi_1 \Rightarrow \pi = (\frac{2}{3}, \frac{1}{3})$.

N14. $\frac{2}{3} H_2(0.3) + \frac{1}{3} H_2(0.4) = \frac{2}{3}(0.8813) + \frac{1}{3}(0.9710) = 0.9112$ bits.

N15. $1 - H_2(0.15) = 1 - 0.6098 = 0.3902$ bits/use.

N16. $5000 \times 0.9 = 4500$ bits.

N17. $p_1 = 1 \oplus 0 \oplus 1 = 0$, $p_2 = 1 \oplus 0 \oplus 1 = 0$, $p_4 = 0 \oplus 0 \oplus 1 = 1$: 0 0 1 1 0 0 1.

N18. $s_1 = 1 \oplus 1 \oplus 1 \oplus 0 = 1$, $s_2 = 0 \oplus 1 \oplus 1 \oplus 0 = 0$, $s_4 = 1 \oplus 1 \oplus 1 \oplus 0 = 1 \Rightarrow 5$; flip bit 5: 1011010, data 1010.

N19. $t = \lfloor 8/2 \rfloor = 4$.

N20. $\binom{5}{3}(0.1)^3(0.9)^2 + \binom{5}{4}(0.1)^4(0.9) + (0.1)^5 = 0.0081 + 0.00045 + 0.00001 = 0.00856$.

Programming challenges: solution notes

P1. Read bytes, `Counter`, Lab 1 `entropy`. Expect about 4.5 bits/byte for text and close to 8 for ZIP and encrypted files.

P2. Sort symbols by (length, symbol); first code 0; next code = (code + 1) $\ll (\ell_{\text{new}} - \ell_{\text{old}})$.

P3. Start all counts at 1; after coding a symbol, increase its count in *both* encoder and decoder; keep total below 2^{30}.

P4. Extend Lab 5: when `len(d)==4096`, reset to the initial dictionary (encoder and decoder at the same step).

P5. Lab 9 `estimate` on letter indices with smoothing; `entropy_rate`; expect about 3.3 bits/letter for English (order 1).

P6. Loop over W in Lab 11 `run(W)`; FNR falls as W grows.

P7. Lab 10 `blahut_arimoto`; a symmetric 3-ary channel with correct-prob $1 - p$ and $p/2$ to each other output has $C = \log_2 3 - H(1 - p, p/2, p/2)$.

P8. Parity bits at positions 1, 2, 4, 8; syndrome = position, as in Lab 7.

P9. Combine Labs 7, 8 and `channel.bsc`; plot on log axes as in Fig. 14.4.

P10. See Chapter 18 `project.py`, which does exactly this.

CHAPTER

18

Final Project: An Information Theory and Compression Laboratory



IN THIS CHAPTER YOU WILL LEARN

- an integrated tool that applies everything in the book;
- a clean separation of source coding and channel coding;
- project architecture and data flow;
- complete working code (`project.py`);
- real sample results with graphs;
- testing instructions and extensions.

18.1 Goals

The laboratory lets a student:

1. enter a text message (any source);
2. calculate symbol probabilities;
3. calculate the source entropy;
4. apply Shannon–Fano coding;
5. apply Huffman coding;
6. apply arithmetic coding;
7. apply LZW compression;
8. compare compression performance;
9. protect the data with Hamming coding;
10. demonstrate two-bit correction with repetition coding;
11. introduce random or controlled errors;
12. recover the data and measure residual errors;
13. generate comparison tables and graphs.

18.2 Source coding and channel coding: two separate layers

Information theory concept • Separation

Source coding removes redundancy to make data short (Part II). **Channel coding** adds structured redundancy to make data robust (Part IV). Shannon's separation theorem says that, for long blocks, doing them in two separate steps loses nothing. In a secure system the

order is: compress → encrypt → channel-encode → channel → channel-decode → decrypt → decompress.

Fig. 18.1 draws this order for a complete secure link.

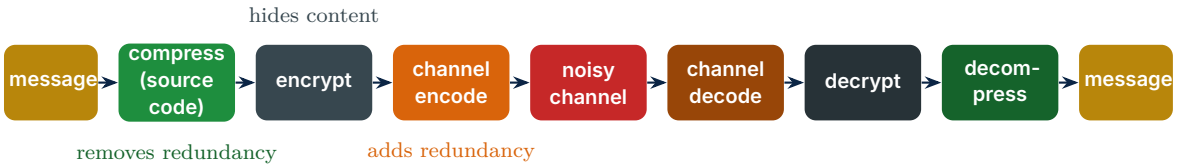


Figure 18.1. Block diagram. The order of the layers in a secure link. Compression must come before encryption, because encrypted data look random and cannot be compressed.

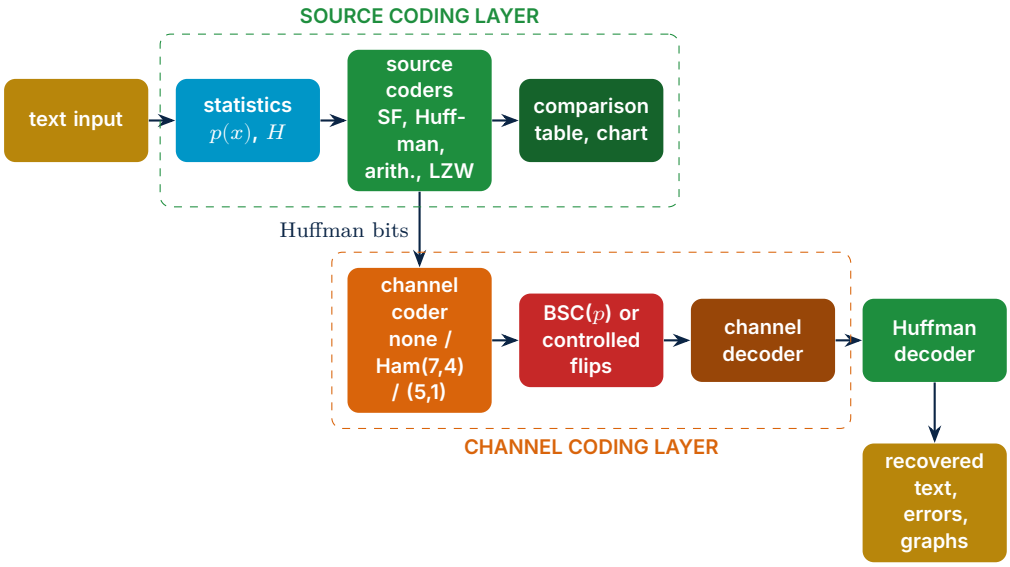


Figure 18.2. Architecture of the final project.

18.3 Project architecture

`entropy.py`	probabilities and entropy (steps 2–3)
`shannon_fano.py`, `huffman.py`, `arithmetic.py`, `lzw.py`	the four coders (steps 4–7)
`compare.py`	fair comparison with headers and round-trip checks (step 8)
`hamming.py`, `repetition.py`	channel codes (steps 9–10)
`channel.py`	BSC model (step 11)
`project.py`	the pipeline, controlled errors, recovery and plots (steps 11–13)
`test_itlab.py`	unit tests of every module

18.4 The program

project.py

```
"""project.py -- Final project: an Information Theory & Compression Laboratory.

Pipeline (source coding, then channel coding):
  text -> probabilities -> entropy -> {Shannon-Fano, Huffman, arithmetic, LZW}
       -> comparison table + chart
  Huffman bits -> channel code {none, Hamming(7,4), (5,1) repetition}
       -> binary symmetric channel (or controlled errors) -> channel decoder
       -> Huffman decoder -> recovered text
Run:  python project.py "YOUR TEXT"     (default text if none is given)
"""
import random, sys
from collections import Counter
import entropy as E, shannon_fano as sf, huffman as hf, arithmetic as ac, lzw
import hamming as hm, repetition as rp, channel as chn
from compare import compare


# ---------------------------------------------------------- channel coding layer
def protect(bits, scheme):
    if scheme == "none":
        return list(bits)
    if scheme == "hamming74":
        padded = list(bits) + [0] * (-len(bits) % 4)
        return [b for i in range(0, len(padded), 4) for b in hm.encode74(padded[i:i + 4])]
    if scheme == "rep5":
        return rp.encode(bits)
    raise ValueError(scheme)
```

```
28
29
30  def recover(received, scheme, n_bits):
31      if scheme == "none":
32          return list(received)[:n_bits]
33      if scheme == "hamming74":
34          out = []
35          for i in range(0, len(received), 7):
36              out += hm.decode74(received[i:i + 7])[0]
37          return out[:n_bits]
38      if scheme == "rep5":
39          return rp.decode(received)[0][:n_bits]
40
41
42  def safe_huffman_decode(bits, codes):
43      """Decode as far as possible (errors may leave a broken tail)."""
44      inverse = {c: s for s, c in codes.items()}
45      out, cur = [], ""
46      for b in bits:
47          cur += str(b)
48          if cur in inverse:
49              out.append(inverse[cur]); cur = ""
50      return "".join(out)
51
52
53  def char_errors(a, b):
54      return sum(x != y for x, y in zip(a, b)) + abs(len(a) - len(b))
55
56
57  def main(text, p=0.01, seed=2026):
58      print("=" * 70)
59      print("STEP 1-3  source statistics")
60      probs = E.probs_from_text(text)
61      H = E.entropy(probs)
62      print(f"  {len(text)} symbols, {len(probs)} distinct, H = {H:.4f} bits/symbol")
63      print("STEP 4-8  source coding and comparison")
64      r = compare(text)
65      print(f"  {'method':13s}{'payload':>9s}{'header':>8s}{'total':>8s}{'ratio':>7s}")
66      for k, (pay, hdr) in r["rows"].items():
67          print(f"  {k:13s}{pay:9d}{hdr:8d}{pay + hdr:8d}{r['raw'] / (pay + hdr):7.2f}")
68      print(f"  entropy bound n*H = {r['entropy_bits']:.0f} bits, raw = {r['raw']} bits")
69
70      print("STEP 9-12 channel coding over a BSC with p =", p)
71      codes = hf.huffman(probs)
72      src = [int(b) for b in hf.encode(text, codes)]
73      rng = random.Random(seed)
74      results = {}
75      for scheme in ("none", "hamming74", "rep5"):
76          tx = protect(src, scheme)
77          rx = [b ^ (rng.random() < p) for b in tx]
78          bits = recover(rx, scheme, len(src))
79          ber = sum(a != b for a, b in zip(bits, src)) / len(src)
80          back = safe_huffman_decode(bits, codes)
81          results[scheme] = (len(tx), ber, back == text, char_errors(back, text))
```

```
        print(f"  {scheme:10s} sent {len(tx):6d} bits, residual BER {ber:.4f}, "
              f"text recovered exactly: {back == text}, symbol errors {char_errors(back, text)}")

    print("STEP 11   controlled errors: two flips inside one protected block")
    blk = src[:4]
    h = hm.encode74(blk); h[0] ^= 1; h[3] ^= 1
    r5 = rp.encode(blk[:1]); r5[0] ^= 1; r5[3] ^= 1
    print("  Hamming(7,4) with 2 errors ->", hm.decode74(h)[0], "sent", blk,
          "(fails: corrects only 1)")
    print("  (5,1) repetition with 2 errors ->", rp.decode(r5)[0], "sent", blk[:1], "(corrected)")
    return r, results


def plots(text, out_prefix="project"):
    import matplotlib; matplotlib.use("Agg")
    import matplotlib.pyplot as plt
    r = compare(text)
    names = list(r["rows"])
    tot = [(r["rows"][k][0] + r["rows"][k][1]) / r["n"] for k in names]
    fig, ax = plt.subplots(figsize=(6, 3))
    ax.bar(names, tot, color=["#2a78d6", "#eb6834", "#1baf7a", "#6250d6"])
    ax.axhline(r["H"], color="k", ls="--", lw=1, label="entropy H")
    ax.set_ylabel("bits per symbol (incl. header)"); ax.legend()
    plt.savefig(out_prefix + "_compare.pdf", bbox_inches="tight"); plt.close()
    ps = [0.001, 0.002, 0.005, 0.01, 0.02, 0.05, 0.1]
    rng = random.Random(1)
    src = [rng.randint(0, 1) for _ in range(40000)]
    fig, ax = plt.subplots(figsize=(6, 3.2))
    for scheme, lab in [("none", "uncoded"), ("hamming74", "Hamming (7,4)"), ("rep5", "(5,1) repetition")]:
        ber = []
        for p in ps:
            tx = protect(src, scheme)
            rx = [b ^ (rng.random() < p) for b in tx]
            out = recover(rx, scheme, len(src))
            ber.append(max(sum(a != b for a, b in zip(out, src)) / len(src), 1e-6))
        ax.loglog(ps, ber, "o-", label=lab)
    ax.set_xlabel("BSC crossover probability p"); ax.set_ylabel("residual bit error rate")
    ax.legend(); ax.grid(True, which="both", lw=0.3)
    plt.savefig(out_prefix + "_ber.pdf", bbox_inches="tight"); plt.close()


DEFAULT = ("INFORMATION THEORY MEASURES SURPRISE. COMPRESSION REMOVES REDUNDANCY. "
           "ERROR CORRECTION ADDS REDUNDANCY BACK, IN A CONTROLLED WAY. ") * 4

if __name__ == "__main__":
    text = sys.argv[1] if len(sys.argv) > 1 else DEFAULT
    main(text)
```

How it works

- `protect` pads the bit stream to a multiple of 4 for Hamming (7,4) and applies the chosen code; `recover` decodes and removes padding.
- `safe_huffman_decode` decodes as much as possible even if errors leave a broken tail, so we can count symbol errors.
- `main` prints all steps; `plots` draws the comparison chart and the residual-BER curves.

18.5 Sample results

Running `python3 project.py` on the built-in 520-character text:

OUTPUT (real run)

```
======================================================================
STEP 1-3  source statistics
  520 symbols, 24 distinct, H = 4.0957 bits/symbol
STEP 4-8  source coding and comparison
  method         payload  header   total  ratio
  Shannon-Fano      2144     592    2736   1.52
  Huffman           2144     592    2736   1.52
  Arithmetic        2131     592    2723   1.53
  LZW               2276       0    2276   1.83
  entropy bound n*H = 2130 bits, raw = 4160 bits
STEP 9-12 channel coding over a BSC with p = 0.01
  none       sent   2144 bits, residual BER 0.0075, text recovered exactly: False, symbol errors 351
  hamming74  sent   3752 bits, residual BER 0.0000, text recovered exactly: True, symbol errors 0
  rep5       sent  10720 bits, residual BER 0.0000, text recovered exactly: True, symbol errors 0
STEP 11   controlled errors: two flips inside one protected block
  Hamming(7,4) with 2 errors -> [0, 1, 0, 0] sent [0, 0, 0, 0] (fails: corrects only 1)
  (5,1) repetition with 2 errors -> [0] sent [0] (corrected)
```

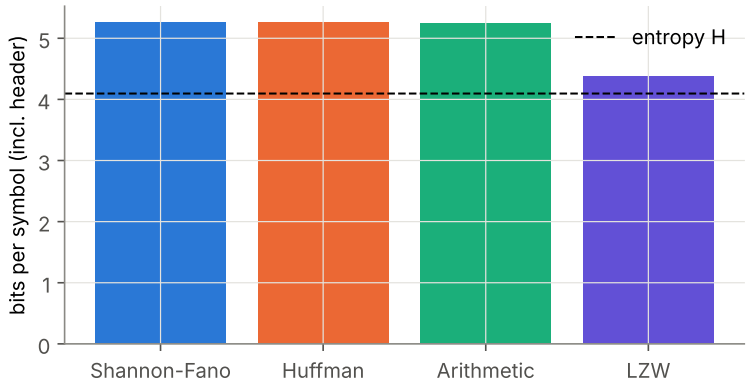


Figure 18.3. Bits per symbol including headers for the project text (dashed: entropy).

Worked Example 18.1 • Interpreting the results

1. **Statistics:** 24 distinct symbols, $H = 4.096$ bits/symbol: the entropy bound is 2130 bits for 520 symbols (raw ASCII: 4160).
2. **Compression:** arithmetic coding reaches the bound (2131 bits) but pays a 592-bit

header. LZW needs no header and wins here (2276 bits, ratio 1.83) because the text repeats itself four times — it exploits memory.

3. **Fragility of compressed data:** without channel coding, a BSC with $p = 0.01$ flips 0.75% of the Huffman bits, but the decoded text has 351 wrong symbols out of 520: a bit error makes the variable-length decoder lose synchronisation. Compressed data must be protected.
4. **Protection:** Hamming (7,4) (75% overhead) and (5,1) repetition (400% overhead) both delivered the text without error in this run.
5. **Controlled errors:** two flips inside one block break Hamming (7,4) but are corrected by the repetition code — the redundancy–reliability trade-off of Chapter 14.

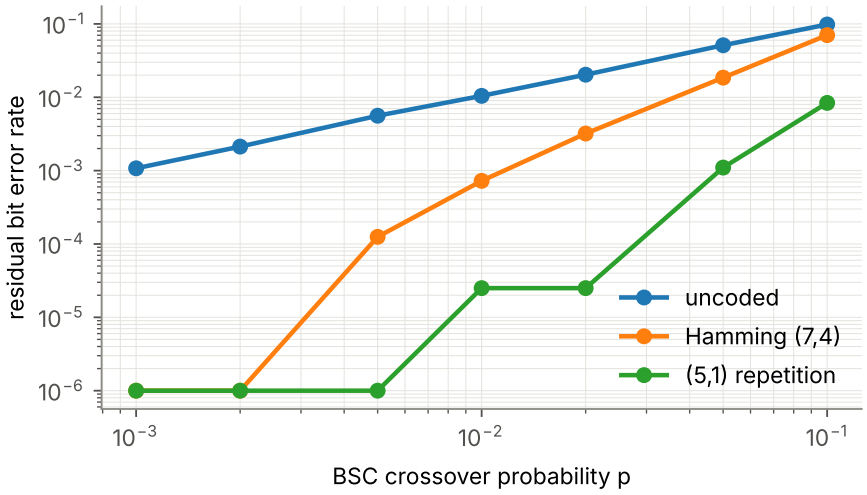


Figure 18.4. Residual BER versus BSC crossover probability (40 000 random data bits per point).

18.6 Testing instructions

1. Run `python3 test_itlab.py`: all 13 tests must pass (Chapter 15 shows the real output).
2. Run `python3 project.py "YOUR TEXT"` with: a single repeated character, a short word, a long English paragraph, and a random string. Check that every coder round-trips (the asserts in `compare.py` stop the program otherwise).
3. Change p in `main(text, p=...)` to 0.001, 0.05, 0.1 and record the residual BER of each scheme.
4. Use `plots(text)` to regenerate the figures.

18.7 Extensions

- Add encryption (e.g. a stream cipher from the `cryptography` package) between compression and channel coding, and measure that the ciphertext is incompressible.
- Replace Hamming (7,4) by BCH(15,7) (Lab 8) and compare at equal overhead.

- Add an interleaver to spread burst errors.
- Add a CRC or HMAC after decompression to detect residual errors (integrity).
- Build a small GUI (Tkinter) or a Jupyter notebook front end.

Appendix A — Mathematical Prerequisites

A.1 Logarithms

- $\log_b x = y \iff b^y = x$. Information uses $\log_2$ (bits); ln gives nats (1 nat = 1.4427 bits).
- $\log(xy) = \log x + \log y$; $\log(x/y) = \log x - \log y$; $\log x^a = a \log x$; $\log_2 x = \ln x / \ln 2 = \log_{10} x / 0.30103$.
- Useful values: $\log_2 3 = 1.5850$, $\log_2 5 = 2.3219$, $\log_2 6 = 2.5850$, $\log_2 7 = 2.8074$, $\log_2 10 = 3.3219$.
- $\ln z \le z - 1$ with equality only at $z = 1$ (used in all entropy inequalities).

A.2 Probability

- Axioms: $0 \le P(A) \le 1$, $P(\Omega) = 1$, $P(A \cup B) = P(A) + P(B)$ for disjoint events.
- Conditional probability $P(A|B) = P(A \cap B)/P(B)$; Bayes $P(A|B) = P(B|A)P(A)/P(B)$; total probability $P(B) = \sum_i P(B|A_i)P(A_i)$.
- Independence: $P(A \cap B) = P(A)P(B)$, i.e. $p(x, y) = p(x)p(y)$.
- Expectation $\mathbb{E}[g(X)] = \sum_x p(x)g(x)$; entropy is $\mathbb{E}[-\log_2 p(X)]$.
- Binomial: $P(k \text{ errors in } n) = \binom{n}{k} p^k (1-p)^{n-k}$, used for block error probabilities.
- Law of large numbers: averages of i.i.d. variables converge to the expectation (basis of the AEP).
- Jensen's inequality: for concave f, $\mathbb{E}\, f(X) \le f(\mathbb{E}\, X)$.

A.3 Binary arithmetic, XOR and GF(2)

$\oplus$	0	1
0	0	1
1	1	0

$\cdot$	0	1
0	0	0
1	0	1

Addition modulo 2 is XOR; multiplication is AND. $a \oplus a = 0$, so subtraction equals addition. A parity bit is the XOR of a set of bits. Binary fractions: $0.b_1 b_2 b_3 \cdots = \sum_i b_i 2^{-i}$, e.g. $0.011_2 = 0.375$ (arithmetic coding).

A.4 Matrices over GF(2)

$\mathbf{c} = \mathbf{dG}$ means: XOR the rows of $\mathbf{G}$ selected by the ones of $\mathbf{d}$. $\mathbf{Hc}^T = \mathbf{0}$ checks parity equations. Example: $(1,0,1,1)\mathbf{G}$ for the Hamming $\mathbf{G}$ of Chapter 14 = row 1 $\oplus$ row 3 $\oplus$ row 4 = $1110000 \oplus 0101010 \oplus 1101001 = 0110011$.

A.5 Polynomials and finite fields

Bit strings are polynomials: $10011 \leftrightarrow x^4 + x + 1$. Division is repeated XOR. GF(2^m) is built from a primitive polynomial of degree m; its non-zero elements are powers $\alpha^0, \ldots, \alpha^{2^m-2}$, multiplication adds exponents modulo $2^m - 1$, addition is XOR of the bit vectors (Chapter 14, BCH codes).

A.6 Inequalities used in the book

Gibbs	$D(p\|q) \geq 0$
Entropy bounds	$0 \leq H(X) \leq \log_2 \|\mathcal{X}\|$
Conditioning	$H(X\|Y) \leq H(X)$
Kraft–McMillan	$\sum 2^{-\ell_i} \leq 1$
Data processing	$I(X;Z) \leq I(X;Y)$ for $X \to Y \to Z$

Appendix B — MATLAB Implementations

B.1 Overview

This appendix gives a MATLAB version of every laboratory of Chapter 15 and of the final project. The algorithms, conventions and examples are the same as in the Python code, so all worked examples of the book can be reproduced in MATLAB. Copy all `.m` files (and the four `data_*.txt` files and `pyref_it.mat`) into one folder and run the lab scripts.

Functions	Purpose	Lab script
`entropy_bits, info_measures, kl_bits`	entropy, joint/conditional entropy, MI, KL	`lab1_entropy`
`shannon_fano_code, huffman_code, prefix_encode, prefix_decode`	prefix codes	`lab2_lab3_codes`
`arith_interval, arith_encode, arith_decode`	exact intervals; 32-bit integer coder	`lab4_arithmetic`
`lzw_encode, lzw_decode, lzw_bits`	LZW with dynamic widths	`lab5_lzw`
`compare_coders`	fair comparison with headers	`lab6_compare`
`hamming74_encode, hamming74_decode, secded84_decode`	Hamming and SECDED	`lab7_hamming`
`rep_decode, gf16_tables, bch1517_encode, bch1517_decode`	(5,1) repetition, BCH(15,7)	`lab8_multibit`
`markov_stationary, markov_entropy_rate, markov_estimate, markov_simulate`	Markov sources	`lab9_markov`
`channel_mi, blahut_arimoto`	capacity	`lab10_channel`
—	anomaly detection	`lab11_anomaly`
—	final project	`final_project_lab`
—	tests against Python	`test_itlab_matlab`

Requirements and testing

MATLAB R2016b or newer (implicit expansion, `containers.Map`, `repelem`); no toolbox is needed. All outputs printed here are real outputs from GNU Octave 8.4, which runs the same code. Deterministic results (codes, intervals, codewords, syndromes, capacities) are

identical to Python. Random experiments (Labs 9 and 11, the project's BSC) use MATLAB's own random generator, so their numbers differ slightly from the Python runs but agree statistically.

! MATLAB conventions

- Arrays start at 1: bit position j of the book's Hamming layout is `r(j)`, but BCH positions and LZW codes are 0-based numbers, as in the book; the functions handle the $+1$.
- Text is a `char` row vector; bit strings are `char` rows of `'0'` and `'1'`; bit vectors are `double` 0/1.
- The integer arithmetic coder uses doubles; keep the total frequency below 2^{16} so that all products stay exact ($< 2^{53}$).

B.2 Lab 1 — Entropy calculator

entropy_bits.m

```
function H = entropy_bits(p)
%ENTROPY_BITS  Shannon entropy H = -sum p log2 p (bits); zero probabilities skipped.
p = p(:);
if abs(sum(p) - 1) > 1e-9, error('probabilities must sum to 1'); end
p = p(p > 0);
H = -sum(p .* log2(p));
end
```

info_measures.m

```
function r = info_measures(P)
%INFO_MEASURES  Joint table P(x,y) (rows = X, columns = Y) -> struct with
%   HX, HY, HXY, HY_given_X, HX_given_Y, I (all in bits).
px = sum(P, 2);  py = sum(P, 1);
r.HX  = entropy_bits(px);
r.HY  = entropy_bits(py);
r.HXY = entropy_bits(P(:));
r.HY_given_X = r.HXY - r.HX;
r.HX_given_Y = r.HXY - r.HY;
r.I = r.HX + r.HY - r.HXY;
end
```

kl_bits.m

```
function D = kl_bits(p, q)
%KL_BITS  Kullback-Leibler divergence D(p||q) in bits (Inf if q=0 where p>0).
p = p(:); q = q(:);
k = p > 0;
if any(q(k) == 0), D = Inf; return; end
D = sum(p(k) .* log2(p(k) ./ q(k)));
end
```

lab1_entropy.m

```
%LAB1_ENTROPY  Entropy calculator (MATLAB version of entropy.py).
text = 'ABRACADABRA';
[sym, ~, idx] = unique(text);
p = accumarray(idx(:), 1)' / numel(text);
fprintf('symbols       : %s\n', sym);
fprintf('probabilities : %s\n', mat2str(round(p*1e4)/1e4));
fprintf('H(X)          : %.4f bits/symbol\n', entropy_bits(p));
fprintf('max possible  : %.4f bits/symbol\n', log2(numel(sym)));
fprintf('H2(0.1)       : %.4f bits\n', entropy_bits([0.1 0.9]));
r = info_measures([0.25 0.25; 0 0.5]);
fprintf('H(X,Y) = %.4f, H(Y|X) = %.4f, I(X;Y) = %.4f\n', r.HXY, r.HY_given_X, r.I);
fprintf('min-entropy of [0.6 0.2 0.1 0.1]: %.4f bits\n', -log2(0.6));
```

OUTPUT (real run)

```
symbols       : ABCDR
probabilities : [0.4545 0.1818 0.0909 0.0909 0.1818]
H(X)          : 2.0404 bits/symbol
max possible  : 2.3219 bits/symbol
H2(0.1)       : 0.4690 bits
H(X,Y) = 1.5000, H(Y|X) = 0.5000, I(X;Y) = 0.3113
min-entropy of [0.6 0.2 0.1 0.1]: 0.7370 bits
```

B.3 Labs 2 and 3 — Shannon–Fano and Huffman

shannon_fano_code.m

```
function codes = shannon_fano_code(sym, p)
%SHANNON_FANO_CODE  Shannon-Fano codes. sym: char array, p: probabilities.
%   Returns a cell array of codeword strings in the order of sym.
%   Sort by decreasing p (ties by symbol); split where |P_left - P_right|
%   is smallest (first such split); left part gets '0', right part '1'.
[~, order] = sortrows([-p(:), double(sym(:))]);
codes = repmat({''}, 1, numel(sym));
codes = split_group(order(:)', p, codes);
end

function codes = split_group(g, p, codes)
if numel(g) < 2, return; end
total = sum(p(g)); run = 0; best = inf; k = 1;
for j = 1:numel(g) - 1
    run = run + p(g(j));
    d = abs(2*run - total);
    if d < best - 1e-12, best = d; k = j; end
end
for j = 1:k,           codes{g(j)} = [codes{g(j)} '0']; end
for j = k+1:numel(g), codes{g(j)} = [codes{g(j)} '1']; end
codes = split_group(g(1:k), p, codes);
codes = split_group(g(k+1:end), p, codes);
end
```

huffman_code.m

```
function [codes, merges] = huffman_code(sym, p)
%HUFFMAN_CODE  Huffman codes with the book's conventions:
%   nodes are kept with an 'age'; the two smallest (ties: older first) are
%   merged, the first removed gets bit '0', the second bit '1'.
n = numel(sym);
codes = repmat({''}, 1, n);
if n == 1, codes{1} = '0'; merges = {}; return; end
[~, alpha] = sort(double(sym));                % leaves enter in symbol order
prob = p(alpha); members = num2cell(alpha); age = 1:n; nextage = n + 1;
merges = {};
while numel(prob) > 1
    [~, o] = sortrows([prob(:), age(:)]);         % smallest, then oldest
    a = o(1); b = o(2);
    for s = members{a}, codes{s} = ['0' codes{s}]; end
    for s = members{b}, codes{s} = ['1' codes{s}]; end
    merges(end+1, :) = {sym(members{a}), prob(a), sym(members{b}), prob(b)}; %#ok<AGROW>
    newp = prob(a) + prob(b); newm = [members{a} members{b}];
    keep = setdiff(1:numel(prob), [a b]);
    prob = [prob(keep) newp]; members = [members(keep) {newm}];
    age = [age(keep) nextage]; nextage = nextage + 1;
end
end
```

prefix_encode.m

```
function bits = prefix_encode(msg, sym, codes)
%PREFIX_ENCODE  Concatenate codewords of the symbols in msg.
bits = '';
for c = msg
    bits = [bits codes{sym == c}]; %#ok<AGROW>
end
end
```

prefix_decode.m

```
function msg = prefix_decode(bits, sym, codes)
%PREFIX_DECODE  Decode a prefix-free code by reading bits until a match.
msg = ''; cur = '';
for b = bits
    cur = [cur b]; %#ok<AGROW>
    k = find(strcmp(codes, cur), 1);
    if ~isempty(k)
        msg = [msg sym(k)]; %#ok<AGROW>
        cur = '';
    end
end
if ~isempty(cur), error('bitstream ended inside a codeword'); end
end
```

lab2_lab3_codes.m

```
%LAB2_LAB3_CODES  Shannon-Fano and Huffman (MATLAB versions of Labs 2 and 3).
sym = 'ABCDEF';  p = [0.30 0.25 0.15 0.12 0.10 0.08];
H = entropy_bits(p);
sf = shannon_fano_code(sym, p);
[hf, merges] = huffman_code(sym, p);
for k = 1:numel(sym)
    fprintf('%s  p = %.2f   Shannon-Fano %-4s  Huffman %-4s\n', sym(k), p(k), sf{k}, hf{k})
    ;
end
Lsf = sum(p .* cellfun(@numel, sf));  Lhf = sum(p .* cellfun(@numel, hf));
fprintf('H = %.4f  L(SF) = %.4f  L(Huffman) = %.4f  efficiency %.2f%%\n', H, Lsf, Lhf, 100*
    H/Lhf);
for k = 1:size(merges, 1)
    fprintf('merge %d: %s(%.2f) + %s(%.2f)\n', k, merges{k,1}, merges{k,2}, merges{k,3},
    merges{k,4});
end
msg = 'ABACADEF';
b = prefix_encode(msg, sym, hf);
fprintf('Huffman: %s -> %s -> %s\n', msg, b, prefix_decode(b, sym, hf));
```

OUTPUT (real run)

```
A  p = 0.30   Shannon-Fano 00    Huffman 11
B  p = 0.25   Shannon-Fano 01    Huffman 01
C  p = 0.15   Shannon-Fano 100   Huffman 101
D  p = 0.12   Shannon-Fano 101   Huffman 100
E  p = 0.10   Shannon-Fano 110   Huffman 001
F  p = 0.08   Shannon-Fano 111   Huffman 000
H = 2.4224  L(SF) = 2.4500  L(Huffman) = 2.4500  efficiency 98.87%
merge 1: F(0.08) + E(0.10)
merge 2: D(0.12) + C(0.15)
merge 3: FE(0.18) + B(0.25)
merge 4: DC(0.27) + A(0.30)
merge 5: FEB(0.43) + DCA(0.57)
Huffman: ABACADEF -> 11011110111100001000 -> ABACADEF
```

The merge table and the codewords are exactly those of Worked Example 7.1.

B.4 Lab 4 — Arithmetic coding

arith_interval.m

```
function [low, high, tag] = arith_interval(msg, sym, p)
%ARITH_INTERVAL  Teaching version: interval of a short message and the
%   shortest binary tag inside it (double precision: use for < 40 symbols).
[sym, o] = sort(sym); p = p(o);
Fh = cumsum(p); Fl = Fh - p;
low = 0; high = 1;
for c = msg
    k = find(sym == c, 1);
    w = high - low;
```

```
    high = low + w * Fh(k);
    low  = low + w * Fl(k);
    fprintf('  after %s: [%.6f, %.6f)  width %.6f\n', c, low, high, high - low);
end
k = 1;
while true
    m = ceil(low * 2^k - 1e-12);
    if (m + 1) / 2^k <= high + 1e-12
        tag = dec2bin(m, k); return
    end
    k = k + 1;
end
end
```

arith_encode.m

```
function bits = arith_encode(msg, sym, counts)
%ARITH_ENCODE  Integer arithmetic coder (32-bit, E1/E2/E3 scaling).
%   msg: char row; sym: char row of symbols (any order); counts: integer
%   frequencies (total <= 2^16 so that all products stay exact in doubles).
%   Symbols are placed in the cumulative table in sorted order.
[sym, o] = sort(sym); counts = counts(o);
cumh = cumsum(counts); cuml = cumh - counts; total = cumh(end);
FULL = 2^32 - 1; HALF = 2^31; QUART = 2^30;
low = 0; high = FULL; pending = 0; bits = blanks(0);
for c = msg
    k = find(sym == c, 1);
    rng = high - low + 1;
    high = low + floor(rng * cumh(k) / total) - 1;
    low  = low + floor(rng * cuml(k) / total);
    while true
        if high < HALF
            [bits, pending] = emit(bits, '0', pending);
        elseif low >= HALF
            [bits, pending] = emit(bits, '1', pending);
            low = low - HALF; high = high - HALF;
        elseif low >= QUART && high < 3*QUART
            pending = pending + 1; low = low - QUART; high = high - QUART;
        else
            break
        end
        low = 2*low; high = 2*high + 1;
    end
end
pending = pending + 1;
if low < QUART, [bits, ~] = emit(bits, '0', pending);
else,           [bits, ~] = emit(bits, '1', pending); end
end

function [bits, pending] = emit(bits, b, pending)
bits = [bits b repmat(char('0' + ('1' - b)), 1, pending)];
pending = 0;
end
```

arith_decode.m

```
function msg = arith_decode(bits, sym, counts, n)
%ARITH_DECODE  Decoder for ARITH_ENCODE; n = number of symbols to decode.
[sym, o] = sort(sym); counts = counts(o);
cumh = cumsum(counts); cuml = cumh - counts; total = cumh(end);
FULL = 2^32 - 1; HALF = 2^31; QUART = 2^30;
stream = bits - '0'; pos = 0;
low = 0; high = FULL; code = 0;
for i = 1:32, [b, pos] = nextbit(stream, pos); code = 2*code + b; end
msg = blanks(n);
for i = 1:n
    rng = high - low + 1;
    target = floor(((code - low + 1) * total - 1) / rng);
    k = find(target >= cuml & target < cumh, 1);
    msg(i) = sym(k);
    high = low + floor(rng * cumh(k) / total) - 1;
    low  = low + floor(rng * cuml(k) / total);
    while true
        if high < HALF
            % nothing to subtract
        elseif low >= HALF
            low = low - HALF; high = high - HALF; code = code - HALF;
        elseif low >= QUART && high < 3*QUART
            low = low - QUART; high = high - QUART; code = code - QUART;
        else
            break
        end
        low = 2*low; high = 2*high + 1;
        [b, pos] = nextbit(stream, pos); code = 2*code + b;
    end
end
end

function [b, pos] = nextbit(stream, pos)
pos = pos + 1;
if pos <= numel(stream), b = stream(pos); else, b = 0; end
end
```

lab4_arithmetic.m

```
%LAB4_ARITHMETIC  Arithmetic coding (MATLAB version of arithmetic.py).
disp('Exact example: p(A)=1/2, p(B)=1/4, p(C)=1/4, message ABAC');
[lo, hi, tag] = arith_interval('ABAC', 'ABC', [0.5 0.25 0.25]);
fprintf('  tag bits: %s\n', tag);
text = 'TOBEORNOTTOBEORTOBEORNOT';
[sym, ~, idx] = unique(text); counts = accumarray(idx(:), 1)';
b = arith_encode(text, sym, counts);
fprintf('integer coder: %d bits; round trip OK: %d\n', numel(b), ...
        strcmp(arith_decode(b, sym, counts, numel(text)), text));
```

OUTPUT (real run)

```
Exact example: p(A)=1/2, p(B)=1/4, p(C)=1/4, message ABAC
  after A: [0.000000, 0.500000)  width 0.500000
  after B: [0.250000, 0.375000)  width 0.125000
  after A: [0.250000, 0.312500)  width 0.062500
  after C: [0.296875, 0.312500)  width 0.015625
  tag bits: 010011
integer coder: 60 bits; round trip OK: 1
```

B.5 Lab 5 — LZW

lzw_encode.m

```
function codes = lzw_encode(text, alphabet)
%LZW_ENCODE  LZW codes. alphabet = '' uses the 256 byte values (code = ASCII).
if nargin < 2 || isempty(alphabet), alphabet = char(0:255); end
d = containers.Map('KeyType', 'char', 'ValueType', 'double');
for i = 1:numel(alphabet), d(alphabet(i)) = i - 1; end
w = ''; codes = [];
for c = text
    if isKey(d, [w c])
        w = [w c];
    else
        codes(end+1) = d(w); %#ok<AGROW>
        d([w c]) = d.Count;      % next free code
        w = c;
    end
end
if ~isempty(w), codes(end+1) = d(w); end
end
```

lzw_decode.m

```
function text = lzw_decode(codes, alphabet)
%LZW_DECODE  Rebuilds the dictionary while decoding; handles the cScSc case.
if nargin < 2 || isempty(alphabet), alphabet = char(0:255); end
d = cell(1, numel(alphabet) + numel(codes));
for i = 1:numel(alphabet), d{i} = alphabet(i); end
n = numel(alphabet);                          % number of entries
w = d{codes(1) + 1}; text = w;
for k = codes(2:end)
    if k < n
        entry = d{k + 1};
    elseif k == n
        entry = [w w(1)];                     % special case
    else
        error('bad code %d', k);
    end
    n = n + 1; d{n} = [w entry(1)];
    text = [text entry]; %#ok<AGROW>
    w = entry;
```

```
end
end
```

lzw_bits.m

```
function total = lzw_bits(codes, n0)
%LZW_BITS  Bits with dynamic widths: code k (0-based) uses ceil(log2(n0+k)).
k = 0:numel(codes) - 1;
total = sum(max(1, ceil(log2(n0 + k))));
end
```

lab5_lzw.m

```
%LAB5_LZW  LZW compressor and decompressor (MATLAB version of lzw.py).
c = lzw_encode('ABABABA', 'AB');
fprintf('ABABABA -> %s -> %s, %d bits\n', mat2str(c), lzw_decode(c, 'AB'), lzw_bits(c, 2));
t = 'TOBEORNOTTOBEORTOBEORNOT';
c = lzw_encode(t, '');
fprintf('%s\n  codes: %s\n  round trip OK: %d, %d bits (original %d)\n', t, mat2str(c), ...
        strcmp(lzw_decode(c, ''), t), lzw_bits(c, 256), 8*numel(t));
```

OUTPUT (real run)

```
ABABABA -> [0 1 2 4] -> ABABABA, 8 bits
TOBEORNOTTOBEORTOBEORNOT
  codes: [84 79 66 69 79 82 78 79 84 256 258 260 265 259 261 263]
  round trip OK: 1, 143 bits (original 192)
```

B.6 Lab 6 — Compression comparison

compare_coders.m

```
function R = compare_coders(text)
%COMPARE_CODERS  Payload and header bits of the four coders, with
%   round-trip checks (header: 16 bits + 24 bits per symbol, as in Python).
[sym, ~, idx] = unique(text); counts = accumarray(idx(:), 1)'; p = counts / numel(text);
hdr = 16 + numel(sym) * 24;
sf = shannon_fano_code(sym, p);  b = prefix_encode(text, sym, sf);
assert(strcmp(prefix_decode(b, sym, sf), text)); R.sf = [numel(b) hdr];
hf = huffman_code(sym, p);       b = prefix_encode(text, sym, hf);
assert(strcmp(prefix_decode(b, sym, hf), text)); R.huffman = [numel(b) hdr];
b = arith_encode(text, sym, counts);
assert(strcmp(arith_decode(b, sym, counts, numel(text)), text)); R.arith = [numel(b) hdr];
c = lzw_encode(text, '');
assert(strcmp(lzw_decode(c, ''), text)); R.lzw = [lzw_bits(c, 256) 0];
R.H = entropy_bits(p); R.n = numel(text);
end
```

lab6_compare.m

```
%LAB6_COMPARE  Fair comparison on the four data sets of Chapter 10
%   (the text files were written by compare.py so both languages use the same data).
names = {'skewed', 'english', 'repetitive', 'uniform'};
for k = 1:numel(names)
    fid = fopen(['data_' names{k} '.txt']); text = fgetl(fid); fclose(fid);
    R = compare_coders(text);
    fprintf('%-10s n = %4d  H = %.3f  nH = %4.0f |', names{k}, R.n, R.H, R.n * R.H);
    fprintf(' SF %d+%d | Huffman %d+%d | Arith %d+%d | LZW %d\n', R.sf, R.huffman, R.arith,
     R.lzw(1));
end
```

OUTPUT (real run)

```
skewed     n = 1000  H = 1.641  nH = 1641 | SF 1680+136 | Huffman 1680+136 | Arith 1643+136 | LZW 2642
english    n = 1020  H = 4.122  nH = 4204 | SF 4266+640 | Huffman 4240+640 | Arith 4205+640 | LZW 4832
repetitive n = 1080  H = 2.419  nH = 2613 | SF 2640+160 | Huffman 2640+160 | Arith 2614+160 | LZW 1214
uniform    n = 1000  H = 3.994  nH = 3994 | SF 4033+400 | Huffman 4000+400 | Arith 3995+400 | LZW 5752
```

Every number is identical to the Python results of Chapter 10.

B.7 Lab 7 — Hamming and SECDED

hamming74_encode.m

```
function c = hamming74_encode(d)
%HAMMING74_ENCODE  d = [d1 d2 d3 d4] -> [p1 p2 d1 p4 d2 d3 d4] (positions 1..7).
p1 = xor(xor(d(1), d(2)), d(4));
p2 = xor(xor(d(1), d(3)), d(4));
p4 = xor(xor(d(2), d(3)), d(4));
c = double([p1 p2 d(1) p4 d(2) d(3) d(4)]);
end
```

hamming74_decode.m

```
function [d, r, s] = hamming74_decode(r)
%HAMMING74_DECODE  Single-error correction. s = syndrome = error position.
H = [1 0 1 0 1 0 1; 0 1 1 0 0 1 1; 0 0 0 1 1 1 1];   % rows s1, s2, s4
sv = mod(H * r(:), 2);
s = sv(1) + 2*sv(2) + 4*sv(3);
if s > 0, r(s) = 1 - r(s); end
d = r([3 5 6 7]);
end
```

secded84_decode.m

```
function [d, status] = secded84_decode(r)
%SECDED84_DECODE  Extended Hamming (8,4): r = [7-bit Hamming word, p0].
[~, ~, s] = hamming74_decode(r(1:7));
overall = mod(sum(r), 2);
if s == 0 && overall == 0
```

```
    d = r([3 5 6 7]); status = 'no error';
elseif overall == 1 && s == 0
    d = r([3 5 6 7]); status = 'parity bit error';
elseif overall == 1
    d = hamming74_decode(r(1:7)); status = 'corrected single error';
else
    d = []; status = 'double error detected';
end
end
```

lab7_hamming.m

```
%LAB7_HAMMING  Hamming (7,4) and SECDED (MATLAB version of hamming.py).
d = [1 0 1 1];
c = hamming74_encode(d);
fprintf('data %s -> codeword %s\n', mat2str(d), mat2str(c));
r = c; r(5) = 1 - r(5);                          % error at position 5
[dd, fixed, s] = hamming74_decode(r);
fprintf('received %s syndrome %d -> corrected %s data %s\n', mat2str(r), s, mat2str(fixed),
     mat2str(dd));
r2 = c; r2([2 6]) = 1 - r2([2 6]);               % two errors
[dd2, ~, s2] = hamming74_decode(r2);
fprintf('two errors -> syndrome %d data %s (WRONG)\n', s2, mat2str(dd2));
c8 = [c mod(sum(c), 2)];  r8 = c8; r8([2 6]) = 1 - r8([2 6]);
[~, st] = secded84_decode(r8);
fprintf('SECDED with the same two errors -> %s\n', st);
```

OUTPUT (real run)

```
data [1 0 1 1] -> codeword [0 1 1 0 0 1 1]
received [0 1 1 0 1 1 1] syndrome 5 -> corrected [0 1 1 0 0 1 1] data [1 0 1 1]
two errors -> syndrome 4 data [1 0 0 1] (WRONG)
SECDED with the same two errors -> double error detected
```

B.8 Lab 8 — Repetition and BCH(15,7)

rep_decode.m

```
function [bits, votes] = rep_decode(r, n)
%REP_DECODE  Majority vote on blocks of n bits (n odd, default 5).
if nargin < 2, n = 5; end
blocks = reshape(r, n, []);
votes = sum(blocks, 1);
bits = double(votes > floor(n/2));
end
```

gf16_tables.m

```
function [EXP, LOG] = gf16_tables()
%GF16_TABLES  EXP(i+1) = alpha^i (i = 0..29) as integers 0..15, from x^4+x+1.
%   LOG(v+1) = i with alpha^i = v (v = 1..15).
```

```
EXP = zeros(1, 30); LOG = zeros(1, 16); v = 1;
for i = 0:14
    EXP(i+1) = v; LOG(v+1) = i;
    v = v * 2;
    if v >= 16, v = bitxor(v, 19); end          % 19 = 10011b
end
EXP(16:30) = EXP(1:15);
end
```

bch1517_encode.m

```
function c = bch1517_encode(m)
%BCH1517_ENCODE  Systematic BCH(15,7): c = [parity(8) message(7)],
%   coefficient order x^0..x^14, g(x) = 1 + x^4 + x^6 + x^7 + x^8.
g = [1 0 0 0 1 0 1 1 1];
r = [zeros(1, 8) m(:)'];                        % x^8 m(x)
for i = 15:-1:9                                 % long division
    if r(i)
        r(i-8:i) = xor(r(i-8:i), g);
    end
end
c = [double(r(1:8)) m(:)'];
end
```

bch1517_decode.m

```
function [m, c, pos] = bch1517_decode(r)
%BCH1517_DECODE  Peterson decoder, corrects up to 2 errors.
%   pos = 0-based error positions found ([] if none, NaN if failure).
[EXP, LOG] = gf16_tables();
mul = @(a, b) (a ~= 0 && b ~= 0) * EXP(mod(LOG(a+1) + LOG(b+1), 15) + 1);
dv  = @(a, b) (a ~= 0) * EXP(mod(LOG(a+1) - LOG(b+1), 15) + 1);
pw  = @(a, k) (a ~= 0) * EXP(mod(LOG(a+1) * k, 15) + 1);
s1 = 0; s3 = 0;
for i = 0:14
    if r(i+1)
        s1 = bitxor(s1, EXP(mod(i, 15) + 1));
        s3 = bitxor(s3, EXP(mod(3*i, 15) + 1));
    end
end
c = r; pos = [];
if s1 == 0 && s3 == 0
    m = r(9:15); return
end
if s1 ~= 0 && s3 == pw(s1, 3)
    pos = LOG(s1 + 1);
elseif s1 == 0
    m = r(9:15); pos = NaN; return
else
    sigma2 = dv(bitxor(s3, pw(s1, 3)), s1);
    for j = 0:14                                % Chien search, x = alpha^-j
        x = EXP(mod(15 - j, 15) + 1);
        if bitxor(bitxor(1, mul(s1, x)), mul(sigma2, mul(x, x))) == 0
            pos(end+1) = j; %#ok<AGROW>
```

```
        end
    end
    if numel(pos) ~= 2, m = r(9:15); pos = NaN; return; end
end
c(pos + 1) = 1 - c(pos + 1);
m = c(9:15);
end
```

lab8_multibit.m

```
%LAB8_MULTIBIT  (5,1) repetition and BCH(15,7) (MATLAB version of Lab 8).
msg = [1 0 1];
c = repelem(msg, 5);
r = c; r([2 4 7 10]) = 1 - r([2 4 7 10]);       % 2 errors in blocks 1 and 2
[dec, votes] = rep_decode(r);
fprintf('repetition: votes %s -> decoded %s\n', mat2str(votes), mat2str(dec));
r3 = c; r3(1:3) = 1 - r3(1:3);
fprintf('3 errors in one block -> %s (failure expected)\n', mat2str(rep_decode(r3)));
m = [1 0 1 1 0 0 1];
cw = bch1517_encode(m);
fprintf('BCH codeword %s\n', mat2str(cw));
rb = cw; rb([3 12]) = 1 - rb([3 12]);           % errors at 0-based positions 2 and 11
[mh, ~, pos] = bch1517_decode(rb);
fprintf('errors found at positions %s, message OK: %d\n', mat2str(pos), isequal(mh, m));
```

OUTPUT (real run)

```
repetition: votes [3 2 5] -> decoded [1 0 1]
3 errors in one block -> [0 0 1] (failure expected)
BCH codeword [0 1 0 0 0 0 1 1 1 0 1 1 0 0 1]
errors found at positions [2 11], message OK: 1
```

B.9 Lab 9 — Markov chains

markov_stationary.m

```
function pi_ = markov_stationary(P)
%MARKOV_STATIONARY  Solve pi*P = pi, sum(pi) = 1.
S = size(P, 1);
A = [P' - eye(S); ones(1, S)];
pi_ = (A \ [zeros(S, 1); 1])';
end
```

markov_entropy_rate.m

```
function H = markov_entropy_rate(P)
%MARKOV_ENTROPY_RATE  H = sum_i pi_i * H(row i), in bits/symbol.
pi_ = markov_stationary(P);
Hr = zeros(1, size(P, 1));
for i = 1:size(P, 1)
    row = P(i, P(i, :) > 0);
```

```
    Hr(i) = -sum(row .* log2(row));
end
H = sum(pi_ .* Hr);
end
```

markov_estimate.m

```
function P = markov_estimate(seq, S, alpha)
%MARKOV_ESTIMATE  ML transition matrix from a state sequence (states 1..S),
%   with optional add-alpha smoothing.
if nargin < 3, alpha = 0; end
N = alpha * ones(S);
for k = 2:numel(seq)
    N(seq(k-1), seq(k)) = N(seq(k-1), seq(k)) + 1;
end
P = N ./ sum(N, 2);
end
```

markov_simulate.m

```
function seq = markov_simulate(P, n, start)
%MARKOV_SIMULATE  n states of a chain (states 1..S); seed rng before calling.
seq = zeros(1, n); seq(1) = start; C = cumsum(P, 2);
for k = 2:n
    seq(k) = find(rand < C(seq(k-1), :), 1);
end
end
```

lab9_markov.m

```
%LAB9_MARKOV  Markov chains (MATLAB version of markov.py).
P = [0.9 0.1; 0.5 0.5];
pi_ = markov_stationary(P);
fprintf('stationary distribution: %s\n', mat2str(round(pi_*1e4)/1e4));
fprintf('entropy rate: %.4f bits/symbol\n', markov_entropy_rate(P));
fprintf('entropy of pi (memoryless view): %.4f\n', entropy_bits(pi_));
rng(1); seq = markov_simulate(P, 20000, 1);
fprintf('estimated P from 20000 samples: %s\n', mat2str(round(markov_estimate(seq, 2)*1e3)
    /1e3));
```

OUTPUT (real run)

```
stationary distribution: [0.8333 0.1667]
entropy rate: 0.5575 bits/symbol
entropy of pi (memoryless view): 0.6500
estimated P from 20000 samples: [0.898 0.102;0.495 0.505]
```

B.10 Lab 10 — Channel capacity

channel_mi.m

```
function I = channel_mi(px, W)
%CHANNEL_MI  I(X;Y) for input px (row) and channel W(x,y) = P(y|x).
py = px * W;
I = 0;
for x = 1:numel(px)
    for y = 1:size(W, 2)
        if px(x) > 0 && W(x, y) > 0
            I = I + px(x) * W(x, y) * log2(W(x, y) / py(y));
        end
    end
end
end
```

blahut_arimoto.m

```
function [C, px] = blahut_arimoto(W, iters)
%BLAHUT_ARIMOTO  Capacity of a DMC W(x,y) = P(y|x) and the best input.
if nargin < 2, iters = 2000; end
nx = size(W, 1); px = ones(1, nx) / nx;
for it = 1:iters
    py = px * W;
    D = zeros(1, nx);
    for x = 1:nx
        k = W(x, :) > 0;
        D(x) = sum(W(x, k) .* log2(W(x, k) ./ py(k)));
    end
    new = px .* 2.^D; new = new / sum(new);
    if max(abs(new - px)) < 1e-12, px = new; break; end
    px = new;
end
C = channel_mi(px, W);
end
```

lab10_channel.m

```
%LAB10_CHANNEL  Channel capacity (MATLAB version of channel.py).
h2 = @(p) -p.*log2(max(p, eps)) - (1-p).*log2(max(1-p, eps));
for p = [0 0.01 0.11 0.5]
    fprintf('BSC p = %.2f  C = %.4f bits/use\n', p, 1 - h2(p));
end
W = [0.9 0.1; 0.2 0.8];
[C, px] = blahut_arimoto(W);
fprintf('channel [0.9 0.1; 0.2 0.8]: C = %.4f, best P(X) = %s\n', C, mat2str(round(px*1e4)
    /1e4));
[C, px] = blahut_arimoto([1 0; 0.5 0.5]);
fprintf('Z-channel q = 0.5: C = %.4f, best P(X) = %s\n', C, mat2str(round(px*1e4)/1e4));
```

OUTPUT (real run)

```
BSC p = 0.00  C = 1.0000 bits/use
BSC p = 0.01  C = 0.9192 bits/use
BSC p = 0.11  C = 0.5001 bits/use
BSC p = 0.50  C = 0.0000 bits/use
channel [0.9 0.1; 0.2 0.8]: C = 0.3978, best P(X) = [0.5176 0.4824]
Z-channel q = 0.5: C = 0.3219, best P(X) = [0.6 0.4]
```

B.11 Lab 11 — Anomaly detection

lab11_anomaly.m

```
%LAB11_ANOMALY  KL- and log-loss-based anomaly detection (MATLAB version).
%   Actions 1 = login, 2 = read, 3 = write, 4 = logout.
Pn = [0.05 0.80 0.10 0.05; 0.02 0.60 0.25 0.13; 0.02 0.50 0.30 0.18; 0.90 0.04 0.03 0.03];
Pa = [0.60 0.10 0.05 0.25; 0.05 0.30 0.55 0.10; 0.05 0.20 0.65 0.10; 0.85 0.05 0.05 0.05];
W = 50; rng(7);
train = markov_simulate(Pn, 20000, 1);
Ph = markov_estimate(train, 4, 1);              % smoothed ML baseline
q = markov_stationary(Ph);
freqw = @(w) (accumarray(w(:), 1, [4 1])' + 0.5) / (numel(w) + 2);
klsc  = @(w) kl_bits(freqw(w), q);
llsc  = @(w) -mean(log2(Ph(sub2ind([4 4], w(1:end-1), w(2:end)))));
win = @(s) reshape(s(1:floor(numel(s)/W)*W), W, [])';
valid = win(markov_simulate(Pn, 200*W, 1));
tn = win(markov_simulate(Pn, 200*W, 1));
ta = win(markov_simulate(Pa, 200*W, 1));
scores = {klsc, llsc}; names = {'KL', 'log-loss'};
fprintf('baseline action frequencies: %s\n', mat2str(round(q*1e3)/1e3));
for s = 1:2
    f = scores{s};
    v = sort(arrayfun(@(i) f(valid(i, :)), 1:size(valid, 1)));
    thr = v(min(numel(v), floor(0.99*numel(v)) + 1));
    fp = mean(arrayfun(@(i) f(tn(i, :)), 1:size(tn, 1)) > thr);
    fn = mean(arrayfun(@(i) f(ta(i, :)), 1:size(ta, 1)) <= thr);
    fprintf('%-8s threshold %.3f  false positive rate %.3f  false negative rate %.3f\n',
    names{s}, thr, fp, fn);
end
```

OUTPUT (real run)

```
baseline action frequencies: [0.128 0.541 0.212 0.118]
KL       threshold 0.136  false positive rate 0.030  false negative rate 0.000
log-loss threshold 1.665  false positive rate 0.020  false negative rate 0.000
```

With MATLAB's random stream the 99th-percentile thresholds are a little lower than in the Python run, so 2–3% of normal test windows raise an alarm and no attack window is missed. This illustrates how thresholds chosen on 200 validation windows vary from run to run; use more validation data for stable thresholds.

B.12 Final project

final_project_lab.m

```
%FINAL_PROJECT_LAB  Information Theory and Compression Laboratory (MATLAB
%   version of project.py): statistics, four compressors, channel coding,
%   BSC errors, recovery.
text = repmat(['INFORMATION THEORY MEASURES SURPRISE. COMPRESSION REMOVES REDUNDANCY. ' ...
               'ERROR CORRECTION ADDS REDUNDANCY BACK, IN A CONTROLLED WAY. '], 1, 4);
[sym, ~, idx] = unique(text); counts = accumarray(idx(:), 1)'; p = counts / numel(text);
fprintf('STEP 1-3  %d symbols, %d distinct, H = %.4f bits/symbol\n', numel(text), numel(sym
    ), entropy_bits(p));
R = compare_coders(text);
fprintf('STEP 4-8  SF %d+%d, Huffman %d+%d, arithmetic %d+%d, LZW %d (raw %d)\n', ...
        R.sf, R.huffman, R.arith, R.lzw(1), 8*numel(text));
hf = huffman_code(sym, p);
src = prefix_encode(text, sym, hf) - '0';
pBSC = 0.01; rng(2026);
schemes = {'none', 'hamming74', 'rep5'};
for s = 1:3
    switch schemes{s}
        case 'none',      tx = src;
        case 'hamming74', pad = [src zeros(1, mod(-numel(src), 4))];
                          tx = cell2mat(arrayfun(@(i) hamming74_encode(pad(i:i+3)), 1:4:
    numel(pad), 'UniformOutput', false));
        case 'rep5',      tx = repelem(src, 5);
    end
    rx = double(xor(tx, rand(size(tx)) < pBSC));
    switch schemes{s}
        case 'none',      out = rx;
        case 'hamming74', out = cell2mat(arrayfun(@(i) hamming74_decode(rx(i:i+6)), 1:7:
    numel(rx), 'UniformOutput', false));
        case 'rep5',      out = rep_decode(rx);
    end
    out = out(1:numel(src));
    ber = mean(out ~= src);
    try
        back = prefix_decode(char(out + '0'), sym, hf);
    catch
        back = '';                                % broken tail after errors
    end
    fprintf('STEP 9-12 %-9s sent %5d bits, residual BER %.4f, text recovered exactly: %d\n'
    , ...
            schemes{s}, numel(tx), ber, strcmp(back, text));
end
```

OUTPUT (real run)

```
STEP 1-3  520 symbols, 24 distinct, H = 4.0957 bits/symbol
STEP 4-8  SF 2144+592, Huffman 2144+592, arithmetic 2131+592, LZW 2276 (raw 4160)
STEP 9-12 none      sent  2144 bits, residual BER 0.0075, text recovered exactly: 0
STEP 9-12 hamming74 sent  3752 bits, residual BER 0.0000, text recovered exactly: 1
STEP 9-12 rep5      sent 10720 bits, residual BER 0.0000, text recovered exactly: 1
```

B.13 Tests against the Python library

test_itlab_matlab.m

```
%TEST_ITLAB_MATLAB  Tests of the MATLAB labs, including comparison with
%   reference results of the Python library (pyref_it.mat).
R = load('pyref_it.mat');
syms = 'ABCDEFGHIJ';
cellrow = @(x) strsplit(strtrim(x), ' ');
okSF = 0; okHF = 0;
for t = 1:50
    k = R.K(t); p = R.P(t, 1:k);
    sf = shannon_fano_code(syms(1:k), p);  hf = huffman_code(syms(1:k), p);
    okSF = okSF + isequal(sf, cellrow(R.SF{t}));
    okHF = okHF + isequal(hf, cellrow(R.HF{t}));
end
fprintf('Shannon-Fano codes identical to Python: %d / 50\n', okSF);
fprintf('Huffman codes identical to Python     : %d / 50\n', okHF);
okA = 0; okL = 0; okRT = 0;
for t = 1:30
    txt = R.T{t};
    [sym, ~, idx] = unique(txt); counts = accumarray(idx(:), 1)';
    b = arith_encode(txt, sym, counts);
    okA = okA + strcmp(b, R.AB{t});
    okRT = okRT + strcmp(arith_decode(b, sym, counts, numel(txt)), txt);
    c = lzw_encode(txt, 'ABCDE');
    okL = okL + isequal(c, str2num(R.LZ{t})) * strcmp(lzw_decode(c, 'ABCDE'), txt); %#ok<
    ST2NM>
end
fprintf('arithmetic bit streams identical      : %d / 30 (round trips %d / 30)\n', okA,
    okRT);
fprintf('LZW codes identical and round trip    : %d / 30\n', okL);
okB = 0; okE = 0;
for t = 1:20
    c = bch1517_encode(R.M(t, :));
    okB = okB + isequal(c, R.C(t, :));
    for i = 0:14
        for j = i:14
            r = c; r(i+1) = 1 - r(i+1);
            if j > i, r(j+1) = 1 - r(j+1); end
            [~, ch] = bch1517_decode(r);
            okE = okE + isequal(ch, c);
        end
    end
end
fprintf('BCH codewords identical               : %d / 20\n', okB);
fprintf('BCH 1- and 2-error patterns corrected : %d / %d\n', okE, 20 * 120);
okH = 0;
for v = 0:15
    d = bitget(v, 4:-1:1); c = hamming74_encode(d);
    for e = 1:7
        r = c; r(e) = 1 - r(e);
        okH = okH + isequal(hamming74_decode(r), d);
    end
```

```
end
fprintf('Hamming single errors corrected       : %d / 112\n', okH);
okR = 0;
for v = 0:31
    e = bitget(v, 1:5);
    okR = okR + (isequal(rep_decode(e), 0) == (sum(e) <= 2));
end
fprintf('(5,1) patterns behaving as predicted  : %d / 32\n', okR);
```

OUTPUT (real run)

```
Shannon-Fano codes identical to Python: 50 / 50
Huffman codes identical to Python     : 50 / 50
arithmetic bit streams identical      : 30 / 30 (round trips 30 / 30)
LZW codes identical and round trip    : 30 / 30
BCH codewords identical               : 20 / 20
BCH 1- and 2-error patterns corrected : 2400 / 2400
Hamming single errors corrected       : 112 / 112
(5,1) patterns behaving as predicted  : 32 / 32
```

`pyref_it.mat` contains codes, bit streams and codewords produced by the Python library for random inputs. The MATLAB functions reproduce all of them exactly, and correct all 2400 one- and two-error patterns of BCH(15,7).

Appendix C — Glossary

AEP	Asymptotic equipartition property: long sequences have probability $\approx 2^{-nH}$.
Anomaly detection	Flagging behaviour that is improbable under a model of normal behaviour.
Arithmetic coding	Coding a whole message as a number inside an interval of width $P(\text{message})$.
BCH code	Algebraic cyclic code correcting t errors; BCH(15,7) corrects 2.
BEC	Binary erasure channel; capacity $1-\epsilon$.
Blahut–Arimoto	Iterative algorithm for the capacity of any DMC.
BSC	Binary symmetric channel; capacity $1-H_2(p)$.
Capacity	Maximum mutual information per channel use; the limit of reliable rate.
Chain rule	$H(X,Y)=H(X)+H(Y\|X)$.
Code rate	$R=k/n$ data bits per code bit.
Compression ratio	Original size divided by compressed size.
Conditional entropy	Average remaining uncertainty $H(Y\|X)$.
Data processing inequality	Processing cannot increase information: $I(X;Z)\le I(X;Y)$.
DMC	Discrete memoryless channel $p(y\|x)$.
Efficiency	$\eta=H/L$ for a source code.
Entropy	$H(X)=-\sum p\log_2 p$: average uncertainty in bits.
Entropy rate	Information per symbol of a source with memory.
False negative / positive	Missed attack / false alarm.
GF(16)	Finite field with 16 elements used by BCH(15,7).
Hamming distance	Number of differing positions between two words.
Hamming code	Single-error-correcting code; (7,4) uses parity at positions 1, 2, 4.
Huffman code	Optimal symbol code built by merging the two least probable nodes.
Instantaneous code	Each codeword is decoded at its last bit (prefix-free).
KL divergence	$D(p\|\|q)$: extra bits for coding p with a code for q.
Kraft–McMillan	$\sum 2^{-\ell_i}\le 1$ for decodable codes.
Leakage	$H(S)-H(S\|O)=I(S;O)$: what an observation reveals about a secret.
LZW	Dictionary compressor (Lempel–Ziv–Welch).
MAC	Message authentication code: keyed integrity check.
Markov chain	Process where the next state depends only on the current state.
Min-entropy	$-\log_2\max p$: one-guess security.
Minimum distance	Smallest distance between two codewords.
Mutual information	$I(X;Y)=H(X)-H(X\|Y)$.

Prefix-free code	No codeword is a prefix of another.
Redundancy	$1 - \eta$; predictable, removable structure.
Repetition code	Each bit sent n times; (5,1) corrects 2 errors.
SECDED	Single error correction, double error detection (extended Hamming).
Self-information	$-\log_2 p(x)$: surprise of an outcome.
Shannon–Fano code	Prefix code built by balanced top-down splits.
Side channel	Unintended observable (time, power, size) that leaks secrets.
Stationary distribution	π with $\pi\mathbf{P} = \pi$.
Syndrome	$\mathbf{H}\mathbf{r}^T$; zero for codewords, identifies correctable errors.
Typical set	Sequences with probability close to 2^{-nH}.
Uniquely decodable	Every bit string has at most one parse.
Wiretap channel	Model with a legitimate receiver and a noisier eavesdropper; secrecy capacity $C_B - C_E$.

Appendix D — Bibliography

Textbooks

[1] F. Bavaud et al., *An Introduction to Information Theory and Applications* — the reference named in the course plan.

[2] T. M. Cover and J. A. Thomas, *Elements of Information Theory*, 2nd ed. Wiley, 2006.

[3] D. J. C. MacKay, *Information Theory, Inference, and Learning Algorithms*. Cambridge University Press, 2003.

[4] K. Sayood, *Introduction to Data Compression*, 5th ed. Morgan Kaufmann, 2017.

[5] S. Lin and D. J. Costello, *Error Control Coding*, 2nd ed. Pearson, 2004.

Foundational papers

[6] C. E. Shannon, "A mathematical theory of communication," *Bell System Technical Journal*, vol. 27, pp. 379–423, 623–656, 1948.

[7] R. M. Fano, "The transmission of information," Technical Report 65, Research Laboratory of Electronics, MIT, 1949.

[8] D. A. Huffman, "A method for the construction of minimum-redundancy codes," *Proc. IRE*, vol. 40, no. 9, pp. 1098–1101, 1952.

[9] J. Ziv and A. Lempel, "A universal algorithm for sequential data compression," *IEEE Trans. Inf. Theory*, vol. 23, no. 3, pp. 337–343, 1977.

[10] J. Ziv and A. Lempel, "Compression of individual sequences via variable-rate coding," *IEEE Trans. Inf. Theory*, vol. 24, no. 5, pp. 530–536, 1978.

[11] T. A. Welch, "A technique for high-performance data compression," *Computer*, vol. 17, no. 6, pp. 8–19, 1984.

[12] I. H. Witten, R. M. Neal and J. G. Cleary, "Arithmetic coding for data compression," *Communications of the ACM*, vol. 30, no. 6, pp. 520–540, 1987.

[13] R. W. Hamming, "Error detecting and error correcting codes," *Bell System Technical Journal*, vol. 29, no. 2, pp. 147–160, 1950.

[14] A. Hocquenghem, "Codes correcteurs d'erreurs," *Chiffres*, vol. 2, pp. 147–156, 1959.

[15] R. C. Bose and D. K. Ray-Chaudhuri, "On a class of error correcting binary group codes," *Information and Control*, vol. 3, no. 1, pp. 68–79, 1960.

[16] W. W. Peterson, "Encoding and error-correction procedures for the Bose–Chaudhuri codes," *IRE Trans. Inf. Theory*, vol. 6, no. 4, pp. 459–470, 1960.

[17] R. Blahut, "Computation of channel capacity and rate-distortion functions," *IEEE Trans. Inf. Theory*, vol. 18, no. 4, pp. 460–473, 1972.

[18] S. Arimoto, "An algorithm for computing the capacity of arbitrary discrete memoryless channels," *IEEE Trans. Inf. Theory*, vol. 18, no. 1, pp. 14–20, 1972.

[19] A. D. Wyner, "The wire-tap channel," *Bell System Technical Journal*, vol. 54, no. 8, pp. 1355–1387, 1975.

Security applications

[20] J. Kelsey, "Compression and information leakage of plaintext," in *Fast Software Encryption (FSE 2002)*, LNCS 2365, Springer, 2002.

[21] G. Smith, "On the foundations of quantitative information flow," in *FoSSaCS 2009*, LNCS 5504, Springer, 2009.

[22] W. Lee and D. Xiang, "Information-theoretic measures for anomaly detection," in *Proc. IEEE Symposium on Security and Privacy*, 2001.

[23] J. Rizzo and T. Duong, "The CRIME attack," presentation at Ekoparty Security Conference, 2012.

[24] Y. Gluck, N. Harris and A. Prado, "BREACH: Reviving the CRIME attack," presentation at Black Hat USA, 2013.